

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: THỰC TẬP CƠ SỞ
MÃ HỌC PHẦN: INT13187**

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG THƯƠNG MẠI
ĐIỆN TỬ KINH DOANH THỜI TRANG**

Sinh viên thực hiện:

Hoàng Minh Trọng B23DCCN847

Tên nhóm: 10

Giảng viên hướng dẫn: TS. Đỗ Thị Liên

HÀ NỘI 2026

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn sâu sắc đến Học viện Công nghệ Bru chính Viễn thông và Khoa Công nghệ Thông tin 1 đã đưa môn học Thực tập cơ sở vào chương trình giảng dạy. Đây là cơ hội quý báu để em có thể tiếp cận thực tế và vận dụng những kiến thức đã học vào xây dựng một sản phẩm cụ thể.

Đặc biệt, em xin bày tỏ lòng biết ơn chân thành đến giảng viên hướng dẫn cô Đỗ Thị Liên, người đã tận tình hướng dẫn, chỉ bảo và truyền đạt những kiến thức quý báu cho em trong suốt quá trình học tập và thực hiện đề tài.

Trong thời gian làm việc cùng cô, em không chỉ được củng cố kiến thức chuyên môn mà còn học hỏi được tinh thần làm việc nghiêm túc, hiệu quả và có trách nhiệm. Dưới sự hướng dẫn tận tình của cô, em đã hoàn thành việc xây dựng và phát triển một website thương mại điện tử, qua đó có cơ hội áp dụng kiến thức lý thuyết vào thực tế, nâng cao kỹ năng và sự tự tin trong quá trình học tập cũng như định hướng nghề nghiệp sau này.

Một lần nữa, em xin gửi tới cô lời cảm ơn chân thành và sâu sắc nhất vì sự tận tâm và những đóng góp quý báu trong suốt thời gian qua. Kính chúc cô luôn mạnh khỏe, hạnh phúc và đạt được nhiều thành công trong sự nghiệp giảng dạy cũng như trong cuộc sống.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	2
MỤC LỤC.....	3
DANH MỤC CÁC HÌNH VẼ.....	6
DANH MỤC CÁC BẢNG BIỂU	6
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT	8
CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN VÀ LĨNH VỰC NGHIÊN CỨU.....	9
1.1 Bối cảnh và lí do chọn đề tài	9
1.2 Mục tiêu của đề tài	9
1.3 Phạm vi đề tài.....	9
1.3.1 Phạm vi hệ thống.....	9
1.3.2 Phạm vi giới hạn.....	10
1.4 Phương pháp nghiên cứu.....	10
1.5 Xác định yêu cầu hệ thống	10
1.5.1 Yêu cầu chức năng	10
1.5.2 Yêu cầu phi chức năng	11
1.6 Khảo sát các hệ thống liên quan.....	12
1.7 Thiết kế tương tác cho sản phẩm.....	13
1.8 Tổng kết chương	13
CHƯƠNG 2. NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ.....	15
2.1 Mô hình tổng quát hệ thống	15
2.1.1 Tổng quan kiến trúc.....	15
2.1.2 Sơ đồ hệ thống.....	15
2.1.3 Mô tả luồng dữ liệu (Data Flow).....	15
2.2 Phương pháp tiếp cận và xây dựng hệ thống	17
2.2.1 Phân tích yêu cầu và Định hướng giải quyết bài toán.....	17
2.2.2 Quy trình xây dựng hệ thống.....	17
2.3 Mô hình phát triển phần mềm	18
2.4 Kiến trúc phần mềm	19
2.4.1 Kiến trúc Client – Server.....	19
2.4.2 Kiến trúc RESTful API	19
2.4.3 Kiến trúc phân tầng (3-tier).....	20
2.5 Lựa chọn công nghệ	20
2.5.1 Tiêu chí lựa chọn.....	20

2.5.2 Các công nghệ sử dụng	20
2.6 Bảo mật hệ thống.....	21
2.6.1 Xác thực và phân quyền	21
2.6.2 Bảo vệ dữ liệu	21
2.6.3 Kiểm soát đầu vào	22
2.7 Tổng kết chương	22
CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ VÀ THỰC NGHIỆM HỆ THỐNG	23
3.1 Phân tích hệ thống.....	23
3.1.1 Xác định tác nhân (Actors).....	23
3.1.2 Biểu đồ Use Case tổng quát	23
3.1.3 Xây dựng Use Case Diagram	25
3.1.4 Đặc tả Use Case (Use Case Specification)	25
3.1.5 Biểu đồ trình tự (Sequence Diagram).....	42
3.2 Thiết kế hệ thống.....	59
3.2.1 Thiết kế kiến trúc tổng thể.....	59
3.2.2 Thiết kế sơ đồ lớp (Class Diagram)	60
3.2.3 Thiết kế cơ sở dữ liệu (Database Design)	61
3.2.4 Thiết kế API (RESTful API)	64
3.2.5 Thiết kế giao diện (UI/UX)	64
CHƯƠNG 4. TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	76
4.1 Môi trường triển khai hệ thống	76
4.1.1 Môi trường phần cứng.....	76
4.1.2 Môi trường phần mềm.....	76
4.1.3 Công cụ hỗ trợ.....	76
4.2 Triển khai hệ thống	76
4.2.1 Cấu hình biến môi trường (.env)	76
4.2.2 Cấu hình Cơ sở dữ liệu đám mây (MongoDB Atlas).....	77
4.2.3 Cấu hình Dịch vụ lưu trữ hình ảnh (Cloudinary)	77
4.2.4 Triển khai phân hệ Backend (Máy chủ API).....	77
4.2.5 Triển khai phân hệ Frontend (Giao diện Khách hàng).....	78
4.2.6 Triển khai phân hệ Admin (Giao diện Quản trị viên)	78
4.3 Kiểm thử hệ thống (System Testing)	78
4.3.1 Kiểm thử API (Kiểm thử độc lập với Postman).....	79
4.3.2 Kiểm thử chức năng (Functional Testing)	80
4.3.3 Kiểm thử các trường hợp đặc biệt (Edge Cases).....	80

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	82
5.1 Kết luận	82
5.2 Hạn chế của hệ thống	82
5.3 Hướng phát triển trong tương lai.....	83
TÀI LIỆU THAM KHẢO.....	84
PHỤ LỤC CÀI ĐẶT VÀ TRIỂN KHAI	85
CÁC SẢN PHẨM KÈM THEO.....	86

DANH MỤC CÁC HÌNH VẼ

Hình 1 - Biểu đồ Use Case tổng quát	24
Hình 2 - Sequence Diagram 1 - Xem trang chủ	43
Hình 3 - Sequence Diagram 2 - Khám phá sản phẩm	44
Hình 4 - Sequence Diagram 3 - Tìm kiếm và lọc sản phẩm	44
Hình 5 - Sequence Diagram 4 - Xem chi tiết sản phẩm	45
Hình 6 - Sequence Diagram 5 - Đăng ký nhận bản tin	46
Hình 7 - Sequence Diagram 6 - Đăng ký tài khoản	47
Hình 8 - Sequence Diagram 7 - Đăng nhập hệ thống	48
Hình 9 - Sequence Diagram 8 - Đăng xuất	49
Hình 10 - Sequence Diagram 9 - Quản lý giỏ hàng	50
Hình 11 - Sequence Diagram 10 - Đặt hàng và Thanh toán	51
Hình 12 - Sequence Diagram 11 - Xem lịch sử đơn hàng	52
Hình 13 - Sequence Diagram 12 - Quản trị viên	53
Hình 14 - Sequence Diagram 13 - Xem Bảng thống kê	54
Hình 15 - Sequence Diagram 14 - Quản lý Danh mục	55
Hình 16 - Sequence Diagram 15 - Quản lý Sản phẩm và Nhập kho	56
Hình 17 - Sequence Diagram 16 - Quản lý Đơn hàng & Tự động hoàn kho	57
Hình 18 - Sequence Diagram 17 - Quản lý Người dùng & Xóa Cascade	59
Hình 19 - Sơ đồ Kiến trúc tổng quan hệ thống	60
Hình 20 - Cấu trúc sơ đồ lớp	61
Hình 21 - Sơ đồ Cấu trúc MongoDB	62
Hình 22 - Giao diện đăng nhập	67
Hình 23 - Giao diện đăng ký	68
Hình 24 - Giao diện trang chủ	68
Hình 25 - Giao diện danh mục sản phẩm	69
Hình 26 - Giao diện chi tiết sản phẩm	69
Hình 27 - Giao diện giỏ hàng	70
Hình 28 - Giao diện thanh toán	71
Hình 29 - Giao diện lịch sử đơn hàng	71
Hình 30 - Giao diện tổng quan	72
Hình 31 - Giao diện thêm sản phẩm	73
Hình 32 - Giao diện danh sách sản phẩm	74
Hình 33 - Giao diện quản lý đơn hàng	74
Hình 34 - Giao diện quản lý người dùng	75
Hình 35 - Ví dụ test thử API Product-Add bằng Postman	79

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Đặc tả Use Case 1 - Xem trang chủ	26
Bảng 2. Đặc tả Use Case 2 - Khám phá sản phẩm	27
Bảng 3. Use Case 3 - Tìm kiếm và lọc sản phẩm	27

Bảng 4. Đặc tả Use Case 4 - Xem chi tiết sản phẩm.....	28
Bảng 5. Đặc tả Use Case 5 - Đăng ký nhận bản tin	29
Bảng 6. Đặc tả Use Case 6 - Đăng ký tài khoản	30
Bảng 7. Đặc tả Use Case 7 - Đăng nhập	31
Bảng 8. Đặc tả Use Case 8 - Đăng xuất	32
Bảng 9. Đặc tả Use Case 9 - Quản lý giỏ hàng	33
Bảng 10. Đặc tả Use Case 10 - Đặt hàng và Thanh toán	34
Bảng 11. Đặc tả Use Case 11 - Xem lịch sử đơn hàng	35
Bảng 12. Đặc tả Use Case 12 - Xác thực Quản trị	36
Bảng 13. Đặc tả Use Case 13 - Xem Bảng thống kê.....	37
Bảng 14. Đặc tả Use Case 14 - Quản lý Danh mục	38
Bảng 15. Đặc tả Use Case 15 - Quản lý Sản phẩm	39
Bảng 16. Đặc tả Use Case 16 - Quản lý Đơn hàng	40
Bảng 17. Đặc tả Use Case 17 - Quản lý Người dùng.....	41

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
API	Application Programming Interface	Giao diện lập trình ứng dụng
BSON	Binary JSON	Định dạng lưu trữ dữ liệu nhị phân của MongoDB
CDN	Content Delivery Network	Mạng phân phối nội dung
COD	Cash on Delivery	Thanh toán khi nhận hàng
CSDL	Database	Cơ sở dữ liệu
ERD	Entity-Relationship Diagram	Sơ đồ thực thể liên kết
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản nhẹ để trao đổi dữ liệu
JWT	JSON Web Token	Tiêu chuẩn mã hóa chuỗi thông tin xác thực an toàn
MERN	MongoDB, Express, React, Node.js	Ngăn xếp công nghệ phát triển web Fullstack
MoM	Month-over-Month	Tốc độ tăng trưởng theo tháng
ODM	Object Data Modeling	Kỹ thuật mô hình hóa dữ liệu đối tượng
REST	Representational State Transfer	Kiến trúc phần mềm cho các hệ thống phân tán
SPA	Single Page Application	Ứng dụng web trang đơn (không tải lại trang)
UI	User Interface	Giao diện người dùng
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất
URL	Uniform Resource Locator	Đường dẫn tài nguyên trên Internet
UX	User Experience	Trải nghiệm người dùng

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN VÀ LĨNH VỰC NGHIÊN CỨU

1.1 Bối cảnh và lí do chọn đề tài

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, thương mại điện tử đã trở thành xu hướng tất yếu, giúp doanh nghiệp mở rộng thị trường và tối ưu hóa hoạt động kinh doanh. Tuy nhiên, nhiều hệ thống bán hàng hiện nay vẫn tồn tại những hạn chế như: quản lý thủ công, dữ liệu phân tán, thiếu khả năng đồng bộ giữa các thành phần, và chưa tối ưu trải nghiệm người dùng trên nhiều thiết bị.

Bên cạnh đó, việc xây dựng một hệ thống thương mại điện tử hoàn chỉnh không chỉ dừng lại ở giao diện mua sắm mà còn đòi hỏi sự tích hợp chặt chẽ giữa các chức năng như quản lý sản phẩm, giỏ hàng, đơn hàng, người dùng, cũng như hệ thống quản trị và phân tích dữ liệu. Nhiều giải pháp hiện có thường thiếu tính linh hoạt, khó mở rộng hoặc không đáp ứng tốt nhu cầu thực tế của doanh nghiệp vừa và nhỏ.

Xuất phát từ những vấn đề trên, đề tài “Infinity E-Commerce” được thực hiện nhằm xây dựng một nền tảng thương mại điện tử hiện đại, đầy đủ chức năng, đảm bảo tính mở rộng và hiệu năng cao. Hệ thống được phát triển theo kiến trúc fullstack MERN (MongoDB, Express, React, Node.js) với mô hình monorepo, bao gồm ba thành phần chính: giao diện người dùng (Frontend), hệ thống quản trị (Admin) và máy chủ xử lý (Backend API).

1.2 Mục tiêu của đề tài

Dự án hướng tới việc xây dựng hoàn chỉnh một hệ thống thương mại điện tử Fullstack hoạt động ổn định, mượt mà và an toàn. Các mục tiêu cụ thể bao gồm:

- Xây dựng giao diện mua sắm trực quan, thân thiện, tập trung tối ưu hóa trải nghiệm tìm kiếm và đặt hàng cho người dùng.
- Phát triển hệ thống quản trị (Admin Dashboard) mạnh mẽ, cung cấp các chỉ số thống kê tức thời giúp kiểm soát tình hình kinh doanh.
- Đảm bảo tính bảo mật thông qua cơ chế xác thực JWT và phân quyền người dùng.
- Số hóa các luồng nghiệp vụ thực tế: Quản lý số lượng tồn kho theo thời gian thực, tự động hoàn kho khi hủy đơn, và triển khai chiến dịch marketing (Bản tin & Mã giảm giá).
- Xây dựng kiến trúc hệ thống hiện đại, dễ dàng nâng cấp, bảo trì và mở rộng trong tương lai.
- Tối ưu hóa hiệu năng xử lý dữ liệu lớn bằng cách áp dụng các kỹ thuật truy vấn nâng cao (Aggregation Pipeline) tại phía máy chủ, hạn chế lưu trữ dữ liệu dư thừa.
- Nâng cao trải nghiệm thanh toán của khách hàng thông qua việc tích hợp các API bên thứ ba (địa chỉ hành chính) để tự động hóa quá trình nhập liệu.

1.3 Phạm vi đề tài

1.3.1 Phạm vi hệ thống

- Ứng dụng Frontend dành cho khách hàng với các tính năng như xem, tìm kiếm sản phẩm, giỏ hàng, thanh toán, quản lý đơn hàng.
- Ứng dụng Admin dành cho ban quản trị để quản lý vòng đời sản phẩm, danh mục, đơn hàng, xem thống kê
- Hệ thống Backend API phục vụ xử lý logic nghiệp vụ, xác thực, giao tiếp với cơ sở dữ liệu

1.3.2 Phạm vi giới hạn

- Trong giai đoạn này, hệ thống tập trung vào luồng xử lý đơn hàng và tính toán tổng tiền, chưa tích hợp trực tiếp các cổng thanh toán quốc tế phức tạp (như PayPal, Stripe) hay các đơn vị vận chuyển bên thứ ba.
- Chưa áp dụng các thuật toán AI phức tạp cho hệ thống gợi ý sản phẩm

1.4 Phương pháp nghiên cứu

Để đạt được mục tiêu đề ra, đề tài áp dụng các phương pháp luận sau:

- Nghiên cứu lý thuyết: Tổng hợp kiến thức về mô hình Client-Server, kiến trúc RESTful API, và cơ sở dữ liệu phiếm chỉ (NoSQL).
- Thiết kế hướng người dùng (User-Centered Design): Khảo sát hành vi mua sắm trực tuyến để xây dựng luồng thao tác ngắn gọn, giảm thiểu số bước từ lúc chọn đồ đến lúc thanh toán.
- Phương pháp phát triển phần mềm Agile: Chia nhỏ dự án thành các module độc lập (User, Product, Cart, Order, Dashboard) để dễ dàng kiểm thử và tích hợp liên tục.
- Tổ chức mã nguồn Monorepo: Quản lý tập trung cả Frontend, Admin và Backend trong cùng một kho lưu trữ, giúp đồng bộ hóa các phiên bản thư viện và dễ dàng triển khai.

1.5 Xác định yêu cầu hệ thống

Đây là nền tảng cốt lõi định hình toàn bộ kiến trúc và tính năng của hệ thống.

1.5.1 Yêu cầu chức năng

- Khối chức năng dành cho Khách hàng:
 - Quản lý tài khoản: Đăng ký, đăng nhập và bảo mật thông tin cá nhân.
 - Khám phá sản phẩm: Duyệt sản phẩm theo danh mục, tìm kiếm, xem chi tiết (kèm nhiều góc độ ảnh) và hệ thống hiển thị top 5 sản phẩm bán chạy nhất (Best Sellers) một cách tự động và linh hoạt dựa trên dữ liệu doanh số thực tế từ API
 - Quản lý mua hàng: Thao tác trên giỏ hàng (thêm, cập nhật số lượng, xóa). Đặt hàng với tính năng kiểm tra số lượng tồn kho thực tế của từng kích cỡ (size).
 - Hệ thống Khuyến mãi: Cho phép đăng ký nhận bản tin (Newsletter) để nhận mã voucher đặc quyền BARCA20 (giảm 20% giá trị đơn hàng). Hệ thống phải áp dụng

cơ chế xác thực 2 lớp (validate 2 lớp: khi áp mã và khi chốt đơn) để đảm bảo mã giảm giá hợp lệ và mỗi tài khoản chỉ được áp dụng một lần duy nhất.

- Thanh toán và Giao hàng thông minh: Xây dựng form địa chỉ giao hàng với hiệu ứng dropdown xếp tầng (cascading dropdown) 2 cấp, tích hợp trực tiếp Vietnam Provinces API v2. Khi người dùng chọn Tỉnh/Thành phố, danh sách Phường/Xã sẽ tự động tải theo thời gian thực, giúp hạn chế sai sót khi nhập liệu.
- Lịch sử giao dịch: Tra cứu và theo dõi trạng thái các đơn hàng đã đặt.
- Khối chức năng dành cho Quản trị viên (Admin):
 - Xác thực độc lập: Đăng nhập an toàn vào bảng điều khiển qua một cổng (port) riêng biệt.
 - Bảng thống kê (Dashboard): Hiển thị tổng quan các chỉ số trọng yếu: Tổng doanh thu, số lượng đơn hàng, người dùng mới, tốc độ tăng trưởng hàng tháng (MoM). Tích hợp thư viện Recharts để vẽ biểu đồ trực quan hóa doanh thu 12 tháng trong năm (chỉ tính các đơn hàng đã giao thành công - Delivered). Cung cấp cái nhìn trực quan qua biểu đồ phân bố trạng thái... Cung cấp cái nhìn trực quan qua biểu đồ phân bố trạng thái đơn hàng, liệt kê 10 đơn hàng gần nhất, thống kê top 5 sản phẩm bán chạy nhất và cảnh báo chính xác sản phẩm sắp hết hàng dựa trên tồn kho của từng biến thể kích cỡ (variant size < 10).
 - Quản lý Sản phẩm: Thêm mới, chỉnh sửa thông tin, xóa sản phẩm. Hỗ trợ tính năng Nhập hàng (Restock) trực tiếp.
 - Quản lý danh mục (Category Management): Quản lý phân loại sản phẩm. Đặc biệt, danh sách danh mục được tự động sắp xếp theo mức độ đóng góp doanh thu. Quá trình này được tính toán động (on-the-fly) thông qua MongoDB Aggregation Pipeline từ các đơn hàng thành công, tuyệt đối không lưu trường doanh thu dư thừa vào cơ sở dữ liệu.
 - Quản lý luồng đơn hàng: Xem danh sách đơn, cập nhật trạng thái thực tế. Khi một đơn hàng bị hủy, hệ thống phải tự động tính toán và hoàn trả chính xác số lượng sản phẩm vào kho.
 - Quản lý người dùng (User Management): Quản trị viên có quyền xem danh sách tài khoản, chủ động khóa/mở khóa (Active/Blocked) các tài khoản có dấu hiệu bất thường. Đặc biệt, hệ thống yêu cầu tính năng xóa tài khoản theo cơ chế Cascade Delete (khi xóa một người dùng, toàn bộ lịch sử đơn hàng và thông tin đăng ký bản tin liên quan phải được tự động dọn dẹp để đảm bảo tính toàn vẹn dữ liệu).

1.5.2 Yêu cầu phi chức năng

- Bảo mật: Mã hóa thông tin nhạy cảm. Phân tách quyền truy cập rõ ràng giữa API của User và Admin thông qua các Middleware.

- Hiệu suất & Tốc độ: Ứng dụng phải hoạt động mượt mà dưới dạng Single Page Application (SPA), không tải lại trang khi chuyển hướng. API phản hồi dưới 500ms đối với các tác vụ thông thường.
- Trải nghiệm người dùng: Giao diện Responsive 100%, tự động thích ứng với kích thước màn hình điện thoại, máy tính bảng và máy tính bàn. Cung cấp phản hồi thao tác tức thì qua hệ thống Toast notifications.
- Tính toàn vẹn dữ liệu (Data Integrity) & Chống gian lận: Mọi tính toán về giá cả (tổng tiền đơn hàng, áp dụng voucher) bắt buộc phải được thực hiện ở phía Server (dựa trên dữ liệu gốc từ Database), tuyệt đối không tin tưởng dữ liệu giá gửi lên từ Client. Nghiệp vụ trừ kho phải được xử lý nguyên tử (Atomic) thông qua MongoDB \$inc để tránh lỗi tương tranh (Race condition). Các trạng thái của đơn hàng và người dùng phải được kiểm soát chặt chẽ bằng Enum Validation.
- Tối ưu hóa truy vấn và Phân tải hệ thống: Đối với các chức năng quản trị có lượng dữ liệu lớn (như danh sách người dùng), hệ thống không tải toàn bộ dữ liệu về Client mà bắt buộc xử lý tìm kiếm, tính toán (tổng chi tiêu) và phân trang ngay tại Database thông qua các luồng Aggregation phức tạp, giúp tiết kiệm băng thông và bộ nhớ.

1.6 Khảo sát các hệ thống liên quan

Qua khảo sát các hệ thống thương mại điện tử hiện nay, từ các nền tảng lớn như Shopee, Lazada đến các website được xây dựng trên nền tảng WordPress, Shopify, có thể rút ra một số nhận định như sau:

- Các nền tảng thương mại điện tử lớn cung cấp hệ thống chức năng mạnh mẽ và toàn diện, đáp ứng được nhu cầu của nhiều đối tượng người dùng. Tuy nhiên, trải nghiệm người dùng thường mang tính đồng nhất, khiến các cửa hàng khó thể hiện được bản sắc thương hiệu riêng.
- Đối với các website thương mại điện tử quy mô nhỏ hoặc tự phát triển, mặc dù có tính linh hoạt cao, nhưng thường chưa đảm bảo tính đồng bộ trong quản lý hệ thống. Cụ thể, nhiều hệ thống chưa xử lý tốt bài toán quản lý tồn kho theo thời gian thực, dẫn đến tình trạng bán vượt số lượng thực tế.
- Ngoài ra, các cơ chế khuyến mãi và mã giảm giá (voucher) trong nhiều hệ thống còn thiếu ràng buộc chặt chẽ, dễ bị lợi dụng, ảnh hưởng đến hiệu quả kinh doanh và tính công bằng giữa người dùng.

Từ những hạn chế trên, hệ thống Infinity E-Commerce được xây dựng nhằm khắc phục các điểm yếu của các hệ thống hiện có, thông qua các giải pháp sau:

- Tự chủ trong việc phát triển và quản lý mã nguồn, cho phép tùy biến linh hoạt và tối ưu hóa theo nhu cầu thực tế
- Tích hợp chặt chẽ cơ chế kiểm tra và cập nhật tồn kho ngay tại bước đặt hàng, đảm bảo tính chính xác của dữ liệu

- Xây dựng hệ thống quản lý voucher với ràng buộc theo tài khoản người dùng, hạn chế việc lạm dụng và nâng cao hiệu quả sử dụng

1.7 Thiết kế tương tác cho sản phẩm

Hệ thống được thiết kế theo nguyên tắc đề cao tính tinh gọn và hiệu quả trong thao tác, nhằm tối ưu trải nghiệm cho cả người dùng và quản trị viên.

- Đối với khách hàng:
 - Hành trình mua sắm được tối giản, rút ngắn số bước từ trang chủ đến khi hoàn tất đơn hàng
 - Chức năng đăng ký nhận bản tin (newsletter) được bố trí nổi bật và tích hợp liền mạch trong giao diện, góp phần thúc đẩy hành vi mua hàng thông qua cơ chế cung cấp mã giảm giá
 - Hệ thống áp dụng cơ chế kiểm tra dữ liệu đầu vào theo thời gian thực (*inline validation*), cho phép phát hiện và hiển thị lỗi ngay tại vị trí nhập liệu (ví dụ: hết hàng, mã giảm giá không hợp lệ), giúp giảm thiểu thao tác và nâng cao trải nghiệm người dùng
 - Form điền thông tin tối ưu (Smart Form): Các trường thông tin địa chỉ tại bước thanh toán được thiết kế liên kết chặt chẽ (cascading). Việc gọi API lấy dữ liệu Phường/Xã diễn ra ngầm và hiển thị tức thì, loại bỏ cảm giác chờ đợi.
- Đối với quản trị viên:
 - Giao diện quản trị được thiết kế theo mô hình Single-page Dashboard, giúp giảm thiểu việc chuyển trang và tăng hiệu quả thao tác
 - Các chức năng quản lý như bổ sung tồn kho (*restock*) hoặc cập nhật trạng thái đơn hàng được thực hiện trực tiếp trên bảng dữ liệu thông qua các thao tác nhanh
 - Thiết kế này giúp tối ưu quy trình quản lý, giảm thời gian xử lý và hạn chế sai sót trong quá trình vận hành hệ thống
 - Trải nghiệm tìm kiếm mượt mà (Debounce Search): Thanh tìm kiếm người dùng được áp dụng kỹ thuật Debounce (độ trễ 400ms). Quản trị viên có thể gõ phím liên tục mà không làm hệ thống gửi hàng loạt request thừa thãi lên máy chủ, tránh giật lag giao diện.

1.8 Tổng kết chương

Chương này đã trình bày tổng quan về dự án **INFINITY E-COMMERCE**, từ việc phân tích bối cảnh và lý do hình thành đề tài cho đến việc xác định rõ mục tiêu, phạm vi nghiên cứu cũng như các phương pháp và công nghệ được sử dụng trong quá trình phát triển hệ thống.

Bên cạnh đó, chương đã tiến hành khảo sát các hệ thống thương mại điện tử hiện có, từ đó rút ra những ưu điểm và hạn chế làm cơ sở định hướng cho việc xây dựng hệ thống. Việc

phân tích này giúp đề tài xác định được các vấn đề thực tiễn cần giải quyết, đặc biệt là trong việc quản lý tồn kho, kiểm soát voucher và tối ưu trải nghiệm người dùng.

Đặc biệt, chương đã hệ thống hóa một cách chi tiết các yêu cầu của hệ thống, bao gồm yêu cầu chức năng (quản lý sản phẩm, giỏ hàng, đơn hàng, tồn kho, voucher, dashboard, ...) và yêu cầu phi chức năng (hiệu năng, bảo mật, khả năng mở rộng, tính khả dụng, ...). Đồng thời, thiết kế tương tác cho sản phẩm cũng đã được trình bày nhằm làm rõ cách thức người dùng và quản trị viên tương tác với hệ thống trong thực tế.

Những nội dung được trình bày trong chương này đóng vai trò là nền tảng quan trọng, làm cơ sở cho việc phân tích, thiết kế kiến trúc hệ thống và xây dựng cơ sở dữ liệu trong các chương tiếp theo.

CHƯƠNG 2. NGHIÊN CỨU PHƯƠNG PHÁP TIẾP CẬN VÀ GIẢI QUYẾT VẤN ĐỀ

2.1 Mô hình tổng quát hệ thống

2.1.1 Tổng quan kiến trúc

Hệ thống INFINITY E-COMMERCE được xây dựng theo kiến trúc Monorepo, chia tách làm ba phân hệ hoạt động độc lập nhưng gắn kết chặt chẽ thông qua giao thức mạng:

- Frontend (Port 5173): Ứng dụng phía khách hàng (Client-side), đảm nhiệm hiển thị giao diện mua sắm trực quan, quản lý trạng thái giỏ hàng và thu thập thông tin đặt hàng.
- Admin (Port 5174): Bảng điều khiển quản trị (Dashboard), cung cấp giao diện để ban quản trị theo dõi doanh thu, quản lý sản phẩm, kho hàng, kiểm duyệt trạng thái đơn hàng và tài khoản người dùng.
- Backend API (Port 4000): Máy chủ trung tâm (Server-side) đóng vai trò cốt lõi. Nơi đây tiếp nhận yêu cầu, xử lý toàn bộ logic nghiệp vụ (tính toán, trừ kho, đối chiếu voucher) và tương tác trực tiếp với cơ sở dữ liệu.

2.1.2 Sơ đồ hệ thống

Hệ thống vận hành theo luồng tương tác Client - Server tiêu chuẩn:

- Lớp Giao diện (React/Vite): Người dùng tương tác trên trình duyệt. Các hành động (thêm giỏ hàng, đặt hàng) được đóng gói thành các HTTP Request thông qua thư viện Axios gửi lên Server.
- Lớp Xử lý (Node.js/Express): Request đi qua các lớp bảo vệ (Middleware) để xác thực. Nếu hợp lệ, Controller sẽ giải quyết logic và tính toán số liệu.
- Lớp Dữ liệu (MongoDB & Cloudinary): Server truy vấn MongoDB (thông qua Mongoose ODM) để đọc/ghi dữ liệu có cấu trúc. Các tệp tin đa phương tiện (ảnh sản phẩm) được upload lên lưu trữ đám mây Cloudinary, Server chỉ lưu trữ đường dẫn URL để tối ưu băng thông.

2.1.3 Mô tả luồng dữ liệu (Data Flow)

Để đảm bảo tính toàn vẹn dữ liệu, độ chính xác trong xử lý nghiệp vụ và hạn chế các hành vi gian lận, hệ thống INFINITY E-COMMERCE được thiết kế với các luồng dữ liệu trọng yếu như sau:

- Luồng xử lý đặt hàng và tính giá phía Server (Server-side Pricing)

Khi người dùng thực hiện thao tác thanh toán, phía Client chỉ gửi thông tin sản phẩm, số lượng và mã voucher (nếu có) lên hệ thống. Backend sẽ không sử dụng trực tiếp dữ liệu tổng tiền từ Client, mà thực hiện các bước xử lý như sau:

- Truy xuất giá gốc của từng sản phẩm từ cơ sở dữ liệu MongoDB

- Kiểm tra tính hợp lệ của mã voucher thông qua cơ chế xác thực hai lớp (khi áp dụng và khi đặt hàng)
- Tính toán lại tổng giá trị đơn hàng hoàn toàn tại phía Server
- Trả về kết quả cuối cùng cho Client

Cơ chế này giúp ngăn chặn hiệu quả hành vi can thiệp thay đổi giá từ phía người dùng (Price Tampering), đảm bảo tính chính xác và minh bạch trong giao dịch.

▪ Luồng quản lý tồn kho nguyên tử (Atomic Inventory Update)

Để tránh các vấn đề về xung đột dữ liệu (Race Condition) khi nhiều người dùng đặt hàng đồng thời, hệ thống không sử dụng phương pháp đọc–ghi thông thường. Thay vào đó, quá trình cập nhật tồn kho được thực hiện theo cơ chế nguyên tử:

- Khi phát sinh đơn hàng, hệ thống sử dụng toán tử \$inc của MongoDB để trừ số lượng tồn kho
- Điều kiện thực hiện: số lượng tồn kho của biến thể sản phẩm (ví dụ: size) phải lớn hơn hoặc bằng số lượng đặt
- Nếu điều kiện không thỏa mãn, thao tác bị từ chối và đơn hàng không được tạo (rollback logic)

Cách tiếp cận này giúp đảm bảo dữ liệu tồn kho luôn chính xác và ngăn chặn tình trạng bán vượt số lượng thực tế (Overselling).

▪ Luồng xoá dữ liệu liên kết (Cascade Delete)

Khi quản trị viên thực hiện thao tác xoá người dùng, hệ thống sẽ kích hoạt cơ chế xử lý dữ liệu liên quan nhằm đảm bảo tính nhất quán:

- Tự động tìm và xoá toàn bộ các bản ghi liên quan đến người dùng đó, bao gồm:
- Lịch sử đơn hàng
- Dữ liệu đăng ký nhận bản tin (Newsletter)
- Đảm bảo không tồn tại các bản ghi không còn liên kết (Orphan Data) trong hệ thống

Cơ chế này giúp duy trì sự toàn vẹn dữ liệu và tối ưu hóa hiệu năng truy vấn của hệ thống.

▪ Luồng tổng hợp và xử lý truy vấn dữ liệu lớn (Aggregation Pipeline)

Đối với các nghiệp vụ yêu cầu thống kê phức tạp (như tính tổng chi tiêu của từng người dùng, hay sắp xếp danh mục sản phẩm theo doanh thu thực tế), hệ thống tuyệt đối không tải toàn bộ dữ liệu về Client để tính toán nhằm tránh quá tải bộ nhớ. Thay vào đó, dữ liệu được xử lý trực tiếp tại tầng Database:

- Sử dụng MongoDB Aggregation Pipeline kết hợp các toán tử nâng cao (\$lookup để nối bảng, \$facet để xử lý đa luồng, \$regex để tìm kiếm).

- Tính toán doanh thu và tổng chi tiêu theo thời gian thực (on-the-fly) chỉ dựa trên các đơn hàng có trạng thái "Delivered".
- Phân trang dữ liệu trực tiếp tại máy chủ (Server-side Pagination).

Cơ chế này giúp hệ thống không cần lưu trữ các trường dữ liệu dư thừa (như trường tổng doanh thu trong bảng Category), từ đó tránh được rủi ro sai lệch dữ liệu khi có cập nhật, đồng thời tối ưu hóa băng thông mạng.

2.2 Phương pháp tiếp cận và xây dựng hệ thống

2.2.1 Phân tích yêu cầu và Định hướng giải quyết bài toán

Việc giải quyết bài toán được tiếp cận từ việc phân tích sâu sát nhu cầu thực tiễn của hai nhóm đối tượng cốt lõi. Đối với khách hàng, hệ thống cần cung cấp một trải nghiệm mua sắm mượt mà, không tải lại trang (Single Page Application - SPA), tích hợp bộ lọc tìm kiếm theo thời gian thực, quy trình thanh toán rõ ràng và hỗ trợ luồng áp dụng mã giảm giá liền mạch. Đối với nhà quản trị, yêu cầu đặt ra là một bảng điều khiển (Dashboard) cung cấp dữ liệu thống kê chính xác kèm phân tích tăng trưởng MoM, khả năng cập nhật trạng thái đơn hàng nhanh chóng với cơ chế tự động hoàn kho, cảnh báo tồn kho tự động và hệ thống quản lý tài khoản người dùng an toàn với trải nghiệm tìm kiếm mượt mà, không giật lag nhờ kỹ thuật Debounce (độ trễ 400ms) giúp giảm tải số lượng Request gửi lên máy chủ.

Dựa trên những phân tích này, hệ thống được định hướng phát triển theo mô hình Client-Server kết hợp chuẩn giao tiếp RESTful API nhằm tách bạch rõ ràng giữa giao diện hiển thị và logic xử lý. Đặc biệt ở tầng dữ liệu, thay vì sử dụng hệ quản trị cơ sở dữ liệu quan hệ (SQL) truyền thống, giải pháp cơ sở dữ liệu phi cấu trúc NoSQL (MongoDB) đã được ưu tiên lựa chọn. Quyết định này mang lại những lợi thế kỹ thuật vượt trội, cực kỳ phù hợp với bối cảnh đặc thù của một hệ thống thương mại điện tử:

- Cấu trúc dữ liệu linh hoạt: Cho phép lưu trữ tự nhiên các thuộc tính đa dạng và dễ biến đổi của sản phẩm (ví dụ: nhiều biến thể về kích cỡ, màu sắc, hay sử dụng mảng `sizesStock` để quản lý số lượng tồn kho theo từng size chuyên biệt).
- Tối ưu hóa thiết kế: Giảm thiểu đáng kể độ phức tạp trong việc thiết kế sơ đồ quan hệ, loại bỏ sự phụ thuộc vào các bảng trung gian (Join tables) cồng kềnh.
- Hiệu năng và khả năng mở rộng: Tăng cường tốc độ truy vấn đối với các khối dữ liệu phân tán. Việc lựa chọn MongoDB không chỉ giải quyết trọn vẹn bài toán kỹ thuật hiện tại mà còn tạo nền tảng tối ưu cho khả năng mở rộng quy mô (Scalability) của hệ thống trong tương lai.

2.2.2 Quy trình xây dựng hệ thống

Hệ thống được phát triển theo phương pháp tiếp cận Bottom-Up, bắt đầu từ tầng dữ liệu cốt lõi và dần hoàn thiện lên các tầng xử lý và giao diện. Cách tiếp cận này giúp đảm bảo tính ổn định của nền tảng dữ liệu và logic nghiệp vụ ngay từ đầu, từ đó giảm thiểu sai sót trong các giai đoạn phát triển sau.

Quy trình xây dựng hệ thống được thực hiện qua các bước sau:

- Thiết kế dữ liệu (Database Design): Xây dựng các mô hình thực thể (Schema) tại tầng cơ sở dữ liệu bằng MongoDB, bao gồm các thực thể chính như User, Product, Order, Category. Các cấu trúc dữ liệu được thiết kế theo hướng linh hoạt, hỗ trợ mở rộng và tối ưu cho truy vấn.
- Phát triển tầng xử lý (Backend Development): Xây dựng các API theo chuẩn RESTful bằng Node.js và Express. Thiết lập các Middleware để xử lý xác thực (JWT), phân quyền (Admin/User) và kiểm soát dữ liệu đầu vào. Các Controller đảm nhiệm việc xử lý logic nghiệp vụ như quản lý đơn hàng, kiểm tra tồn kho, áp dụng voucher.
- Phát triển tầng giao diện (Frontend/Admin Development): Xây dựng giao diện người dùng và bảng quản trị bằng React. Các component được tổ chức theo hướng tái sử dụng, kết nối với Backend thông qua Axios. Trạng thái ứng dụng được quản lý tập trung nhằm đảm bảo luồng dữ liệu nhất quán.
- Kiểm thử và hoàn thiện (Testing & Optimization): Tiến hành kiểm thử các API bằng Postman, kiểm tra các luồng nghiệp vụ quan trọng như đặt hàng, thanh toán và quản lý tồn kho. Đồng thời tối ưu hiệu năng hệ thống, cải thiện trải nghiệm người dùng và xử lý các lỗi phát sinh.

Ngoài ra, quá trình phát triển được thực hiện theo hướng lặp lại (iterative), trong đó mỗi giai đoạn đều có thể được điều chỉnh và cải tiến dựa trên kết quả kiểm thử và phản hồi thực tế.

2.3 Mô hình phát triển phần mềm

Dự án áp dụng Agile – phương pháp phát triển phần mềm linh hoạt, chia nhỏ toàn bộ quá trình thành các chu kỳ ngắn hạn (Sprint) liên tiếp. Đối với một hệ thống thương mại điện tử chứa nhiều logic nghiệp vụ phức tạp như INFINITY, việc áp dụng mô hình tuần tự truyền thống (Waterfall) tiềm ẩn rủi ro lớn do khó thích ứng với các thay đổi phát sinh. Ngược lại, Agile đề cao tính linh hoạt, ưu tiên sản phẩm chạy được và phản hồi liên tục. Cơ chế này cho phép kiểm thử sớm các module trọng yếu ngay khi hoàn thành, giúp nhanh chóng phát hiện và khắc phục kịp thời các lỗi nghiêm trọng (như tính sai tổng tiền hay trừ lỗi tồn kho) qua từng vòng lặp cải tiến.

Quá trình xây dựng hệ thống được triển khai theo hướng kế thừa qua 4 Sprint cốt lõi:

- Sprint 1 (Khởi tạo & Bảo mật): Thiết lập cấu trúc Monorepo, kết nối cơ sở dữ liệu MongoDB và xây dựng luồng xác thực an toàn bằng JWT.
- Sprint 2 (Quản trị Dữ liệu): Phát triển các API CRUD cho danh mục, sản phẩm. Xây dựng Admin Dashboard và triển khai tính năng quản lý tồn kho (Restock).
- Sprint 3 (Nghiệp vụ Mua bán): Hoàn thiện giao diện khách hàng (tìm kiếm, giỏ hàng). Xây dựng luồng thanh toán (Checkout) tích hợp cơ chế tính giá phía máy chủ (Server-side Pricing), trừ kho nguyên tử và xử lý Voucher.

- Sprint 4 (Hoàn thiện & Tối ưu): Hoàn thiện luồng quản lý đơn hàng cho Admin (cập nhật trạng thái, tự động hoàn kho), tối ưu trải nghiệm người dùng (UI/UX) và tiến hành kiểm thử tổng thể.

Nhờ cách tiếp cận lặp lại và kiểm thử liên tục này, hệ thống không chỉ bám sát được tiến độ mà còn đảm bảo tối đa tính chính xác, tính ổn định và toàn vẹn dữ liệu trước khi đưa vào vận hành.

2.4 Kiến trúc phần mềm

2.4.1 Kiến trúc Client – Server

Hệ thống INFINITY được xây dựng theo kiến trúc Client – Server, trong đó các thành phần được phân tách rõ ràng nhằm đảm bảo tính độc lập, khả năng mở rộng và dễ bảo trì.

Trong mô hình này, hệ thống được chia thành hai phía chính:

- Thinn Client (React 19): Phía máy khách đảm nhiệm vai trò hiển thị giao diện người dùng và xử lý tương tác cơ bản. Ứng dụng được xây dựng theo mô hình Single Page Application (SPA), sử dụng React Router để điều hướng và Axios để giao tiếp với Backend thông qua các API. Client chỉ chịu trách nhiệm thu thập dữ liệu đầu vào và hiển thị kết quả, không xử lý các logic nghiệp vụ quan trọng.
- Thick Server (Node.js/Express): Phía máy chủ đóng vai trò trung tâm xử lý toàn bộ logic nghiệp vụ của hệ thống. Server thực hiện các chức năng như xác thực người dùng (JWT), kiểm soát phân quyền, xử lý đơn hàng, tính toán giá trị giao dịch và quản lý tồn kho. Đồng thời, server chịu trách nhiệm truy vấn và cập nhật dữ liệu từ cơ sở dữ liệu MongoDB.

Sự phân tách giữa Thinn Client và Thick Server mang lại các lợi ích sau:

- Tăng cường bảo mật: Các thuật toán quan trọng (tính giá, kiểm tra voucher, quản lý tồn kho) được xử lý hoàn toàn phía server
- Tối ưu hiệu năng: Giảm tải cho phía client, giúp giao diện hoạt động mượt mà
- Dễ mở rộng: Có thể phát triển thêm mobile app hoặc các client khác sử dụng chung API
- Dễ bảo trì: Logic được tập trung tại backend, thuận tiện cho việc cập nhật và nâng cấp

Nhờ kiến trúc này, hệ thống đảm bảo được sự cân bằng giữa hiệu năng, bảo mật và khả năng mở rộng trong môi trường thực tế.

2.4.2 Kiến trúc RESTful API

Chuẩn giao tiếp RESTful API được thiết lập làm cầu nối giữa Client và Server. Kiến trúc này mang tính phi trạng thái (Stateless), mỗi yêu cầu từ Client phải chứa đầy đủ thông tin để Server có thể xử lý. Việc tận dụng tối đa các phương thức HTTP tiêu chuẩn (GET, POST) kết hợp với định dạng dữ liệu trả về JSON tạo ra một giao thức giao tiếp đồng nhất và dễ dàng nâng cấp.

2.4.3 Kiến trúc phân tầng (3-tier)

Máy chủ Backend được thiết kế theo cấu trúc phân tầng (3-tier Architecture) nhằm tăng tính độc lập và dễ bảo trì:

- Tầng Định tuyến (Routes): Tiếp nhận các HTTP Request theo đúng định dạng điểm cuối (Endpoint) và điều hướng đến các bộ điều khiển tương ứng.
- Tầng Nghiệp vụ (Controllers & Middleware): Nơi xử lý logic cốt lõi. Middleware làm nhiệm vụ tiền xử lý (xác thực Token, phân tích form-data chứa ảnh). Controllers thực thi các thuật toán tính toán và gọi lệnh CSDL.
- Tầng Dữ liệu (Models): Sử dụng Mongoose ODM để định nghĩa cấu trúc dữ liệu, thiết lập các quy tắc hợp lệ (Validation) và giao tiếp trực tiếp với MongoDB.

2.5 Lựa chọn công nghệ

2.5.1 Tiêu chí lựa chọn

Bộ công nghệ được lựa chọn dựa trên các tiêu chí: Khả năng xử lý đầu vào/đầu ra bất đồng bộ (Async I/O) tốt để chịu tải nhiều request cùng lúc; tính linh hoạt trong việc tạo dựng giao diện Responsive; và sự đồng nhất về ngôn ngữ để tối ưu hóa thời gian lập trình.

Hệ sinh thái MERN Stack (MongoDB, Express, React, Node.js) mang lại lợi thế vượt trội nhờ việc đồng nhất một ngôn ngữ duy nhất là JavaScript/TypeScript trên toàn bộ nền tảng. Định dạng cấu trúc dữ liệu JSON giao tiếp qua API hoàn toàn tương thích với chuẩn lưu trữ BSON/Document của MongoDB. Thêm vào đó, MongoDB cung cấp tính linh hoạt vượt trội, giải quyết triệt để các bài toán lưu trữ mảng dữ liệu lồng nhau mà CSDL quan hệ (SQL) thường gặp khó khăn.

2.5.2 Các công nghệ sử dụng

Hệ thống INFINITY được xây dựng dựa trên tập hợp các công nghệ hiện đại thuộc mô hình MERN Stack, kết hợp với các công cụ hỗ trợ nhằm đảm bảo hiệu năng, khả năng mở rộng và trải nghiệm người dùng.

Các công nghệ chính được sử dụng bao gồm:

- React 19: Thư viện JavaScript dùng để xây dựng giao diện người dùng theo mô hình component-based. React giúp tái sử dụng code hiệu quả và hỗ trợ xây dựng ứng dụng dạng Single Page Application (SPA).
- Vite 7: Công cụ build và môi trường phát triển (development server) có tốc độ khởi động nhanh, hỗ trợ Hot Module Replacement (HMR), giúp tăng hiệu suất trong quá trình phát triển.
- React Router v7: Thư viện định tuyến phía client, cho phép điều hướng giữa các trang trong ứng dụng mà không cần tải lại trang, nâng cao trải nghiệm người dùng.
- Tailwind CSS 3: Framework CSS dạng utility-first, giúp xây dựng giao diện nhanh chóng, linh hoạt và dễ tùy chỉnh.

- Node.js: Môi trường thực thi JavaScript phía server, cho phép xây dựng hệ thống backend với hiệu năng cao và khả năng xử lý bất đồng bộ.
- Express 5: Framework web nhẹ cho Node.js, hỗ trợ xây dựng RESTful API một cách nhanh chóng và có tổ chức.
- MongoDB Atlas: Hệ quản trị cơ sở dữ liệu NoSQL dạng document, được triển khai trên nền tảng đám mây, cho phép lưu trữ dữ liệu linh hoạt và dễ mở rộng.
- Mongoose: Thư viện ODM giúp tương tác với MongoDB, hỗ trợ định nghĩa schema, validation và thao tác dữ liệu một cách có cấu trúc.
- JWT (JSON Web Token): Cơ chế xác thực dựa trên token, giúp bảo mật API và phân quyền người dùng.
- Bcrypt: Thư viện mã hóa mật khẩu, đảm bảo an toàn thông tin người dùng khi lưu trữ trong cơ sở dữ liệu.
- Multer: Middleware xử lý upload file từ phía client lên server, thường dùng cho việc tải ảnh sản phẩm.
- Cloudinary: Dịch vụ lưu trữ và quản lý hình ảnh trên nền tảng đám mây, giúp giảm tải cho server và tối ưu băng thông.
- Axios: Thư viện gửi HTTP request từ phía client đến server, hỗ trợ xử lý bất đồng bộ và quản lý response hiệu quả.
- React Toastify: Thư viện hiển thị thông báo (toast), giúp cải thiện trải nghiệm người dùng thông qua phản hồi trực quan.
- Recharts: Thư viện xây dựng biểu đồ động cho React. Được ứng dụng trong phân hệ Admin Dashboard để trực quan hóa dữ liệu doanh thu 12 tháng dưới dạng biểu đồ cột, giúp ban quản trị dễ dàng theo dõi chỉ số tăng trưởng MoM.
- Vietnam Provinces API v2: Giao diện lập trình ứng dụng mở (Open API) cung cấp dữ liệu hành chính Việt Nam. Được tích hợp vào luồng thanh toán (Checkout) để xây dựng Form địa chỉ thông minh (Cascading dropdown), tự động tải danh sách Phường/Xã tương ứng khi người dùng chọn Tỉnh/Thành phố.

2.6 Bảo mật hệ thống

2.6.1 Xác thực và phân quyền

Hệ thống triển khai cơ chế xác thực không trạng thái (Stateless Authentication) bằng JSON Web Token (JWT). Hệ thống phân quyền được chia thành hai lớp bảo vệ độc lập thông qua các Middleware: auth.js để kiểm soát các tác vụ của người dùng thông thường và adminAuth.js để giới hạn quyền truy cập vào các API nhạy cảm của ban quản trị, ngăn ngừa rủi ro leo thang đặc quyền.

2.6.2 Bảo vệ dữ liệu

Bảo mật thông tin cá nhân là yêu cầu bắt buộc. Mật khẩu người dùng được băm (Hash) một chiều bằng thuật toán Bcrypt trước khi lưu trữ, đảm bảo an toàn ngay cả trong trường hợp cơ sở dữ liệu bị xâm nhập. Các chuỗi cấu hình nhạy cảm như khóa bí mật JWT hay thông tin kết nối Database đều được cô lập trong tệp biến môi trường [.env] và tuyệt đối không đưa lên các kho lưu trữ mã nguồn mở.

2.6.3 Kiểm soát đầu vào

Để ngăn chặn các luồng dữ liệu rác hoặc hành vi chèn ép nghiệp vụ (Injection/Tampering), hệ thống ứng dụng các biện pháp kiểm soát nghiêm ngặt:

- Enum Validation & Payload Mapping: Các trường dữ liệu trạng thái (như OrderStatus hay UserStatus) được định nghĩa cứng tại tầng Model (Ví dụ: trạng thái đơn hàng chỉ nhận các giá trị Order Placed, Shipped, Delivered...). Mọi giá trị không hợp lệ sẽ bị từ chối. Đặc biệt, hệ thống hỗ trợ cơ chế Mapping linh hoạt (ví dụ: cho phép truyền ID dạng số từ 1 đến 6 qua Postman và tự động chuyển đổi thành chuỗi Enum tương ứng tại Server), giúp việc kiểm thử và tích hợp dễ dàng hơn mà vẫn đảm bảo tính chặt chẽ.
- Xác thực Logic phía máy chủ: Mọi nghiệp vụ liên quan đến tài chính, giỏ hàng, hoặc trừ kho đều không tin tưởng dữ liệu tính toán từ Client, bắt buộc phải truy xuất nguồn dữ liệu gốc từ Server để đối chiếu trước khi thực thi.

2.7 Tổng kết chương

Chương 2 đã làm rõ phương pháp tiếp cận và mô hình giải quyết vấn đề của dự án INFINITY. Bằng việc phân tích kỹ lưỡng các yêu cầu, hệ thống đã lựa chọn mô hình phát triển Agile kết hợp kiến trúc phân tầng Client-Server dựa trên hệ sinh thái MERN Stack hiện đại. Những giải pháp kiến trúc phần mềm linh hoạt, cùng với các biện pháp kiểm soát dữ liệu chặt chẽ (Atomic Update, Server-side Pricing, JWT) không chỉ đáp ứng tốt hiệu năng hoạt động mà còn bảo đảm an toàn thông tin tối đa. Đây là nền tảng lý thuyết và kỹ thuật vững chắc để bước vào quá trình thiết kế cơ sở dữ liệu chi tiết trong chương tiếp theo.

CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ VÀ THỰC NGHIỆM HỆ THỐNG

3.1 Phân tích hệ thống

Quá trình phân tích hệ thống giúp làm rõ các yêu cầu chức năng thông qua việc xác định những đối tượng tương tác với nền tảng (Actors) và mô hình hóa các hành vi đó dưới dạng biểu đồ Use Case và biểu đồ trình tự (Sequence Diagram).

3.1.1 Xác định tác nhân (Actors)

Dựa trên kiến trúc phân tách hai phân hệ Frontend và Admin, hệ thống INFINITY E-COMMERCE được mô hình hóa với 2 nhóm tác nhân cốt lõi:

- Khách hàng (Customer/User): Đại diện cho toàn bộ người dùng truy cập vào phân hệ giao diện mua sắm (Công 5173). Tác nhân này hoạt động dưới hai trạng thái phân quyền:
 - *Khi chưa xác thực (Khách vắng lai)*: Có quyền duyệt danh mục, tìm kiếm, xem chi tiết sản phẩm, đăng ký tài khoản và đăng ký nhận bản tin (Newsletter) để nhận mã voucher.
 - *Khi đã xác thực (Cấp phát Token JWT)*: Được mở khóa toàn bộ các đặc quyền giao dịch bao gồm: Quản lý giỏ hàng (thêm/sửa/xóa), thực hiện quy trình thanh toán (Checkout), áp dụng mã giảm giá và theo dõi lịch sử/trạng thái đơn hàng cá nhân.
- Quản trị viên (Admin): Đại diện cho người điều hành hệ thống, truy cập thông qua một phân hệ quản trị độc lập (Công 5174) bằng bộ thông tin xác thực nội bộ (ADMIN_EMAIL). Tác nhân này có đặc quyền toàn năng để thao tác với dữ liệu: Xem thống kê tổng quan (Tích hợp biểu đồ Recharts và phân tích tăng trưởng MoM), quản lý sản phẩm và danh mục (tính toán doanh thu on-the-fly), xử lý luồng đơn hàng (kích hoạt tự động hoàn kho khi hủy đơn) và quản lý người dùng (tìm kiếm/phân trang Server-side, khóa/mở khóa tài khoản, thực hiện xóa Cascade).

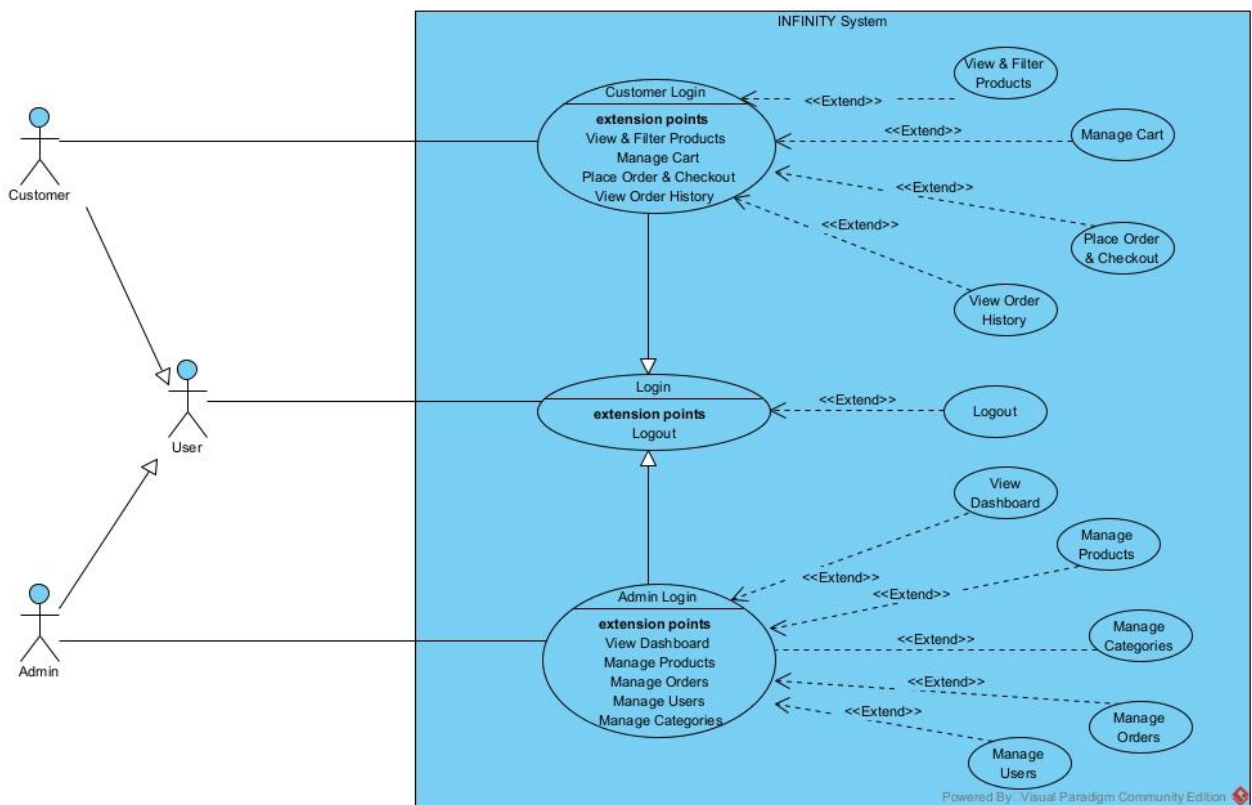
3.1.2 Biểu đồ Use Case tổng quát

Biểu đồ Use Case tổng quát cung cấp một cái nhìn toàn cảnh về kiến trúc chức năng của hệ thống và giới hạn quyền hạn của từng đối tượng tham gia.

Hệ thống được thiết kế với hai tác nhân (Actor) chính là **Customer** (Khách hàng) và Admin (Quản trị viên). Cả hai đều kế thừa các đặc tính từ một tác nhân trừu tượng chung là User (Người dùng) với các nghiệp vụ cơ sở như Đăng nhập (Login) và Đăng xuất (Logout). Thông qua các điểm mở rộng (Extension points), hệ thống rẽ nhánh thành hai phân hệ độc lập:

- Phân hệ Khách hàng (Customer Module): Bao gồm các Use Case phục vụ cho trải nghiệm mua sắm trực tuyến. Sau khi xác thực tài khoản thành công qua Customer Login, khách hàng có thể tiếp cận các chức năng mở rộng (<<Extend>>):
 - View & Filter Products (Xem và lọc sản phẩm): Duyệt danh mục, tìm kiếm và xem chi tiết hàng hóa theo thời gian thực.

- Manage Cart (Quản lý giỏ hàng): Thêm sản phẩm, chọn kích cỡ và điều chỉnh số lượng.
 - Place Order & Checkout (Đặt hàng và thanh toán): Nhập thông tin vận chuyển thông minh, áp dụng mã giảm giá và thanh toán.
 - View Order History (Lịch sử đơn hàng): Theo dõi tiến trình xử lý và trạng thái đơn hàng cá nhân.
- Phân hệ Quản trị (Admin Module): Đây là khu vực vận hành nội bộ, yêu cầu mức độ bảo mật cao. Sau khi đi qua cổng xác thực Admin Login, ban quản trị được cấp quyền truy cập vào các Use Case nghiệp vụ nặng:
- View Dashboard (Xem thống kê): Xem biểu đồ doanh thu, phân tích tăng trưởng và cảnh báo tồn kho.
 - Manage Categories (Quản lý danh mục): Thêm, xóa và tự động phân tích doanh thu theo cấu trúc phân loại.
 - Manage Products (Quản lý sản phẩm): Thêm mới, chỉnh sửa thông tin, upload hình ảnh và nhập kho (Restock) trực tiếp.
 - Manage Orders (Quản lý đơn hàng): Cập nhật trạng thái giao hàng và tự động kích hoạt hoàn kho khi hủy đơn.
 - Manage Users (Quản lý người dùng): Kiểm soát danh sách tài khoản, khóa/mở khóa và thực hiện xóa dữ liệu dây chuyền (Cascade Delete).



Hình 1 - Biểu đồ Use Case tổng quát

3.1.3 Xây dựng Use Case Diagram

Hệ thống INFINITY E-COMMERCE được mô hình hóa với 2 nhóm tác nhân (Actors) cốt lõi: Khách hàng (Customer) và Quản trị viên (Admin).

3.1.3.1 Nhóm Use Case dành cho Khách hàng (Customer)

Tác nhân này đại diện cho mọi người dùng truy cập vào Frontend (cả chưa đăng nhập và đã đăng nhập). Các nghiệp vụ được phân rã như sau:

- Nghiệp vụ Tương tác & Khám phá:
 - UC01 - Xem trang chủ
 - UC02 - Khám phá sản phẩm
 - UC03 - Tìm kiếm và lọc (Real-time filtering)
 - UC04 - Xem chi tiết sản phẩm
 - UC05 - Đăng ký nhận bản tin (Cấp phát mã giảm giá)
- Nghiệp vụ Tài khoản:
 - UC06 - Đăng ký tài khoản
 - UC07 - Đăng nhập (Xác thực JWT)
 - UC08 - Đăng xuất
- Nghiệp vụ Giao dịch & Quản lý:
 - UC09 - Quản lý giỏ hàng
 - UC10 - Đặt hàng và thanh toán (Checkout - Tích hợp API hành chính & Trừ kho nguyên tử)
 - UC11 - Xem lịch sử đơn hàng

3.1.3.2 Nhóm Use Case dành cho Quản trị viên (Admin)

Đại diện cho ban quản trị, hoạt động trên phân hệ nội bộ độc lập với các đặc quyền toàn năng:

- UC12 - Xác thực quản trị (Cấp quyền Admin Privilege)
- UC13 - Xem bảng thống kê (Dashboard - Phân tích doanh thu on-the-fly)
- UC14 - Quản lý danh mục (Category - Tự động sắp xếp theo doanh số)
- UC15 - Quản lý sản phẩm và Nhập kho (Product & Restock)
- UC16 - Quản lý đơn hàng (Order - Tự động hoàn kho khi hủy)
- UC17 - Quản lý người dùng (User - Phân trang Server-side & Xóa Cascade)

3.1.4 Đặc tả Use Case (Use Case Specification)

Để làm rõ logic hoạt động bên trong hệ thống, đặc biệt là các luồng nghiệp vụ phức tạp ở tầng Backend, phần này tiến hành đặc tả chi tiết các Use Case cốt lõi nhất của phân hệ Khách hàng và phân hệ Quản trị.

1. Đặc tả Use Case 1: Xem trang chủ (UC01)

Trang chủ là điểm chạm đầu tiên của khách hàng với hệ thống. Đặc tả này tập trung vào việc tự động hóa hiển thị các chiến dịch và danh sách sản phẩm nổi bật dựa trên dữ liệu doanh số thực tế, giúp tăng khả năng chuyển đổi.

Bảng 1. Đặc tả Use Case 1 - Xem trang chủ

Thành phần	Mô tả chi tiết
Tên Use Case	Xem trang chủ (UC01)
Tác nhân	Khách hàng
Mô tả	Cho phép người dùng xem giao diện chính, banner chiến dịch và danh sách các sản phẩm bán chạy nhất (Top 5 Best Sellers).
Tiền điều kiện	Không yêu cầu.
Luồng sự kiện chính	1. Người dùng nhập URL của nền tảng Frontend (Cổng 5173) vào trình duyệt. 2. Ứng dụng React khởi chạy, nạp các component cốt lõi (Hero Banner, Navbar, Footer). 3. Frontend tự động gọi API GET /api/dashboard/best-sellers (Public Endpoint). 4. Backend sử dụng hàm aggregateBestSellers kết hợp Aggregation Pipeline để tính toán doanh số thực tế trực tiếp từ CSDL MongoDB, sau đó trả về danh sách Top 5 sản phẩm bán chạy nhất dưới định dạng JSON. 5. Frontend render giao diện Trang chủ hoàn chỉnh và hiển thị cho người dùng.
Luồng ngoại lệ	A1 - Lỗi kết nối máy chủ: Tại bước 3, nếu Backend không phản hồi khiến Frontend không tải được dữ liệu sản phẩm, hệ thống sẽ render giao diện khung chờ (Skeleton loading) để giữ bố cục, đồng thời hiển thị thông báo " <i>Không thể kết nối đến máy chủ</i> ".
Hậu điều kiện	Trang chủ được hiển thị thành công với đầy đủ thông tin tĩnh và động.

2. Đặc tả Use Case 2: Khám phá sản phẩm (UC02)

Nhằm cung cấp cái nhìn toàn cảnh về kho hàng, Use Case này mô tả quá trình hệ thống truy xuất, lọc bỏ các mặt hàng ngừng kinh doanh và trình bày toàn bộ sản phẩm đang hoạt động lên giao diện một cách trực quan.

Bảng 2. Đặc tả Use Case 2 - Khám phá sản phẩm

Thành phần	Mô tả chi tiết
Tên Use Case	Khám phá sản phẩm (UC02)
Tác nhân	Khách hàng
Mô tả	Cho phép khách hàng duyệt toàn bộ danh mục hàng hóa đang được kinh doanh trên hệ thống.
Tiền điều kiện	Không yêu cầu.
Luồng sự kiện chính	1. Khách hàng nhấp vào menu "Collection" trên thanh điều hướng. 2. Trình duyệt gọi API GET /api/product/list. 3. Backend truy xuất toàn bộ dữ liệu từ collection products trong MongoDB. 4. Backend trả về mảng đối tượng sản phẩm (chỉ các sản phẩm đang có trạng thái kích hoạt) cho Client. 5. Frontend render danh sách sản phẩm dưới dạng lưới (Grid Layout) tương thích đa thiết bị.
Luồng ngoại lệ	A1 - Dữ liệu trống: Tại bước 3, nếu cơ sở dữ liệu chưa có sản phẩm nào, màn hình Frontend sẽ hiển thị thông báo " <i>Hiện chưa có sản phẩm nào trong cửa hàng</i> ".
Hậu điều kiện	Danh sách toàn bộ sản phẩm được tải và hiển thị đầy đủ trên màn hình.

3. Đặc tả Use Case 3: Tìm kiếm và lọc sản phẩm (UC03)

Để giúp khách hàng nhanh chóng tiếp cận đúng sản phẩm mục tiêu, hệ thống áp dụng cơ chế lọc dữ liệu thời gian thực (Real-time) ngay tại Client-side, mang lại trải nghiệm tương tác tức thì mà không cần tải lại trang.

Bảng 3. Use Case 3 - Tìm kiếm và lọc sản phẩm

Thành phần	Mô tả chi tiết
Tên Use Case	Tìm kiếm và lọc sản phẩm (UC03)
Tác nhân	Khách hàng

Mô tả	Cho phép người dùng tìm kiếm sản phẩm theo từ khóa (tên) hoặc lọc dữ liệu theo các danh mục (Category) một cách trực quan.
Tiền điều kiện	Khách hàng đang ở trang Collection (đã có dữ liệu sản phẩm gốc).
Luồng sự kiện chính	1. Khách hàng nhập từ khóa vào thanh tìm kiếm (SearchBar) hoặc tích chọn các ô phân loại danh mục (Checkbox). 2. React State lập tức ghi nhận sự thay đổi dữ liệu đầu vào. 3. Trình duyệt tự động thực hiện thuật toán lọc (Filter) trên mảng dữ liệu sản phẩm hiện tại dựa trên các điều kiện khớp nhau. 4. Giao diện cập nhật danh sách kết quả ngay lập tức (Real-time) mà không cần tải lại trang.
Luồng ngoại lệ	A1 - Không tìm thấy kết quả: Tại bước 3, nếu không có sản phẩm nào khớp với từ khóa hoặc bộ lọc, giao diện hiển thị thông báo " <i>Không tìm thấy sản phẩm phù hợp</i> ".
Hậu điều kiện	Danh sách sản phẩm trên màn hình được thu hẹp đúng với điều kiện tìm kiếm/lọc của khách hàng.

4. Đặc tả Use Case 4: Xem chi tiết sản phẩm (UC04)

Đây là bước đệm quan trọng trước khi phát sinh giao dịch. Đặc tả này làm rõ luồng truy xuất thông tin mô tả, giá cả và yêu cầu bắt buộc người dùng chọn kích cỡ (Size) để kiểm tra lượng tồn kho khả dụng.

Bảng 4. Đặc tả Use Case 4 - Xem chi tiết sản phẩm

Thành phần	Mô tả chi tiết
Tên Use Case	Xem chi tiết sản phẩm (UC04)
Tác nhân	Khách hàng
Mô tả	Cho phép khách hàng xem thông tin giá bán, mô tả chi tiết, bộ sưu tập hình ảnh và kiểm tra lượng tồn kho của các biến thể kích cỡ (Size).
Tiền điều kiện	Không yêu cầu.
Luồng sự kiện chính	1. Khách hàng nhấp vào một thẻ sản phẩm bất kỳ trên giao diện.

	2. React Router điều hướng URL sang trang chi tiết có đính kèm Product ID. 3. Frontend tìm kiếm dữ liệu chi tiết của sản phẩm tương ứng thông qua ID này. 4. Giao diện render khu vực xem ảnh tĩnh, hiển thị giá tiền, và liệt kê các nút bấm đại diện cho các kích cỡ khả dụng trong mảng sizesStock.
Luồng ngoại lệ	A1 - Sản phẩm không tồn tại: Tại bước 3, nếu Product ID truyền vào URL không hợp lệ (do nhập tay sai hoặc link cũ), hệ thống điều hướng người dùng quay lại trang chủ hoặc hiển thị màn hình lỗi 404 Not Found.
Hậu điều kiện	Giao diện chi tiết sản phẩm hiển thị thành công, khách hàng có thể thao tác chọn Size để đưa vào giỏ hàng.

5. Đặc tả Use Case 5: Đăng ký nhận bản tin (UC05)

Phục vụ cho các chiến dịch Marketing và chăm sóc khách hàng, nghiệp vụ này mô tả luồng thu thập email người dùng và tự động cấp phát mã giảm giá nội bộ một cách an toàn.

Bảng 5. Đặc tả Use Case 5 - Đăng ký nhận bản tin

Thành phần	Mô tả chi tiết
Tên Use Case	Đăng ký nhận bản tin và lấy mã Voucher (UC05)
Tác nhân	Khách hàng
Mô tả	Cho phép khách hàng đăng ký địa chỉ email để theo dõi tin tức và nhận mã giảm giá giới hạn của hệ thống (giảm 20% giá trị đơn hàng).
Tiền điều kiện	Không yêu cầu.
Luồng sự kiện chính	1. Khách hàng kéo xuống khối NewsletterBox, nhập Email cá nhân và nhấn "Subscribe". 2. Trình duyệt gửi Payload (Email) thông qua API POST /api/newsletter/subscribe. 3. Backend tiếp nhận, kiểm tra định dạng email và xác thực email chưa tồn tại trong collection newsletters. 4. Backend tạo bản ghi mới trong CSDL và trả về HTTP Status 200 kèm mã voucher.

	5. Frontend hiển thị Toast thông báo đăng ký thành công và xuất hiện mã giảm giá "BARCA20" trên màn hình.
Luồng ngoại lệ	▪ A1 - Sai định dạng Email: Tại bước 2, nếu email nhập vào không đúng định dạng (thiếu @...), Frontend chặn gọi API và yêu cầu nhập lại. ▪ A2 - Trùng lặp Email: Tại bước 3, nếu email đã được đăng ký trước đó, Backend trả về lỗi 400. Frontend hiển thị Toast: "Email này đã được đăng ký".
Hậu điều kiện	Bản ghi Email được lưu trữ thành công, người dùng nhận được thông tin mã giảm giá.

6. Đặc tả Use Case 6: Đăng ký tài khoản (UC06)

Nhằm đảm bảo tính định danh và an toàn thông tin cho khách hàng, luồng nghiệp vụ này nhấn mạnh vào cơ chế băm mật khẩu một chiều (Bcrypt) và cấp phát Token xác thực ngay khi khởi tạo tài khoản.

Bảng 6. Đặc tả Use Case 6 - Đăng ký tài khoản

Thành phần	Mô tả chi tiết
Tên Use Case	Đăng ký tài khoản (UC06)
Tác nhân	Khách hàng
Mô tả	Cho phép người dùng tạo tài khoản mới để định danh trên hệ thống và sử dụng các tính năng giao dịch.
Tiền điều kiện	Người dùng chưa đăng nhập.
Luồng sự kiện chính	1. Khách hàng điền thông tin: Tên, Email, Mật khẩu vào form Đăng ký. 2. Trình duyệt gọi API POST /api/user/register. 3. Backend kiểm tra Email trong collection users để đảm bảo chưa tồn tại. 4. Backend sử dụng thư viện Bcrypt để băm (Hash) mật khẩu người dùng. 5. Backend khởi tạo đối tượng User mới với mật khẩu đã băm và lưu vào MongoDB. 6. Hệ thống tạo và cấp phát một chuỗi JWT Token trả về cho Client.

	7. Frontend lưu Token vào hệ thống (Context), cập nhật trạng thái đã đăng nhập và tự động điều hướng về Trang chủ.
Luồng ngoại lệ	A1 - Email đã tồn tại: Nếu Email đã có trong hệ thống, Backend trả về lỗi 409. Frontend hiển thị Toast thông báo: <i>"Tài khoản (Email) này đã tồn tại"</i> .
Hậu điều kiện	Một bản ghi User mới được tạo trong CSDL, khách hàng được cấp phiên làm việc hợp lệ.

7. Đặc tả Use Case 7: Đăng nhập (UC07)

Đây là chốt chặn kiểm soát quyền truy cập hệ thống. Đặc tả làm rõ quá trình đối chiếu dữ liệu mã hóa và kiểm tra trạng thái khóa (Blocked) của tài khoản trước khi cấp phiên làm việc.

Bảng 7. Đặc tả Use Case 7 - Đăng nhập

Thành phần	Mô tả chi tiết
Tên Use Case	Đăng nhập hệ thống (UC07)
Tác nhân	Khách hàng
Mô tả	Xác thực định danh tài khoản để cấp quyền sử dụng các tính năng quản lý giỏ hàng và thanh toán.
Tiền điều kiện	Người dùng đã có tài khoản trên hệ thống.
Luồng sự kiện chính	1. Khách hàng nhập Email và Mật khẩu vào form Đăng nhập. 2. Trình duyệt gọi API POST /api/user/login. 3. Backend truy xuất MongoDB tìm tài khoản tương ứng với Email. 4. Backend dùng thư viện Bcrypt đối chiếu (Compare) mật khẩu nhập vào với mã băm lưu trong CSDL. 5. Backend kiểm tra trường trạng thái status của tài khoản để đảm bảo tài khoản đang hoạt động (Active). 6. Nếu hợp lệ, Backend tạo và cấp phát JWT Token, trả về cho Client. 7. Frontend lưu Token, cập nhật Global State và điều hướng người dùng.

Luồng ngoại lệ	- A1 - Sai thông tin xác thực: Nếu Email không tồn tại hoặc sai mật khẩu, Backend trả về lỗi 401. Frontend hiển thị Toast: <i>"Thông tin đăng nhập không chính xác"</i>. - A2 - Tài khoản bị khóa: Nếu Backend phát hiện trạng thái tài khoản là Blocked, hệ thống trả về lỗi 403. Frontend hiển thị Toast: <i>"Tài khoản của bạn đã bị khóa bởi Quản trị viên"</i>.
Hậu điều kiện	Một bản ghi User mới được tạo trong CSDL, khách hàng được cấp phiên làm việc hợp lệ.

8. Đặc tả Use Case 8: Đăng xuất (UC08)

Luồng nghiệp vụ diễn ra hoàn toàn tại phía Client, đảm bảo việc xóa bỏ an toàn các dữ liệu nhạy cảm và Token khỏi bộ nhớ trình duyệt, đưa người dùng trở về trạng thái khách vắng lại.

Bảng 8. Đặc tả Use Case 8 - Đăng xuất

Thành phần	Mô tả chi tiết
Tên Use Case	Đăng xuất (UC08)
Tác nhân	Khách hàng
Mô tả	Cho phép kết thúc phiên làm việc an toàn để bảo mật thông tin cá nhân.
Tiền điều kiện	Khách hàng đang trong phiên đăng nhập hợp lệ (sở hữu Token).
Luồng sự kiện chính	- Khách hàng nhấn nút "Đăng xuất" trên thanh điều hướng. - Frontend thực thi hàm xóa JWT Token khỏi bộ nhớ (LocalStorage). - Frontend reset các State liên quan đến thông tin cá nhân và dữ liệu Giỏ hàng trong React Context về giá trị mặc định (Rỗng). - Hệ thống tự động điều hướng người dùng quay lại Trang chủ.
Luồng ngoại lệ	Không có ngoại lệ do tiến trình xử lý diễn ra 100% tại Client-side.

Hậu điều kiện	Phiên làm việc kết thúc, toàn bộ dữ liệu nhạy cảm trên trình duyệt bị xóa, khách hàng trở về trạng thái Khách vắng lại.
---------------	---

9. Đặc tả Use Case 9: Quản lý giỏ hàng (UC09)

Để duy trì tính xuyên suốt của quá trình mua sắm trên nhiều thiết bị, đặc tả này mô tả cách hệ thống đồng bộ hóa trạng thái giỏ hàng vào cơ sở dữ liệu MongoDB dựa trên định danh JWT Token.

Bảng 9. Đặc tả Use Case 9 - Quản lý giỏ hàng

Thành phần	Mô tả chi tiết
Tên Use Case	Quản lý giỏ hàng (UC09)
Tác nhân	Khách hàng
Mô tả	Cho phép người dùng thêm sản phẩm mới, cập nhật số lượng hoặc xóa sản phẩm khỏi giỏ hàng.
Tiền điều kiện	Khách hàng đã đăng nhập vào hệ thống (có JWT Token hợp lệ).
Luồng sự kiện chính	1. Tại trang chi tiết, khách hàng chọn một kích cỡ (Size) và nhấn "Add to cart" (hoặc thay đổi số lượng tại trang Cart). 2. Frontend gọi API POST /api/cart/add (hoặc /update) và đính kèm JWT Token vào Header Authorization. 3. Yêu cầu đi qua Middleware auth.js. Middleware giải mã Token, trích xuất ID người dùng và cho phép đi tiếp. 4. Backend tìm kiếm bản ghi User, cập nhật cấu trúc dữ liệu giỏ hàng (mảng cartData) và lưu đè vào CSDL. 5. Backend trả về kết quả thành công. Frontend hiển thị Toast "Đã cập nhật giỏ hàng" và thay đổi con số thống kê giỏ hàng trên Navbar.
Luồng ngoại lệ	▪ A1 - Chưa chọn kích cỡ: Nếu nhấn Thêm vào giỏ nhưng chưa chọn Size, Frontend chặn gọi API và báo Toast: "Vui lòng chọn Kích cỡ sản phẩm". ▪ A2 - Lỗi xác thực: Nếu Token hết hạn hoặc chưa đăng nhập, Middleware chặn request và trả lỗi 401.

	Frontend báo Toast " <i>Vui lòng đăng nhập</i> " và đẩy sang trang Login.
Hậu điều kiện	Dữ liệu mảng cartData của tài khoản người dùng trong CSDL MongoDB được cập nhật chính xác.

10. Đặc tả Use Case 10: Đặt hàng và Thanh toán (UC10)

Đây là luồng nghiệp vụ xương sống và phức tạp nhất của hệ thống. Đặc tả tích hợp hàng loạt kỹ thuật kiểm soát chéo: tính giá tại Server, đối chiếu Voucher 2 lớp, tích hợp API hành chính và trừ kho nguyên tử (\$inc) để chống bán vượt số lượng.

Bảng 10. Đặc tả Use Case 10 - Đặt hàng và Thanh toán

Thành phần	Mô tả chi tiết
Tên Use Case	Đặt hàng và thanh toán (UC10)
Tác nhân	Khách hàng
Mô tả	Cho phép khách hàng hoàn tất quá trình mua sắm các sản phẩm đang có trong giỏ hàng.
Tiền điều kiện	Khách hàng đã đăng nhập và giỏ hàng hiện tại có ít nhất một sản phẩm.
Luồng sự kiện chính	1. Khách hàng truy cập trang Thanh toán (Place Order) từ Giỏ hàng. 2. Hệ thống hiển thị form nhập thông tin giao hàng thông minh. Frontend tích hợp trực tiếp Vietnam Provinces API v2 để tạo hiệu ứng Cascading Dropdown 2 cấp (chọn Tỉnh/Thành phố tự động tải Phường/Xã). 3. Khách hàng điền đầy đủ thông tin, nhập mã Voucher (nếu có) và nhấn nút "Đặt hàng". 4. Client gửi dữ liệu Payload lên Backend (bỏ qua dữ liệu tổng tiền gửi từ Client). 5. Backend truy xuất CSDL, lấy giá gốc của từng sản phẩm để tự tính toán tổng tiền. 6. Backend áp dụng cơ chế xác thực 2 lớp mã Voucher: kiểm tra tính hợp lệ (còn hạn) và đối chiếu với mảng usedVouchers trong hồ sơ User để đảm bảo chưa từng sử dụng, sau đó trừ 20% tổng tiền.

	7. Backend thực thi lệnh trừ kho nguyên tử (\$inc) đối với từng biến thể kích cỡ (sizesStock) của sản phẩm. 8. Khi tồn kho đủ, hệ thống khởi tạo bản ghi đơn hàng (Order) mới trong CSDL và lưu thông tin Voucher đã dùng vào hồ sơ người dùng. 9. Hệ thống tự động làm sạch giỏ hàng của khách hàng. 10. Hiện thị thông báo đặt hàng thành công và chuyển hướng sang trang Lịch sử đơn hàng.
Luồng ngoại lệ	▪ A1 - Lỗi mã giảm giá: Nếu mã Voucher sai, hết hạn hoặc người dùng đã sử dụng trước đó, hệ thống chặn giao dịch và báo lỗi <i>"Mã giảm giá không hợp lệ hoặc đã được sử dụng"</i>. ▪ A2 - Lỗi thiếu tồn kho (Out of stock): Nếu tồn kho trong CSDL nhỏ hơn số lượng muốn đặt, giao dịch lập tức bị hủy bỏ (Rollback). Hệ thống báo lỗi <i>"Sản phẩm không đủ số lượng"</i>.
Hậu điều kiện	Đơn hàng mới được tạo ở trạng thái "Order Placed", tồn kho sản phẩm giảm đi tương ứng.

11. Đặc tả Use Case 11: Xem lịch sử đơn hàng (UC11)

Cung cấp không gian cá nhân hóa để khách hàng theo dõi giao dịch. Hệ thống sử dụng cơ chế bảo mật JWT để lọc và chỉ trả về đúng dữ liệu đơn hàng thuộc sở hữu của người dùng đang truy cập.

Bảng 11. Đặc tả Use Case 11 - Xem lịch sử đơn hàng

Thành phần	Mô tả chi tiết
Tên Use Case	Xem lịch sử đơn hàng (UC11)
Tác nhân	Khách hàng
Mô tả	Cho phép khách hàng tra cứu danh sách các đơn hàng đã đặt và theo dõi tiến trình giao hàng theo thời gian thực.
Tiền điều kiện	Khách hàng đã đăng nhập vào hệ thống (có JWT Token hợp lệ).
Luồng sự kiện chính	1. Khách hàng truy cập trang "Orders" từ thanh điều hướng (hoặc từ menu tài khoản).

	- Trình duyệt gọi API POST /api/order/userorders và đính kèm JWT Token vào Header Authorization. - Yêu cầu đi qua Middleware auth.js. Middleware giải mã Token, trích xuất thông tin userId và cho phép đi tiếp. - Backend truy vấn collection orders trong MongoDB, lọc ra danh sách các đơn hàng có trường userId khớp với ID vừa trích xuất, sắp xếp theo thời gian đặt hàng mới nhất. - Backend trả về danh sách đơn hàng cho Client. - Frontend render danh sách, hiển thị rõ các thông tin: Mã đơn, Danh sách sản phẩm, Tổng tiền, và Trạng thái hiện tại.
Luồng ngoại lệ	A1 - Lỗi xác thực: Tại bước kiểm tra Token, nếu Token không hợp lệ hoặc đã hết hạn, Backend trả về lỗi 401. Frontend hiển thị Toast " <i>Phiên đăng nhập hết hạn, vui lòng đăng nhập lại</i> " và chuyển hướng về trang Login.
Hậu điều kiện	Khách hàng nắm bắt được thông tin trạng thái của các giao dịch cá nhân.

12. Đặc tả Use Case 12: Xác thực Quản trị (UC12)

Đóng vai trò là cổng bảo mật đầu tiên của phân hệ Admin. Đặc tả làm rõ cơ chế đối chiếu thông tin xác thực trực tiếp với các biến môi trường để cấp đặc quyền quản trị cấp cao.

Bảng 12. Đặc tả Use Case 12 - Xác thực Quản trị

Thành phần	Mô tả chi tiết
Tên Use Case	Xác thực Quản trị viên (UC12)
Tác nhân	Admin
Mô tả	Cho phép quản trị viên đăng nhập vào bảng điều khiển nội bộ để thực hiện các thao tác quản trị toàn quyền trên hệ thống.
Tiền điều kiện	Người dùng có thông tin đăng nhập dành riêng cho quản trị viên (cấu hình trong file .env).
Luồng sự kiện chính	- Admin truy cập đường dẫn trang quản trị, giao diện Admin khởi chạy và hiển thị Form đăng nhập. - Admin điền Email và Password rồi nhấn "Login".

	3. Trình duyệt gọi API POST /api/user/admin. 4. Backend tiếp nhận dữ liệu, đối chiếu trực tiếp thông tin gửi lên với các biến môi trường ADMIN_EMAIL và ADMIN_PASSWORD. 5. Nếu thông tin khớp hoàn toàn, hệ thống khởi tạo và cấp phát một chuỗi JWT Token mang quyền hạn quản trị (Admin Privilege). 6. Giao diện Admin lưu Token này vào hệ thống và điều hướng người dùng vào trang Dashboard.
Luồng ngoại lệ	A1 - Sai thông tin quản trị: Nếu thông tin không khớp, Backend lập tức từ chối và trả về lỗi 401 Unauthorized. Frontend hiển thị Toast thông báo " <i>Thông tin đăng nhập không hợp lệ</i> " và chặn quyền truy cập.
Hậu điều kiện	Khách hàng nắm bắt được thông tin trạng thái của các giao dịch cá nhân.

13. Đặc tả Use Case 13: Xem Bảng thống kê (UC13)

Là trung tâm theo dõi hiệu suất kinh doanh, luồng này nhấn mạnh việc sử dụng Aggregation Pipeline để xử lý lượng lớn dữ liệu thành các KPI quan trọng (Doanh thu, MoM), kết hợp thư viện Recharts để trực quan hóa biểu đồ.

Bảng 13. Đặc tả Use Case 13 - Xem Bảng thống kê

Thành phần	Mô tả chi tiết
Tên Use Case	Xem bảng thống kê tổng quan (UC13)
Tác nhân	Admin
Mô tả	Cho phép Admin theo dõi các KPI thiết yếu (Doanh thu, Đơn hàng, Người dùng, Sản phẩm), phân tích tăng trưởng, theo dõi luồng phân bố trạng thái đơn (Order Pipeline), kiểm tra cảnh báo kho bãi, giám sát giao dịch mới nhất và xem danh sách Top 5 sản phẩm bán chạy.
Tiền điều kiện	Admin đang trong phiên đăng nhập hợp lệ (có JWT Token Admin).
Luồng sự kiện chính	1. Ngay khi Admin truy cập vào Dashboard, Frontend gọi API GET /api/dashboard/stats đính kèm Token. 2. Yêu cầu đi qua Middleware bảo vệ adminAuth.js để xác minh quyền hạn quản trị.

	3. Backend sử dụng các câu truy vấn phức tạp (Aggregation Framework) trong MongoDB để tính toán đồng thời: Tổng doanh thu, Tổng số đơn hàng, Số lượng người dùng, Tổng số sản phẩm đang hoạt động, Tỷ lệ tăng trưởng theo tháng (MoM), số lượng đơn theo từng trạng thái (Order Pipeline) và dữ liệu doanh thu 12 tháng trong năm (chỉ tính đơn "Delivered"). 4. Backend tiếp tục truy vấn danh sách Top 5 sản phẩm bán chạy nhất (Best Sellers), danh sách các sản phẩm sắp hết hàng (tồn kho của Size bất kỳ < 10) và lấy 10 đơn hàng phát sinh gần nhất. 5. Backend trả toàn bộ tập dữ liệu JSON về cho Client. 6. Frontend tiếp nhận và sử dụng thư viện Recharts để vẽ biểu đồ cột trực quan hóa doanh thu 12 tháng, đồng thời hiển thị các khối thẻ KPI, luồng trạng thái đơn hàng và các danh sách thống kê.
Luồng ngoại lệ	A1 - Vi phạm đặc quyền: Nếu một User thông thường cố tình gọi API này, Middleware adminAuth.js sẽ phát hiện sai phân quyền và trả về mã lỗi 403 Forbidden.
Hậu điều kiện	Bảng điều khiển được hiển thị đầy đủ thông tin để Admin ra quyết định kinh doanh.

14. Đặc tả Use Case 14: Quản lý Danh mục (UC14)

Hỗ trợ ban quản trị định hình cấu trúc phân loại hàng hóa. Đặc tả bao gồm nghiệp vụ tính toán doanh thu động (on-the-fly) cho từng danh mục để tự động sắp xếp danh sách một cách thông minh.

Bảng 14. Đặc tả Use Case 14 - Quản lý Danh mục

Thành phần	Mô tả chi tiết
Tên Use Case	Quản lý Danh mục sản phẩm (UC14)
Tác nhân	Admin
Mô tả	Cho phép Admin thiết lập cấu trúc phân loại hàng hóa bằng cách thêm mới hoặc xóa các danh mục.
Tiền điều kiện	Admin đang trong phiên đăng nhập hợp lệ (có JWT Token Admin).

Luồng sự kiện chính	1. Ngay khi Admin truy cập, giao diện gọi API GET /api/category/list. 2. Backend sử dụng Aggregation Pipeline để tính toán doanh thu động (on-the-fly) của từng danh mục và tự động sắp xếp danh sách trả về theo thứ tự doanh thu từ cao xuống thấp. 3. Admin điền tên danh mục mới vào form và nhấn "Thêm". 4. Frontend gọi API POST /api/category/add. 5. Backend tiếp nhận, lưu bản ghi Category mới vào CSDL MongoDB. 6. Trả kết quả thành công, Frontend hiển thị Toast "Đã thêm danh mục" và render lại danh sách.
Luồng ngoại lệ	A1 - Trùng lặp danh mục: Nếu tên danh mục gửi lên đã tồn tại, Backend trả lỗi 400. Frontend báo Toast "Danh mục này đã tồn tại".
Hậu điều kiện	Cấu trúc phân loại sản phẩm của hệ thống được cập nhật.

15. Đặc tả Use Case 15: Quản lý Sản phẩm (UC15)

Bao quát toàn bộ vòng đời của hàng hóa trên hệ thống. Đặc tả tích hợp luồng xử lý hình ảnh qua dịch vụ đám mây Cloudinary và nghiệp vụ bổ sung tồn kho (Restock) an toàn bằng toán tử cộng dồn.

Bảng 15. Đặc tả Use Case 15 - Quản lý Sản phẩm

Thành phần	Mô tả chi tiết
Tên Use Case	Quản lý Sản phẩm và Nhập kho (UC15)
Tác nhân	Admin
Mô tả	Cho phép Admin thêm mới sản phẩm kèm upload hình ảnh, chỉnh sửa, xóa và bổ sung số lượng tồn kho theo từng biến thể kích cỡ.
Tiền điều kiện	Admin đang trong phiên đăng nhập hợp lệ (có JWT Token Admin).
Luồng sự kiện chính	1. Thêm sản phẩm: Admin điền đầy đủ thông tin sản phẩm, chọn danh mục, thiết lập số lượng tồn kho ban đầu cho các Size và đính kèm các tệp hình ảnh. 2. Frontend đóng gói dữ liệu dạng multipart/form-data và gọi API POST /api/product/add.

	3. Tại Backend, Middleware Multer tiếp nhận và phân tích các tệp ảnh, tự động đẩy lên hệ thống Cloudinary. 4. Cloudinary trả về các đường dẫn (URL) ảnh an toàn, Backend dùng các URL này kết hợp với thông tin text để tạo bản ghi Product mới trong MongoDB. 5. Nhập hàng (Restock): Khi có hàng mới về, Admin mở danh sách sản phẩm, chọn sản phẩm và nhập số lượng cần bổ sung vào từng Size. 6. Gọi API POST /api/product/update-stock. Backend sử dụng toán tử \$inc cộng dồn chính xác số lượng mới vào trường sizesStock đang có và trả kết quả thành công.
Luồng ngoại lệ	A1 - Lỗi Upload ảnh: Tại bước upload, nếu định dạng ảnh không hợp lệ hoặc Cloudinary timeout, Backend hủy bỏ tiến trình tạo sản phẩm và trả lỗi " <i>Không thể upload hình ảnh</i> ".
Hậu điều kiện	Dữ liệu sản phẩm và số lượng tồn kho thực tế được đồng bộ hóa lên hệ thống.

16. Đặc tả Use Case 16: Quản lý Đơn hàng (UC16)

Nhằm giám sát chặt chẽ tiến trình giao hàng, đặc tả này không chỉ mô tả việc cập nhật trạng thái mà còn làm bật lên cơ chế Trigger tự động hoàn trả số lượng sản phẩm vào kho khi có đơn hàng bị hủy.

Bảng 16. Đặc tả Use Case 16 - Quản lý Đơn hàng

Thành phần	Mô tả chi tiết
Tên Use Case	Quản lý và cập nhật trạng thái đơn hàng (UC16)
Tác nhân	Admin
Mô tả	Cho phép Admin theo dõi tiến trình giao hàng và cập nhật các trạng thái. Nổi bật với luồng tự động hoàn kho khi hủy đơn.
Tiền điều kiện	Admin đang trong phiên đăng nhập hợp lệ (có JWT Token Admin).
Luồng sự kiện chính	1. Admin truy cập trang Quản lý Đơn hàng, giao diện hiển thị danh sách toàn bộ giao dịch.

	- Admin chọn một đơn hàng cụ thể và thay đổi trạng thái giao hàng từ menu thả xuống. - Trình duyệt gọi API POST /api/order/status. - Backend tiếp nhận, đối chiếu trạng thái gửi lên với cấu trúc Enum Validation được định nghĩa khắt khe trong Model. - Hệ thống hỗ trợ Payload Mapping (chuyển ID số thành chuỗi Enum) để đảm bảo linh hoạt. - Backend cập nhật trạng thái mới vào bản ghi Order trong CSDL và trả về kết quả thành công.
Luồng ngoại lệ	A1 - Hủy đơn & Hoàn kho tự động: Nếu Admin chuyển trạng thái sang "Cancelled", sau khi cập nhật CSDL, Backend kích hoạt Trigger nghiệp vụ lặp qua toàn bộ mảng sản phẩm trong đơn hàng bị hủy. Backend tự động gọi lệnh \$inc để cộng dồn trả lại số lượng vào trường sizesStock. Hệ thống báo <i>"Đã hủy đơn và tự động hoàn trả tồn kho thành công"</i>.
Hậu điều kiện	Trạng thái đơn hàng được cập nhật, dữ liệu kho bãi tự động bảo toàn nguyên vẹn.

17. Đặc tả Use Case 17: Quản lý Người dùng (UC17)

Đảm bảo tính toàn vẹn dữ liệu và an ninh hệ thống. Đặc tả làm rõ kỹ thuật tìm kiếm/phân trang Server-side, kiểm soát trạng thái tài khoản và đặc biệt là nghiệp vụ xóa dữ liệu liên đới (Cascade Delete).

Bảng 17. Đặc tả Use Case 17 - Quản lý Người dùng

Thành phần	Mô tả chi tiết
Tên Use Case	Quản lý tài khoản người dùng (UC17)
Tác nhân	Admin
Mô tả	Cho phép Admin giám sát danh sách khách hàng, khóa/mở khóa tài khoản hoặc xóa vĩnh viễn tài khoản kèm toàn bộ dữ liệu liên đới (Cascade Delete).
Tiền điều kiện	Admin đang trong phiên đăng nhập hợp lệ (có JWT Token Admin).

Luồng sự kiện chính	1. Admin truy cập mục Quản lý Người dùng. Frontend gọi API lấy danh sách kết hợp kỹ thuật Debounce (độ trễ 400ms) trên thanh tìm kiếm. 2. Backend sử dụng \$lookup, \$facet, \$regex trong một Pipeline duy nhất để Tìm kiếm Server-side, Phân trang Server-side và tính toán động cột Tổng chi tiêu. 3. Trả dữ liệu về cho phép Admin nhấp để sắp xếp (Sort) tăng/giảm. 4. Khi phát hiện tài khoản bất thường, Admin nhấp đổi trạng thái từ Active sang Blocked. 5. Backend ghi nhận thay đổi vào CSDL, người dùng này lập tức bị chặn thao tác gọi API.
Luồng ngoại lệ	A1 - Xóa Cascade: Nếu Admin chọn "Xóa tài khoản", hệ thống yêu cầu xác nhận. Backend thực thi lệnh xóa bản ghi User khỏi MongoDB. Luồng Cascade Delete kích hoạt, tự động quét và xóa toàn bộ bản ghi thuộc collection orders và newsletters có liên kết với userId vừa xóa. Hệ thống thông báo <i>"Đã xóa tài khoản và dọn dẹp dữ liệu thành công"</i>.
Hậu điều kiện	Tài khoản vi phạm bị loại bỏ hoàn toàn, CSDL sạch sẽ không chứa dữ liệu mồ côi (Orphan data).

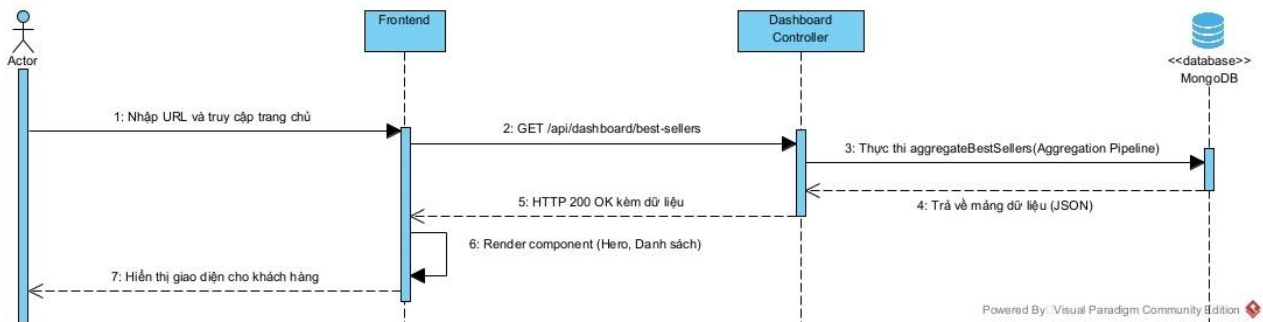
3.1.5 Biểu đồ trình tự (Sequence Diagram)

1. Biểu đồ trình tự: Luồng Xem trang chủ (UC01)

Luồng này mô tả quá trình khách hàng truy cập vào hệ thống và ứng dụng tự động tải dữ liệu danh sách sản phẩm bán chạy để hiển thị.

- Các đối tượng tham gia:
 - Khách hàng: Người dùng truy cập.
 - Frontend (React): Giao diện ứng dụng máy khách (Cổng 5173).
 - Dashboard Controller: Bộ điều khiển xử lý dữ liệu thống kê.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khách hàng nhập URL và truy cập vào trang chủ.

- Frontend khởi chạy các component cốt lõi và tự động gửi request GET /api/dashboard/best-sellers lên Backend.
- Dashboard Controller tiếp nhận và thực thi hàm aggregateBestSellers, gửi lệnh truy vấn gộp (Aggregation) vào MongoDB.
- MongoDB tính toán doanh số thực tế và trả về mảng dữ liệu Top 5 sản phẩm (JSON).
- Dashboard Controller gửi phản hồi HTTP 200 OK kèm dữ liệu về cho Frontend.
- Frontend tiếp nhận, render các component (Hero Banner, Danh sách sản phẩm) và hiển thị giao diện cho khách hàng.

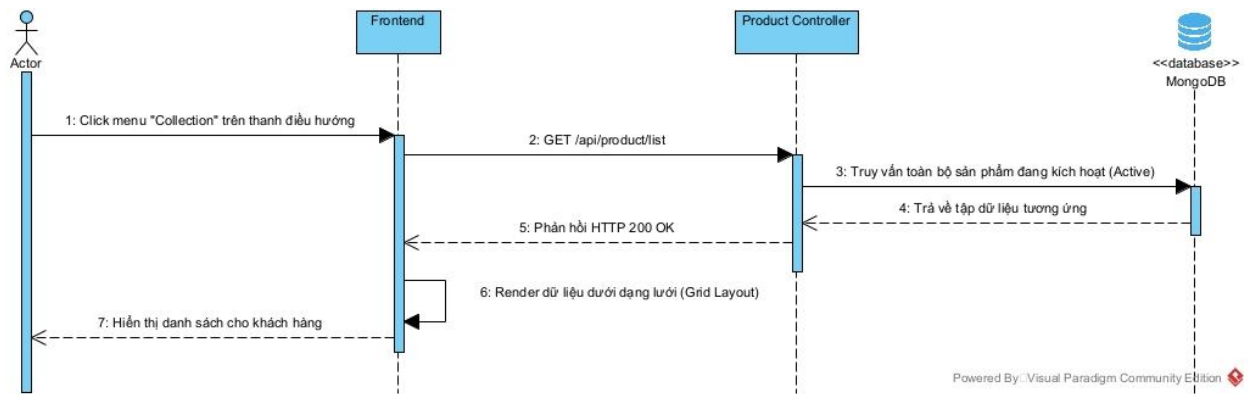


Hình 2 - Sequence Diagram 1 - Xem trang chủ

2. Biểu đồ trình tự: Luồng Khám phá sản phẩm (UC02)

Luồng này mô tả quá trình khách hàng duyệt danh mục sản phẩm toàn cục để xem toàn bộ hàng hóa đang kinh doanh.

- Các đối tượng tham gia:
 - Khách hàng: Người dùng thao tác.
 - Frontend (React): Giao diện ứng dụng.
 - Product Controller: Bộ điều khiển xử lý sản phẩm.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khách hàng click vào menu "Collection" trên thanh điều hướng.
 - Frontend gửi request GET /api/product/list lên Backend.
 - Product Controller truy vấn lấy toàn bộ sản phẩm đang được kích hoạt (Active) từ MongoDB.
 - MongoDB trả về tập dữ liệu tương ứng.
 - Product Controller phản hồi HTTP 200 OK cho Frontend.
 - Frontend render dữ liệu dưới dạng lưới (Grid Layout) và hiển thị cho khách hàng.

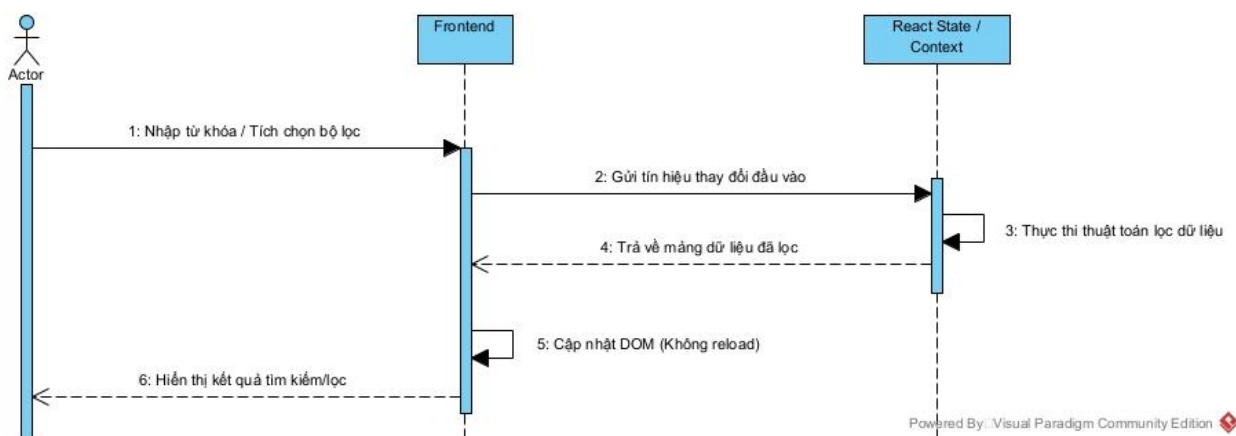


Hình 3 - Sequence Diagram 2 - Khám phá sản phẩm

3. Biểu đồ trình tự: Luồng Tìm kiếm và lọc sản phẩm (UC03)

Luồng này diễn ra chủ yếu ở Client-side (React) nhờ việc dữ liệu đã được nạp từ trước, giúp tối ưu hiệu năng (SPA) mà không cần gọi API nhiều lần.

- Các đối tượng tham gia:
 - Khách hàng: Người dùng tìm kiếm.
 - Frontend (React): Component hiển thị giao diện.
 - React State / Context: Bộ quản lý trạng thái luồng dữ liệu của React.
- Trình tự tương tác:
 - Khách hàng nhập từ khóa vào thanh tìm kiếm hoặc tích chọn các ô lọc danh mục.
 - Frontend gửi tín hiệu thay đổi đầu vào đến React State.
 - React State ghi nhận và thực thi thuật toán lọc ngay trên mảng dữ liệu sản phẩm hiện hành.
 - React State trả về mảng dữ liệu mới (đã được lọc).
 - Frontend cập nhật lại DOM (không tải lại trang) và hiển thị kết quả cho khách hàng.

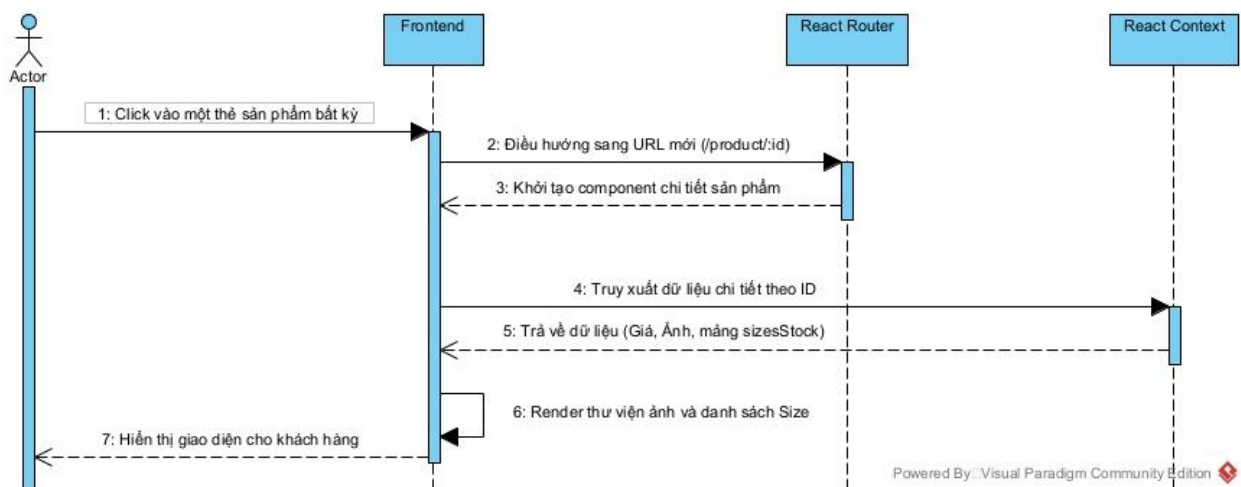


Hình 4 - Sequence Diagram 3 - Tìm kiếm và lọc sản phẩm

4. Biểu đồ trình tự: Luồng Xem chi tiết sản phẩm (UC04)

Khách hàng chọn xem chi tiết một sản phẩm để kiểm tra các thông tin mở rộng như hình ảnh, giá và các biến thể tồn kho (Size).

- Các đối tượng tham gia:
 - Khách hàng: Người dùng xem sản phẩm.
 - Frontend (React): Giao diện tương tác.
 - React Router: Bộ định hướng URL của React.
 - React Context: Bộ nhớ chứa dữ liệu toàn cục.
- Trình tự tương tác:
 - Khách hàng click vào một thẻ sản phẩm bất kỳ.
 - Frontend yêu cầu React Router điều hướng sang URL mới (Ví dụ: /product/:id).
 - React Router xác nhận và khởi tạo component chi tiết sản phẩm.
 - Frontend truy xuất dữ liệu chi tiết của sản phẩm đó trong React Context thông qua ID.
 - React Context trả về dữ liệu tương ứng (Giá, Ảnh, mảng sizesStock).
 - Frontend render thư viện ảnh, danh sách Size và hiển thị cho khách hàng.



Hình 5 - Sequence Diagram 4 - Xem chi tiết sản phẩm

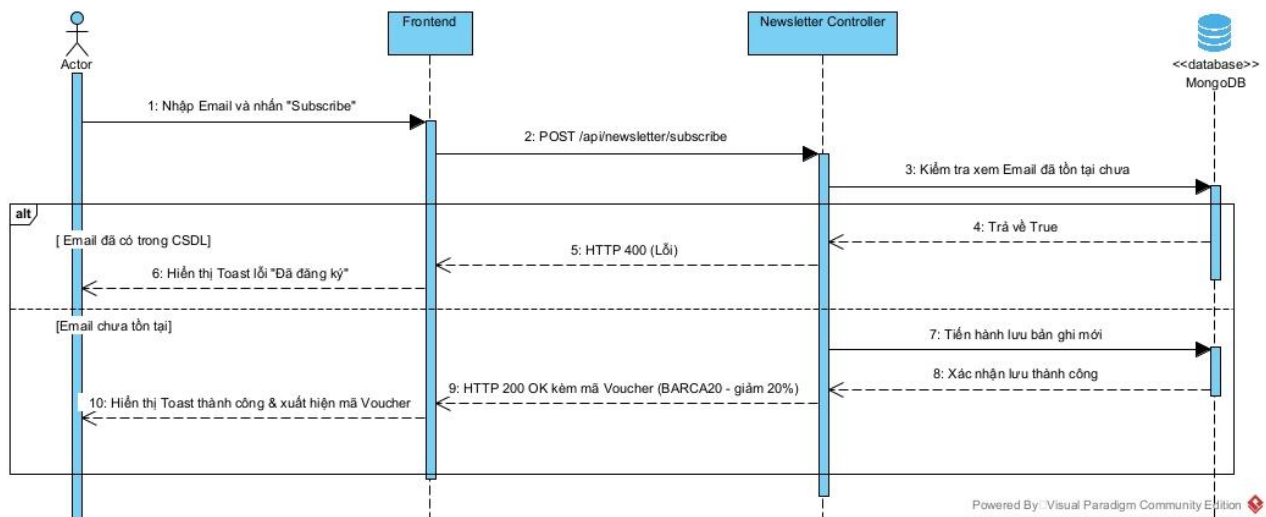
5. Biểu đồ trình tự: Luồng Đăng ký nhận bản tin (UC05)

Luồng nhập email đăng ký nhận tin tức và hệ thống cấp phát mã Voucher "BARCA20" để khuyến khích mua sắm.

- Các đối tượng tham gia:
 - Khách hàng: Người nhập email.
 - Frontend (React): Form đăng ký bản tin.
 - Newsletter Controller: Bộ điều khiển xử lý logic đăng ký.
 - MongoDB: Cơ sở dữ liệu.

▪ Trình tự tương tác:

- Khách hàng nhập Email và nhấn "Subscribe".
- Frontend gửi request POST /api/newsletter/subscribe lên Backend.
- Newsletter Controller kiểm tra xem Email này đã tồn tại trong MongoDB chưa.
- (*Luồng lỗi*): Nếu Email đã có, CSDL trả về True. Controller báo lỗi 400 và Frontend hiển thị Toast lỗi "Đã đăng ký".
- Nếu Email chưa có, Controller tiến hành lưu bản ghi mới vào MongoDB.
- MongoDB xác nhận lưu thành công.
- Controller phản hồi HTTP 200 OK kèm mã Voucher (BARCA20 - giảm 20%).
- Frontend hiển thị Toast thông báo thành công và xuất hiện mã Voucher cho khách hàng.



Hình 6 - Sequence Diagram 5 - Đăng ký nhận bản tin

6. Biểu đồ trình tự: Luồng Đăng ký tài khoản (UC06)

Quá trình tạo tài khoản mới, mã hóa mật khẩu an toàn bằng Bcrypt và cấp phát phiên đăng nhập thông qua JWT Token.

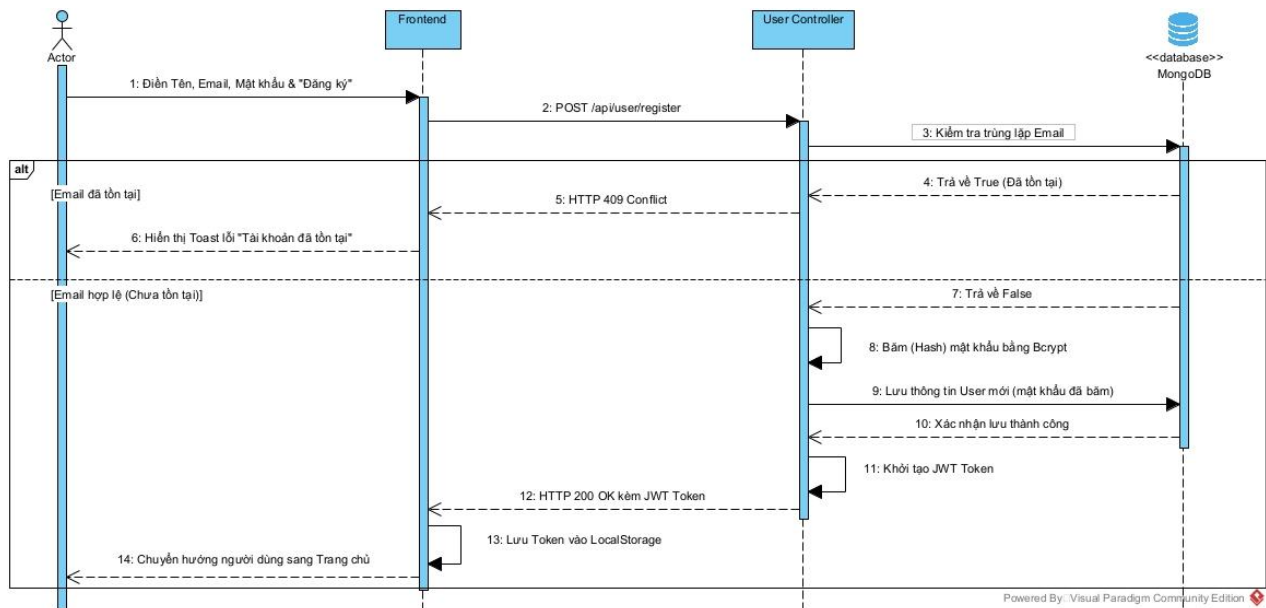
▪ Các đối tượng tham gia:

- Khách vãng lai: Người chưa có tài khoản.
- Frontend (React): Form đăng ký.
- User Controller: Bộ điều khiển xử lý tài khoản.
- MongoDB: Cơ sở dữ liệu.

▪ Trình tự tương tác:

- Khách vãng lai điền Tên, Email, Mật khẩu và nhấn "Đăng ký".
- Frontend gọi API POST /api/user/register gửi dữ liệu lên Backend.

- User Controller kiểm tra trùng lặp Email trong MongoDB.
- (*Luồng lỗi*): Nếu Email đã tồn tại, trả lỗi 409 Conflict. Frontend hiển thị lỗi.
- Nếu hợp lệ, User Controller sử dụng Bcrypt băm (Hash) mật khẩu.
- User Controller lưu thông tin User mới (mật khẩu đã băm) vào MongoDB.
- User Controller khởi tạo JWT Token chứa thông tin định danh.
- Gửi phản hồi HTTP 200 kèm JWT Token về Frontend.
- Frontend lưu Token vào LocalStorage và chuyển hướng người dùng sang Trang chủ.



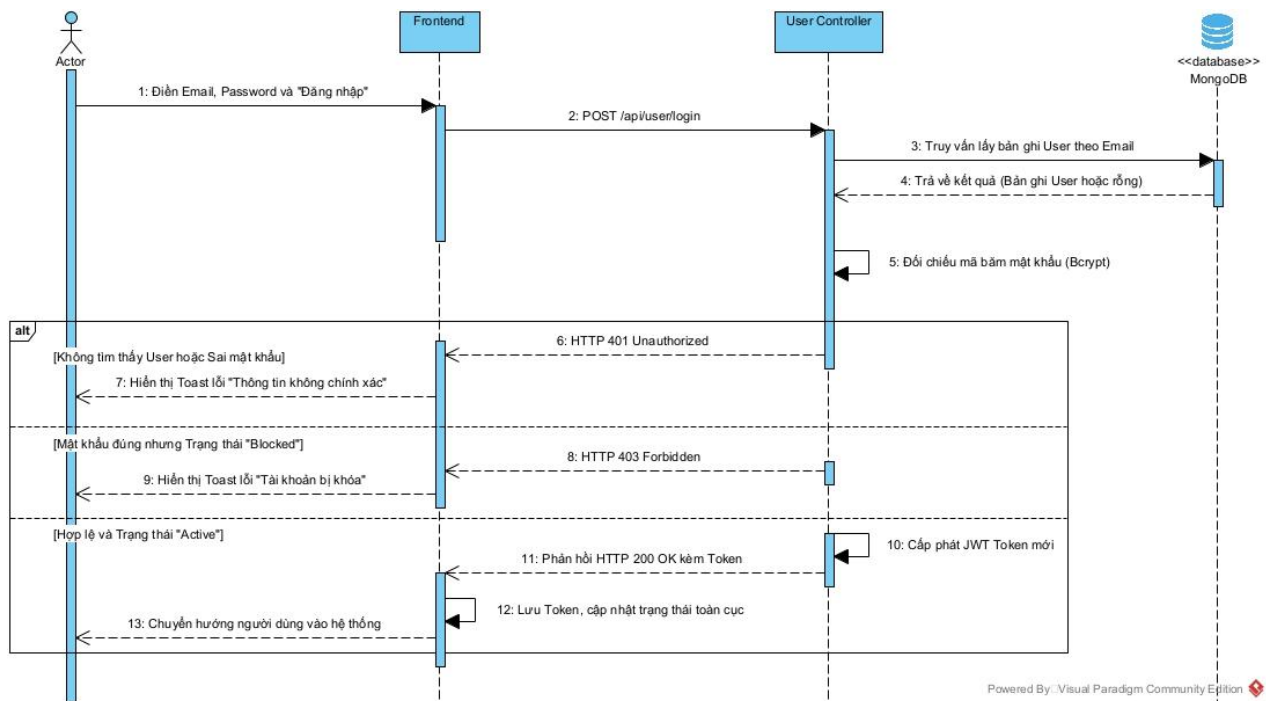
Hình 7 - Sequence Diagram 6 - Đăng ký tài khoản

7. Biểu đồ trình tự: Luồng Đăng nhập hệ thống (UC07)

Quá trình xác thực thông tin đăng nhập, đối chiếu mã băm và kiểm tra trạng thái khóa tài khoản trước khi cấp quyền.

- Các đối tượng tham gia:
 - Khách hàng: Người dùng đã có tài khoản.
 - Frontend (React): Form đăng nhập.
 - User Controller: Bộ điều khiển xử lý tài khoản.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khách hàng điền Email, Password và nhấn "Đăng nhập".
 - Frontend gọi API POST /api/user/login.
 - User Controller truy vấn MongoDB để lấy bản ghi User theo Email.

- (Luồng lỗi 1): Nếu không tìm thấy hoặc mật khẩu (so sánh bằng Bcrypt) không khớp, trả lỗi 401.
- (Luồng lỗi 2): Nếu mật khẩu đúng nhưng trạng thái tài khoản là "Blocked", trả lỗi 403 Forbidden.
- Nếu hợp lệ và trạng thái "Active", User Controller cấp phát JWT Token mới.
- Phản hồi HTTP 200 kèm Token về Frontend.
- Frontend lưu Token, cập nhật trạng thái toàn cục và chuyển hướng người dùng.



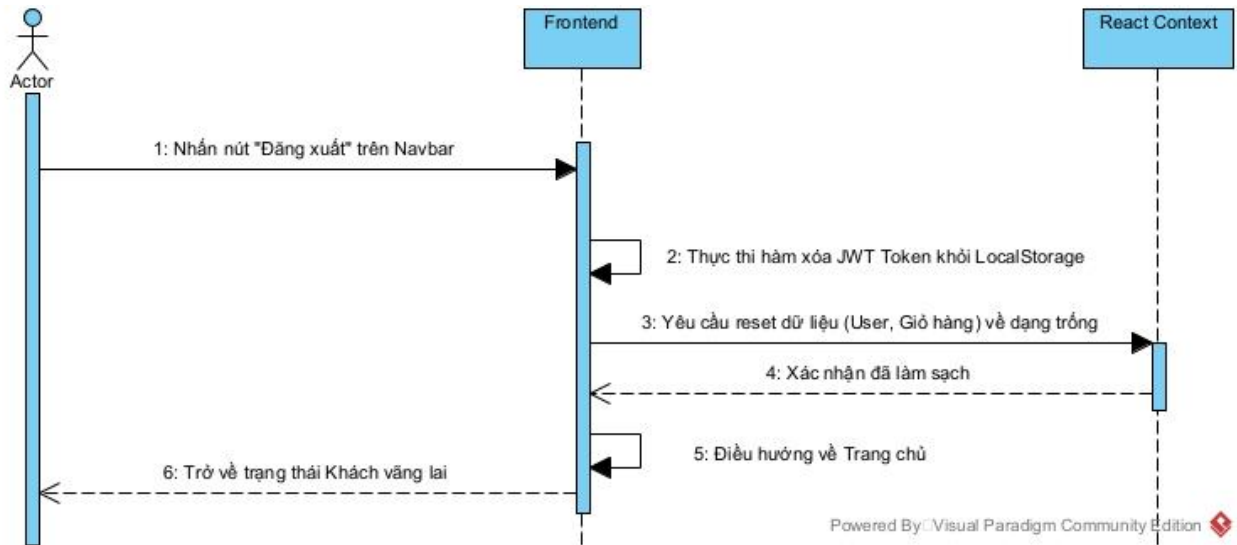
Hình 8 - Sequence Diagram 7 - Đăng nhập hệ thống

8. Biểu đồ trình tự: Luồng Đăng xuất (UC08)

Luồng đăng xuất diễn ra hoàn toàn ở phía Client để xóa phiên làm việc và bảo mật thông tin.

- Các đối tượng tham gia:
 - Khách hàng: Người dùng muốn thoát.
 - Frontend (React): Giao diện hệ thống.
 - React Context: Bộ nhớ quản lý State toàn cục.
- Trình tự tương tác:
 - Khách hàng nhấn nút "Đăng xuất" trên Navbar.
 - Frontend thực thi hàm xóa JWT Token khỏi LocalStorage.
 - Frontend yêu cầu React Context reset toàn bộ dữ liệu (Thông tin User, Giỏ hàng) về dạng trống.
 - React Context xác nhận đã làm sạch.

- Frontend điều hướng khách hàng về lại Trang chủ dưới tư cách Khách vắng lai.

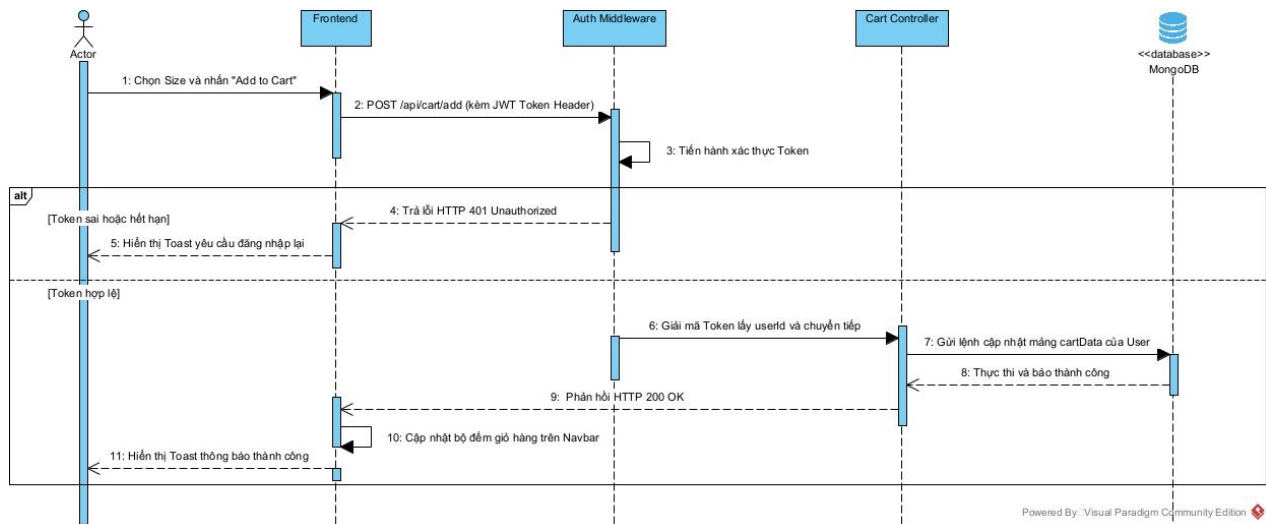


Hình 9 - Sequence Diagram 8 - Đăng xuất

9. Biểu đồ trình tự: Luồng Thêm sản phẩm vào giỏ hàng (UC09)

Tương tác có yêu cầu bảo mật, sử dụng Auth Middleware để xác định danh tính người dùng trước khi lưu giỏ hàng vào CSDL.

- Các đối tượng tham gia:
 - Khách hàng: Người thực hiện thêm hàng.
 - Frontend (React): Giao diện sản phẩm.
 - Auth Middleware: Lóp rào chắn xác thực JWT.
 - Cart Controller: Bộ điều khiển xử lý giỏ hàng.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khách hàng chọn Size mong muốn và nhấn "Add to Cart".
 - Frontend gọi API POST /api/cart/add kèm theo JWT Token trong Header.
 - Auth Middleware tiếp nhận và tiến hành xác thực Token.
 - (Luồng lỗi): Nếu Token sai hoặc hết hạn, trả lỗi 401 Unauthorized yêu cầu đăng nhập lại.
 - Nếu hợp lệ, Middleware giải mã Token lấy userId và chuyển tiếp cho Cart Controller.
 - Cart Controller gửi lệnh cập nhật mảng cartData của User tương ứng vào MongoDB.
 - MongoDB thực thi và báo thành công.
 - Cart Controller phản hồi HTTP 200 OK.
 - Frontend hiển thị Toast thông báo thành công và cập nhật bộ đếm giỏ hàng trên thanh Navbar.



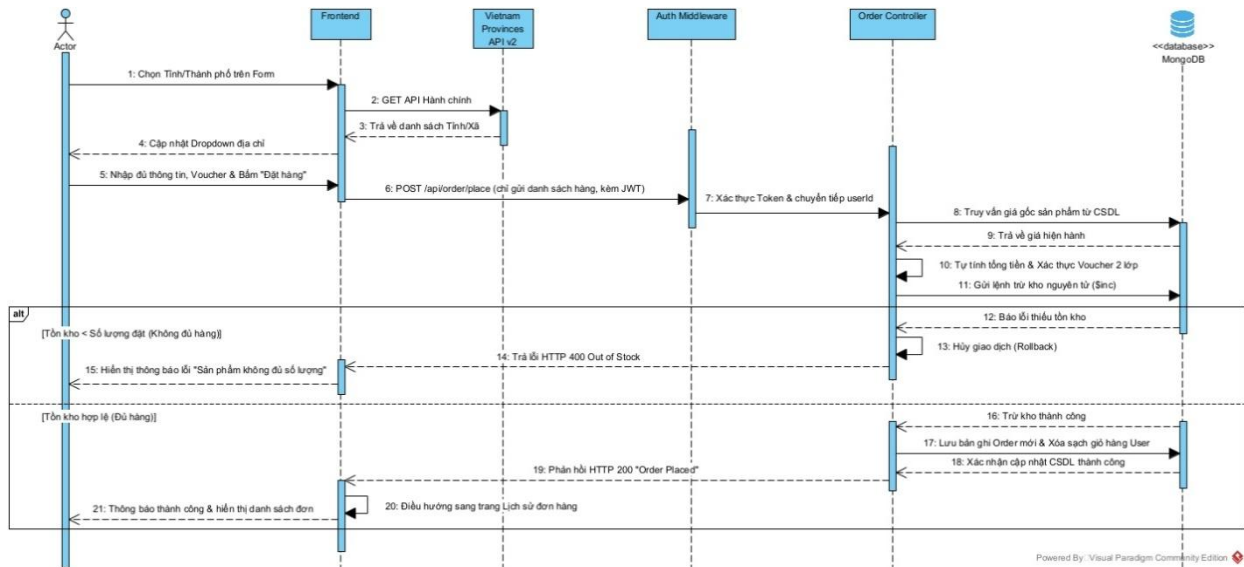
Hình 10 - Sequence Diagram 9 - Quản lý giỏ hàng

10. Biểu đồ trình tự: Luồng Đặt hàng và Thanh toán (UC10)

Luồng nghiệp vụ cốt lõi tích hợp kiểm tra giá phía máy chủ (Server-side Pricing), gọi API địa chỉ hành chính, xác thực voucher 2 lớp và trừ kho nguyên tử (\$inc).

- Các đối tượng tham gia:
 - Khách hàng: Người dùng đặt mua.
 - Frontend (React): Form thanh toán.
 - Vietnam Provinces API: API cung cấp dữ liệu hành chính.
 - Auth Middleware: Lớp xác thực.
 - Order Controller: Bộ điều khiển xử lý đơn hàng.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khách hàng chọn Tỉnh/Thành phố trên Form giao hàng.
 - Frontend tự động gọi GET tới Vietnam Provinces API v2 để lấy danh sách Quận/Huyện/Xã tương ứng và cập nhật giao diện (Cascading Dropdown).
 - Khách hàng điền nốt thông tin, nhập mã Voucher (nếu có) và nhấn "Đặt hàng".
 - Frontend gọi API POST /api/order/place (chỉ gửi danh sách hàng, bỏ qua tổng tiền) kèm JWT Token.
 - Auth Middleware xác thực Token và chuyển tiếp userId cho Order Controller.
 - Order Controller lấy giá gốc hiện hành của sản phẩm từ MongoDB để tự tính tổng tiền (Server-side).
 - Order Controller thực hiện xác thực 2 lớp cho mã Voucher (tồn tại & chưa nằm trong mảng usedVouchers của User), sau đó tính toán lại tổng tiền.
 - Order Controller gửi lệnh cập nhật tồn kho bằng toán tử \$inc vào MongoDB.

- (Luồng lỗi): Nếu MongoDB báo không đủ hàng (Tồn kho < Số lượng đặt), Controller hủy giao dịch (Rollback) và trả lỗi 400 Out of Stock.
- Nếu hợp lệ, tồn kho được trừ thành công. Order Controller lưu bản ghi Order mới, lưu mã Voucher vào hồ sơ User và xóa sạch Giỏ hàng.
- Phản hồi HTTP 200 "Order Placed" về Frontend.
- Frontend thông báo thành công và điều hướng khách hàng sang trang Lịch sử đơn hàng.



Hình 11 - Sequence Diagram 10 - Đặt hàng và Thanh toán

11. Biểu đồ trình tự: Luồng Xem lịch sử đơn hàng (UC11)

Luồng này giúp khách hàng tra cứu lại các giao dịch cá nhân và theo dõi tiến trình xử lý đơn hàng theo thời gian thực.

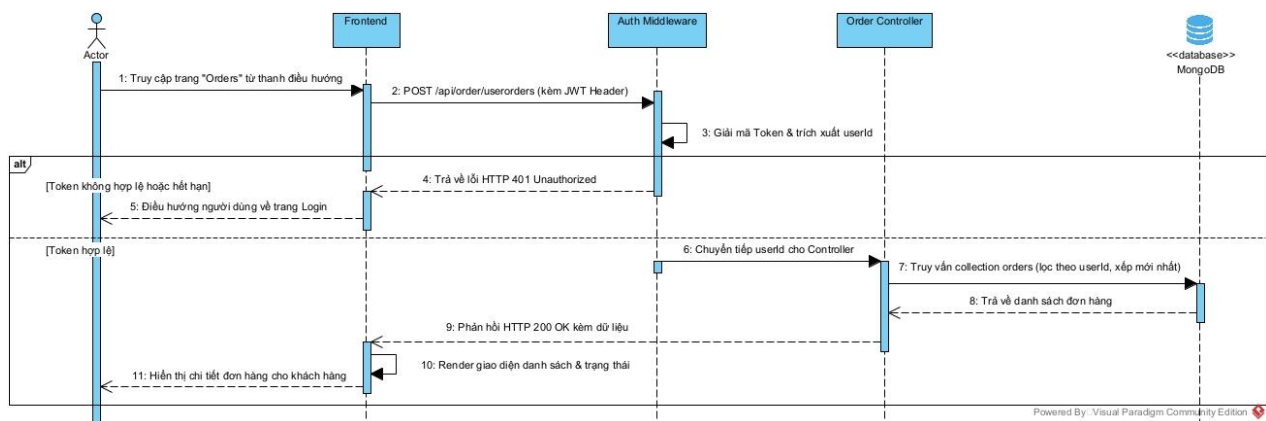
■ Các đối tượng tham gia:

- Khách hàng: Người dùng muốn tra cứu đơn hàng.
- Frontend (React): Giao diện ứng dụng máy khách.
- Auth Middleware: Lớp rào chắn xác thực JWT.
- Order Controller: Bộ điều khiển xử lý dữ liệu đơn hàng.
- MongoDB: Cơ sở dữ liệu.

■ Trình tự tương tác:

- Khách hàng truy cập trang "Orders" từ thanh điều hướng.
- Frontend gọi API POST /api/order/userorders và đính kèm JWT Token vào Header Authorization.
- Yêu cầu đi qua Middleware auth.js. Middleware giải mã Token, trích xuất thông tin userId và cho phép đi tiếp.

- (*Luồng lỗi*): Nếu Token không hợp lệ hoặc đã hết hạn, Backend trả về lỗi 401. Frontend điều hướng người dùng về trang Login.
- Nếu hợp lệ, Order Controller tiếp nhận userId và truy vấn collection orders trong MongoDB, lọc ra danh sách các đơn hàng thuộc sở hữu của User này, sắp xếp theo thời gian mới nhất.
- MongoDB trả về danh sách đơn hàng cho Controller.
- Controller gửi phản hồi HTTP 200 OK kèm dữ liệu về Client.
- Frontend render giao diện danh sách, hiển thị trạng thái giao hàng chi tiết cho khách hàng.



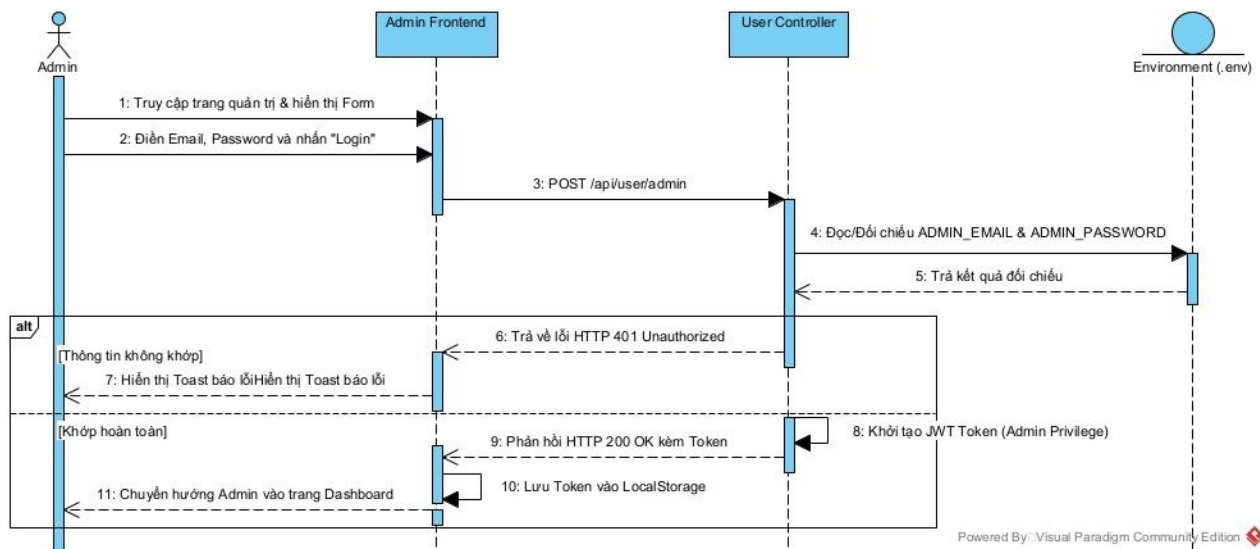
Hình 12 - Sequence Diagram 11 - Xem lịch sử đơn hàng

12. Biểu đồ trình tự: Luồng Xác thực Quản trị (UC12)

Luồng xác thực độc lập dành riêng cho ban quản trị, đối chiếu thông tin trực tiếp với các biến môi trường để cấp quyền cao nhất.

- Các đối tượng tham gia:
 - Admin: Quản trị viên hệ thống.
 - Admin Frontend: Bảng điều khiển quản trị (Cổng 5174).
 - User Controller: Bộ điều khiển xử lý logic xác thực.
 - Environment (.env): Nơi lưu trữ thông tin cấu hình bảo mật.
- Trình tự tương tác:
 - Admin truy cập đường dẫn trang quản trị và hiển thị Form đăng nhập.
 - Admin điền Email và Password rồi nhấn "Login".
 - Admin Frontend gửi request POST /api/user/admin lên Backend.
 - User Controller tiếp nhận dữ liệu và đối chiếu trực tiếp với các biến môi trường ADMIN_EMAIL và ADMIN_PASSWORD lấy từ file .env.
 - (*Luồng lỗi*): Nếu thông tin không khớp, Backend từ chối và trả về lỗi 401 Unauthorized. Frontend hiển thị Toast báo lỗi.

- Nếu khớp hoàn toàn, User Controller khởi tạo một chuỗi JWT Token mang đặc quyền quản trị (Admin Privilege).
- Gửi phản hồi HTTP 200 OK kèm Token về Admin Frontend.
- Admin Frontend lưu Token vào LocalStorage và chuyển hướng Admin vào trang Dashboard.



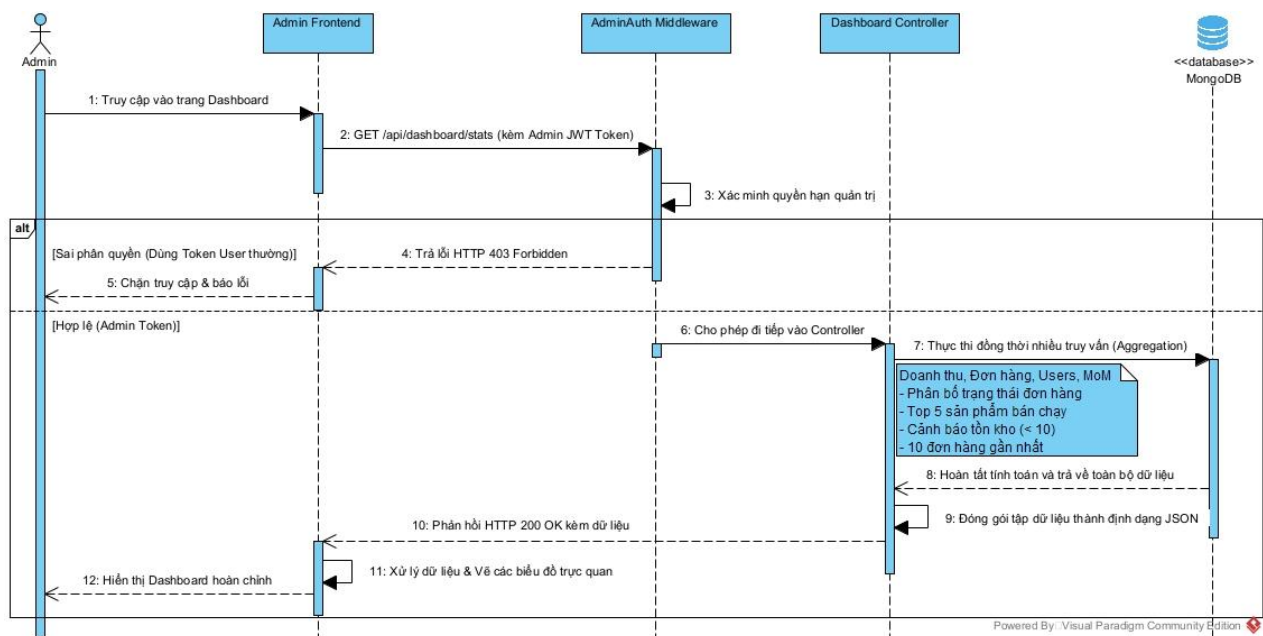
Hình 13 - Sequence Diagram 12 - Quản trị viên

13. Biểu đồ trình tự: Luồng Xem Bảng thống kê Dashboard (UC13)

Luồng xử lý dữ liệu quy mô lớn, tính toán các chỉ số kinh doanh thiết yếu bằng Aggregation Framework của MongoDB.

- Các đối tượng tham gia:
 - Admin: Quản trị viên.
 - Admin Frontend: Giao diện bảng điều khiển.
 - AdminAuth Middleware: Lớp kiểm tra quyền quản trị.
 - Dashboard Controller: Bộ điều khiển xử lý tính toán thống kê.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Khi Admin truy cập vào Dashboard, Admin Frontend gọi API GET /api/dashboard/stats đính kèm Admin JWT Token.
 - Middleware adminAuth.js xác minh quyền hạn quản trị. Nếu sai phân quyền (dùng token user thường), trả lỗi 403 Forbidden.
 - Nếu hợp lệ, Dashboard Controller thực thi đồng thời nhiều câu truy vấn MongoDB:
 - Tính Tổng doanh thu, Tổng đơn hàng, Số lượng người dùng, Tỷ lệ tăng trưởng theo tháng (MoM).

- Lọc và nhóm dữ liệu để vẽ Biểu đồ phân bố trạng thái đơn hàng.
 - Truy vấn thống kê Top 5 sản phẩm bán chạy nhất dựa trên doanh số.
 - Quét cảnh báo tồn kho: Tìm các sản phẩm có bất kỳ biến thể kích cỡ nào chứa số lượng < 10.
 - Lấy danh sách 10 đơn hàng phát sinh gần nhất.
- MongoDB hoàn tất tính toán và trả về toàn bộ dữ liệu.
 - Controller đóng gói thành định dạng JSON và phản hồi HTTP 200 về Admin Frontend.
 - Admin Frontend tiếp nhận và vẽ các biểu đồ trực quan lên màn hình.



Hình 14 - Sequence Diagram 13 - Xem Bảng thống kê

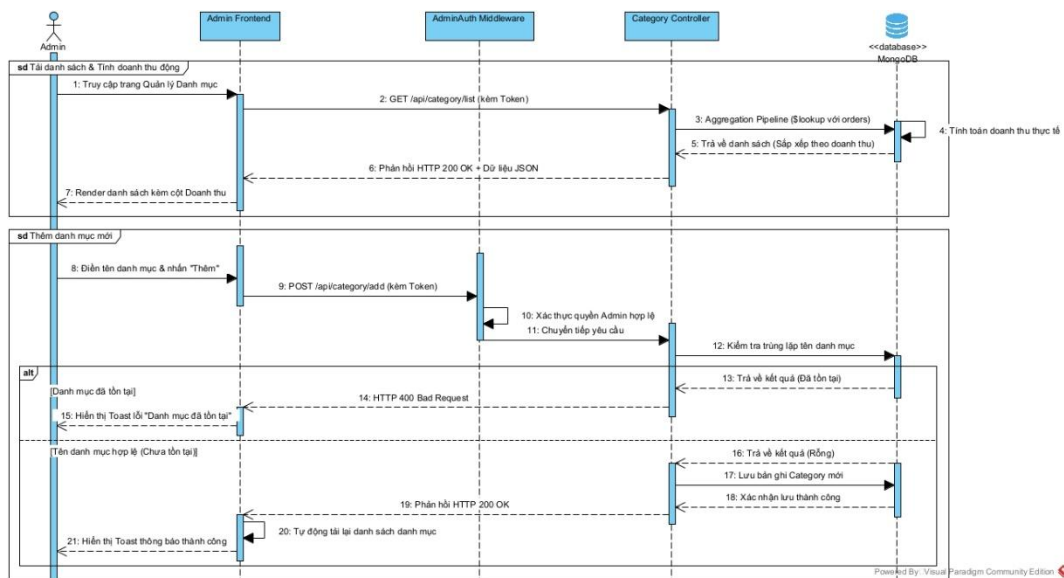
14. Biểu đồ trình tự: Luồng Quản lý Danh mục (UC14)

Luồng cho phép Admin mở rộng hoặc cấu trúc lại cây phân loại hàng hóa của hệ thống.

- Các đối tượng tham gia:
 - Admin: Quản trị viên.
 - Admin Frontend: Giao diện Admin.
 - AdminAuth Middleware: Lớp rào chắn quyền Admin.
 - Category Controller: Bộ điều khiển danh mục.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - (Luồng Tải danh sách): Khi truy cập trang, Admin Frontend gọi API GET /api/category/list. Backend sử dụng Aggregation Pipeline (\$lookup với collection

orders) để tính toán doanh thu động (on-the-fly) và trả về danh sách đã sắp xếp theo doanh thu.

- (Luồng Thêm danh mục): Admin điền tên danh mục mới và nhấn "Thêm".
- Admin Frontend gọi API POST /api/category/add đính kèm Token.
- Yêu cầu đi qua AdminAuth Middleware để xác thực.
- Category Controller kiểm tra trùng lặp tên trong CSDL.
- (Luồng lỗi): Nếu danh mục đã tồn tại, trả lỗi 400. Frontend báo Toast "Danh mục đã tồn tại".
- Nếu hợp lệ, Controller lưu bản ghi Category mới vào MongoDB và trả về HTTP 200 OK. Frontend tự động tải lại danh sách.



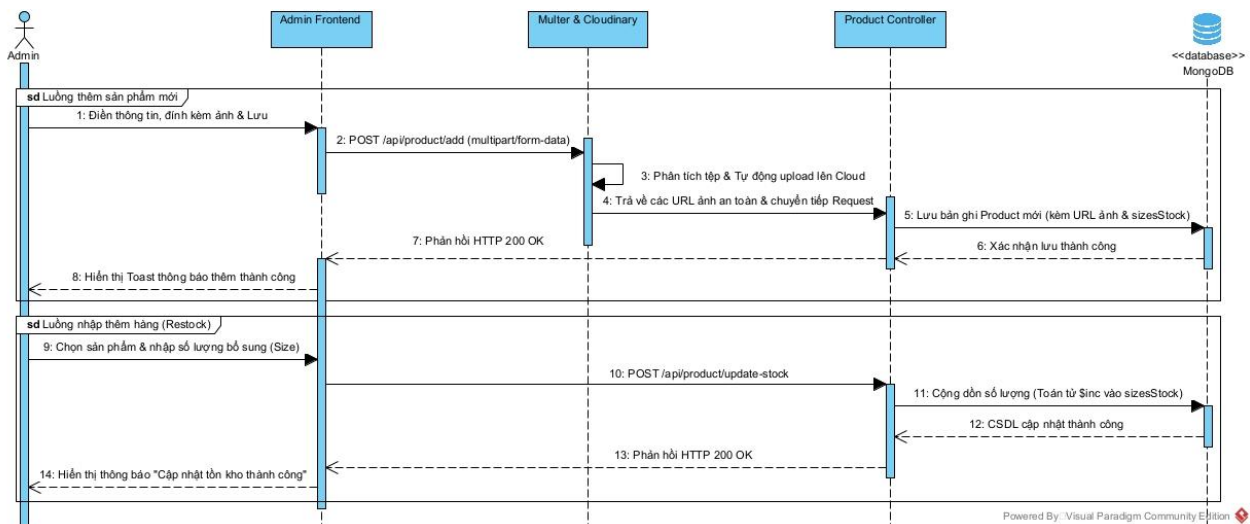
Hình 15 - Sequence Diagram 14 - Quản lý Danh mục

15. Biểu đồ trình tự: Luồng Quản lý Sản phẩm và Nhập kho (UC15)

Luồng xử lý phức tạp bao gồm tương tác với dịch vụ đám mây bên thứ ba (Cloudinary) và cập nhật số lượng tồn kho nguyên tử.

- Các đối tượng tham gia:
 - Admin: Quản trị viên.
 - Admin Frontend: Giao diện thêm sản phẩm.
 - Multer & Cloudinary: Middleware xử lý file và hệ thống lưu trữ ảnh.
 - Product Controller: Bộ điều khiển sản phẩm.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:
 - Luồng thêm sản phẩm mới:

- Admin điền thông tin, đính kèm các tệp hình ảnh và gửi qua API POST /api/product/add (định dạng multipart/form-data).
 - Middleware Multer tiếp nhận, phân tích tệp và tự động upload lên Cloudinary.
 - Cloudinary trả về các đường dẫn URL ảnh an toàn.
 - Product Controller lưu bản ghi Product mới (kèm URL ảnh và số lượng sizesStock ban đầu) vào MongoDB.
- Luồng nhập thêm hàng (Restock):
- Admin chọn sản phẩm, nhập số lượng cần bổ sung vào các Size tương ứng.
 - Admin Frontend gọi API POST /api/product/update-stock.
 - Product Controller nhận dữ liệu, sử dụng toán tử \$inc của MongoDB để cộng dồn chính xác số lượng mới vào trường sizesStock hiện tại.
 - CSDL cập nhật thành công, Controller phản hồi HTTP 200 OK. Frontend thông báo "Cập nhật tồn kho thành công".



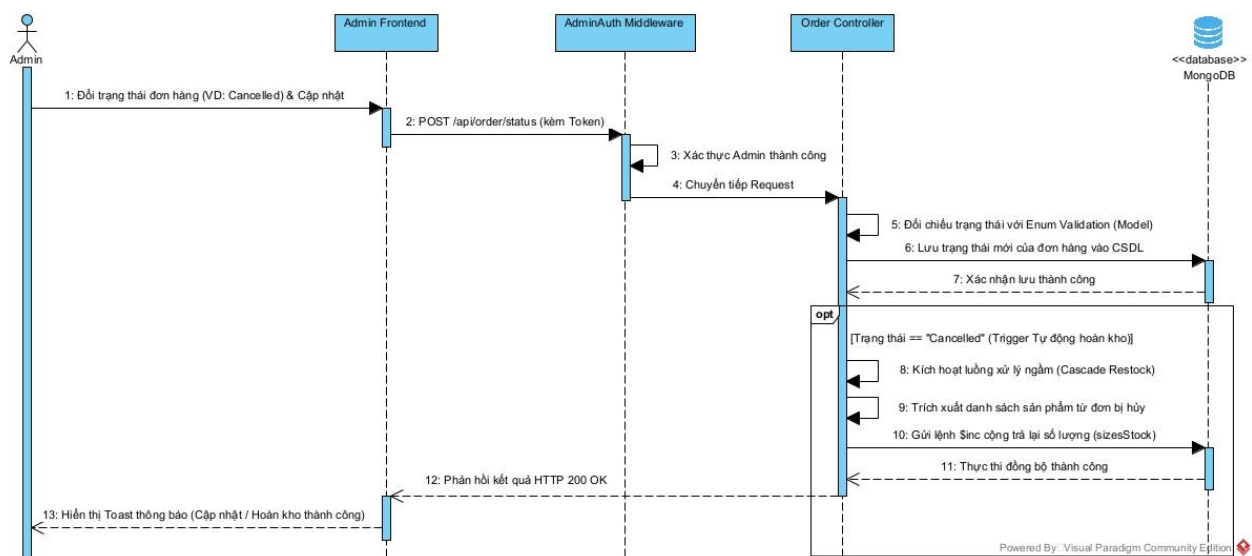
Hình 16 - Sequence Diagram 15 - Quản lý Sản phẩm và Nhập kho

16. Biểu đồ trình tự: Luồng Quản lý Đơn hàng & Tự động hoàn kho (UC16)

Luồng nổi bật với cơ chế trigger tự động phục hồi số lượng tồn kho khi đơn hàng bị đánh dấu là "Đã hủy".

- Các đối tượng tham gia:
 - Admin: Quản trị viên.
 - Admin Frontend: Bảng quản lý đơn hàng.
 - AdminAuth Middleware: Lớp xác thực Admin.
 - Order Controller: Bộ điều khiển xử lý luồng đơn.
 - MongoDB: Cơ sở dữ liệu.
- Trình tự tương tác:

- Admin chọn đổi trạng thái đơn hàng trên giao diện (ví dụ: sang trạng thái Cancelled) và nhấn Cập nhật.
- Admin Frontend gửi API POST /api/order/status đính kèm Token.
- AdminAuth Middleware xác thực thành công. Order Controller tiếp nhận request.
- Controller đổi chiều trạng thái gửi lên với cấu trúc Enum Validation trong Model.
- Order Controller lưu trạng thái mới của đơn hàng vào MongoDB.
- (Trigger Tự động hoàn kho - Cascade Restock):
 - Nếu trạng thái được lưu là Cancelled, Controller kích hoạt một luồng xử lý ngầm.
 - Controller trích xuất danh sách sản phẩm nằm trong đơn hàng bị hủy.
 - Gửi lệnh truy vấn sử dụng toán tử \$inc vào MongoDB để cộng trả lại số lượng vào đúng biến thể sizesStock của các sản phẩm đó.
- CSDL thực thi đồng bộ. Controller phản hồi kết quả HTTP 200 OK.
- Admin Frontend hiển thị Toast thông báo: "Đã hủy đơn và hoàn trả tồn kho thành công".



Hình 17 - Sequence Diagram 16 - Quản lý Đơn hàng & Tự động hoàn kho

17. Biểu đồ trình tự: Luồng Quản lý Người dùng & Xóa Cascade (UC17)

Luồng bảo vệ tính toàn vẹn của cơ sở dữ liệu bằng cách dọn dẹp sạch sẽ toàn bộ dữ liệu mồ côi khi một tài khoản bị xóa.

- Các đối tượng tham gia:
 - Admin: Quản trị viên.
 - Admin Frontend: Bảng quản lý tài khoản.
 - User Controller: Bộ điều khiển tài khoản.

- MongoDB: Cơ sở dữ liệu.

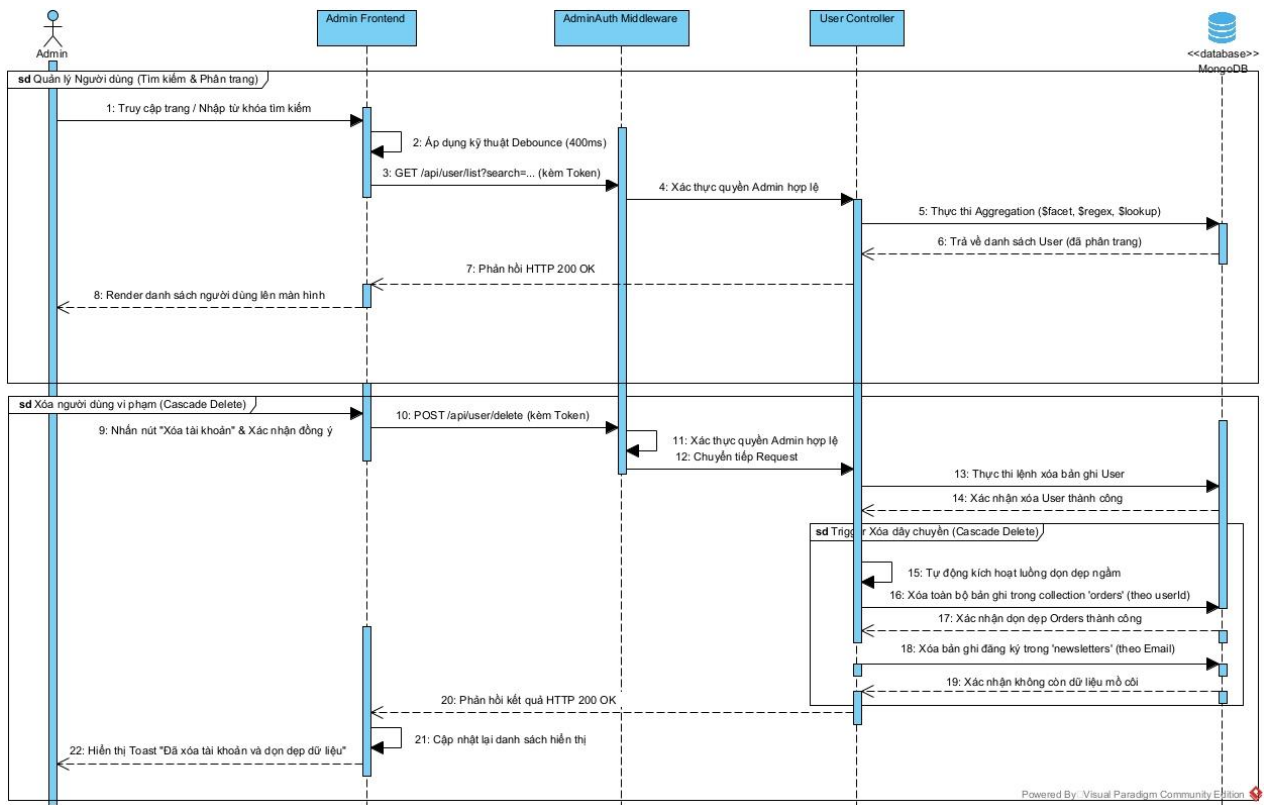
- Trình tự tương tác:

(Luồng Tìm kiếm & Phân trang):

- Admin truy cập trang hoặc nhập từ khóa tìm kiếm. Admin Frontend áp dụng kỹ thuật Debounce (độ trễ 400ms) rồi gọi API GET /api/user/list.
- Yêu cầu đi qua lớp AdminAuth Middleware để xác thực.
- Backend sử dụng Aggregation Pipeline (kết hợp \$facet, \$regex, \$lookup) để thực hiện Tìm kiếm và Phân trang trực tiếp tại MongoDB. Trả kết quả về để Frontend render danh sách.

(Luồng Xóa dây chuyền - Cascade Delete):

- Admin nhấn nút "Xóa tài khoản" đối với một người dùng có hành vi vi phạm.
- Giao diện yêu cầu xác nhận. Khi đồng ý, Admin Frontend gọi API POST /api/user/delete đính kèm Token.
- Yêu cầu tiếp tục đi qua xác thực quyền Admin. User Controller thực thi lệnh xóa bản ghi User đó khỏi MongoDB.
- Ngay sau khi xóa User thành công, Controller tự động kích hoạt luồng dọn dẹp (Trigger).
- Controller tìm và xóa toàn bộ các bản ghi trong collection orders (Lịch sử đơn hàng) có chứa userId vừa xóa.
- Tiếp tục tìm và xóa bản ghi đăng ký trong collection newsletters (nếu Email của user đó có đăng ký).
- MongoDB xác nhận đã xóa sạch dữ liệu liên đới, không còn dữ liệu mồ côi trong hệ thống.
- User Controller trả về phản hồi HTTP 200 OK.
- Admin Frontend cập nhật lại danh sách và hiển thị Toast "Đã xóa tài khoản và dọn dẹp dữ liệu thành công".



Hình 18 - Sequence Diagram 17 - Quản lý Người dùng & Xóa Cascade

3.2 Thiết kế hệ thống

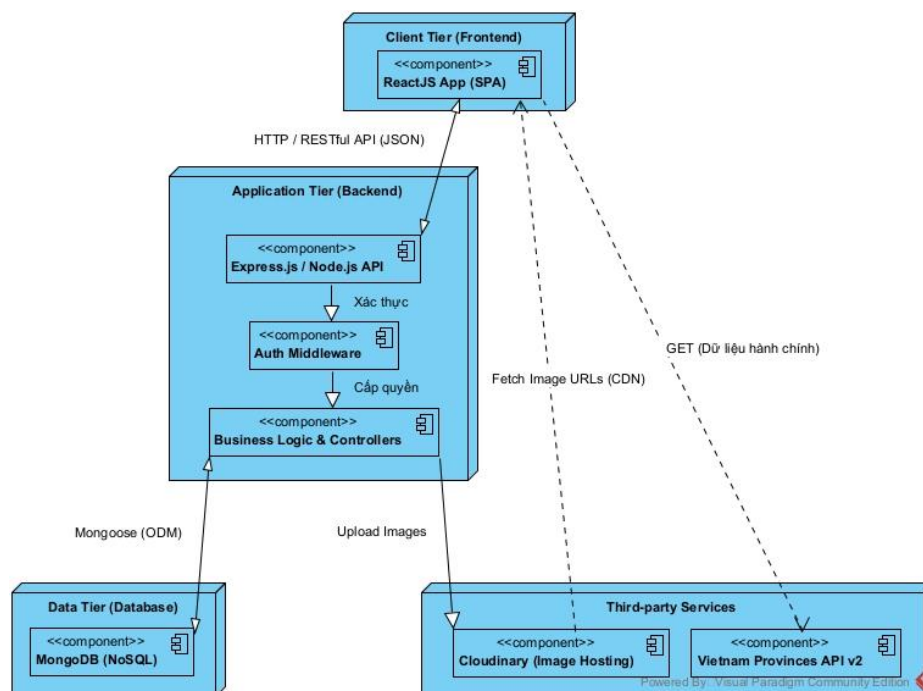
3.2.1 Thiết kế kiến trúc tổng thể

Hệ thống thương mại điện tử INFINITY được xây dựng dựa trên kiến trúc Client-Server hiện đại, áp dụng mô hình phân lớp (3-Tier Architecture) nhằm đảm bảo tính toàn vẹn, dễ bảo trì và khả năng mở rộng trong tương lai. Nền tảng công nghệ cốt lõi được sử dụng là MERN Stack (MongoDB, Express.js, ReactJS, Node.js), kết hợp với các dịch vụ đám mây và API mở.

Kiến trúc hệ thống được chia thành 3 lớp phân tách rõ ràng về mặt nhiệm vụ:

- Lớp Trình diễn (Presentation Tier - Frontend / Client-side): Được phát triển bằng thư viện ReactJS, hoạt động theo cơ chế Single Page Application (SPA). Lớp này chịu trách nhiệm hiển thị giao diện người dùng tương thích đa thiết bị (Responsive Design) nhờ tích hợp Tailwind CSS. Việc quản lý luồng trạng thái (State Management) được xử lý thông qua React Context và điều hướng mượt mà bằng React Router. Mọi giao tiếp với máy chủ đều được thực hiện thông qua các yêu cầu HTTP/RESTful API.
- Lớp Ứng dụng (Application Tier - Backend / Server-side): Đóng vai trò là "bộ não" xử lý logic của toàn bộ hệ thống, được xây dựng trên môi trường Node.js và framework Express.js. Lớp này làm nhiệm vụ tiếp nhận các API Request từ Frontend, đi qua các rào chắn bảo mật (Auth Middleware xác thực JWT), phân tích và xử lý các nghiệp vụ cốt lõi phức tạp (như tính toán giá Server-side, phân trang, bảo mật mật khẩu bằng Bcrypt), và định tuyến (Routes) luồng dữ liệu một cách an toàn.

- Lớp Dữ liệu (Data Tier - Database): Sử dụng hệ quản trị cơ sở dữ liệu phi quan hệ (NoSQL) MongoDB. Dữ liệu được cấu trúc dưới dạng các document JSON linh hoạt, cho phép lưu trữ khối lượng lớn thông tin về sản phẩm, đơn hàng và người dùng. Backend giao tiếp với CSDL thông qua thư viện Mongoose (Object Data Modeling), tận dụng tối đa sức mạnh của Aggregation Framework cho các bài toán thống kê phức tạp và toán tử nguyên tử (\$inc) để đảm bảo tính toàn vẹn của dữ liệu tồn kho.
- Dịch vụ Mở rộng (Third-party Services): Để tối ưu hóa băng thông và tài nguyên máy chủ, hệ thống tích hợp nền tảng Cloudinary để lưu trữ, quản lý và phân phối tệp hình ảnh sản phẩm độc lập. Đồng thời, hệ thống kết nối với Vietnam Provinces API (v2) để tự động hóa việc trích xuất và hiển thị dữ liệu hành chính trong quy trình giao hàng của khách hàng.



Hình 19 - Sơ đồ Kiến trúc tổng quan hệ thống

3.2.2 Thiết kế sơ đồ lớp (Class Diagram)

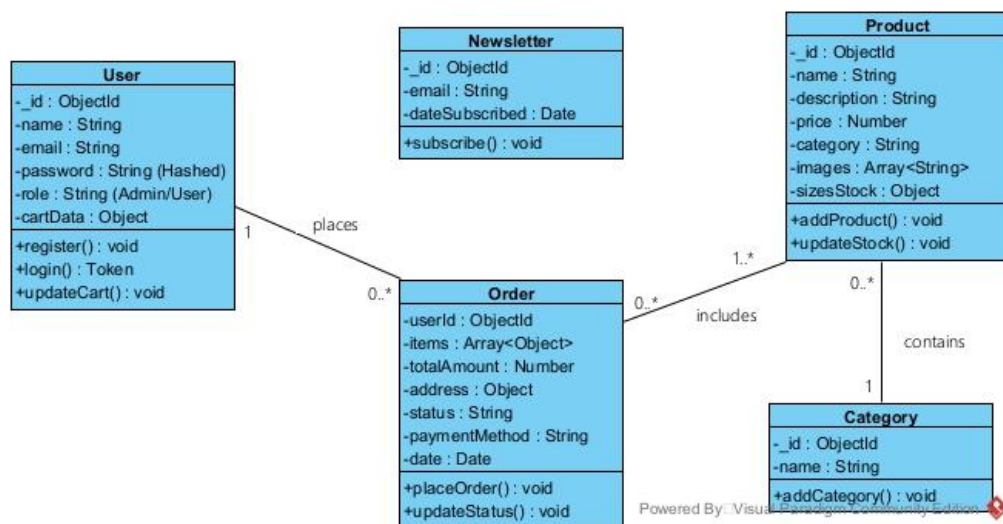
Do hệ thống INFINITY sử dụng cơ sở dữ liệu phi quan hệ (NoSQL MongoDB), kiến trúc dữ liệu không bị ràng buộc bởi các bảng (Tables) khắt khe như hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Thay vào đó, hệ thống ứng dụng thư viện Mongoose (Object Data Modeling - ODM) để định nghĩa cấu trúc dữ liệu tại tầng Backend.

Sơ đồ lớp (Class Diagram) dưới đây mô tả các Mongoose Schemas (Models) cốt lõi của hệ thống, các thuộc tính (Attributes) và các phương thức xử lý nghiệp vụ (Operations) tương ứng:

▪ Các lớp thực thể chính:

- User (Người dùng): Quản lý định danh tài khoản, phân quyền hệ thống (Role: Admin/User), bảo mật (mật khẩu đã mã hóa) và dữ liệu trạng thái giỏ hàng (cartData).

- Product (Sản phẩm): Quản lý thông tin chi tiết hàng hóa, hình ảnh và đặc biệt là biến thể số lượng tồn kho theo kích cỡ (sizesStock).
 - Order (Đơn hàng): Lớp trung tâm xử lý giao dịch. Lưu trữ thông tin định danh người mua, danh sách sản phẩm đã đặt (items), tổng tiền thanh toán và tiến trình xử lý đơn hàng.
 - Category (Danh mục): Quản lý cây phân loại, hỗ trợ điều hướng và lọc sản phẩm.
 - Newsletter (Bản tin): Quản lý tệp khách hàng đăng ký nhận thông tin khuyến mãi.
- **Mối quan hệ logic:**
- User - Order (1 - N): Một khách hàng có thể tạo nhiều đơn đặt hàng, nhưng mỗi đơn hàng chỉ thuộc về một khách hàng duy nhất.
 - Category - Product (1 - N): Một danh mục chứa nhiều sản phẩm.
 - Order - Product (N - N): Mối quan hệ nhiều - nhiều này được MongoDB tối ưu hóa bằng cách nhúng (Embed) trực tiếp mảng các đối tượng sản phẩm vào bên trong lớp Order, loại bỏ nhu cầu sử dụng bảng trung gian.

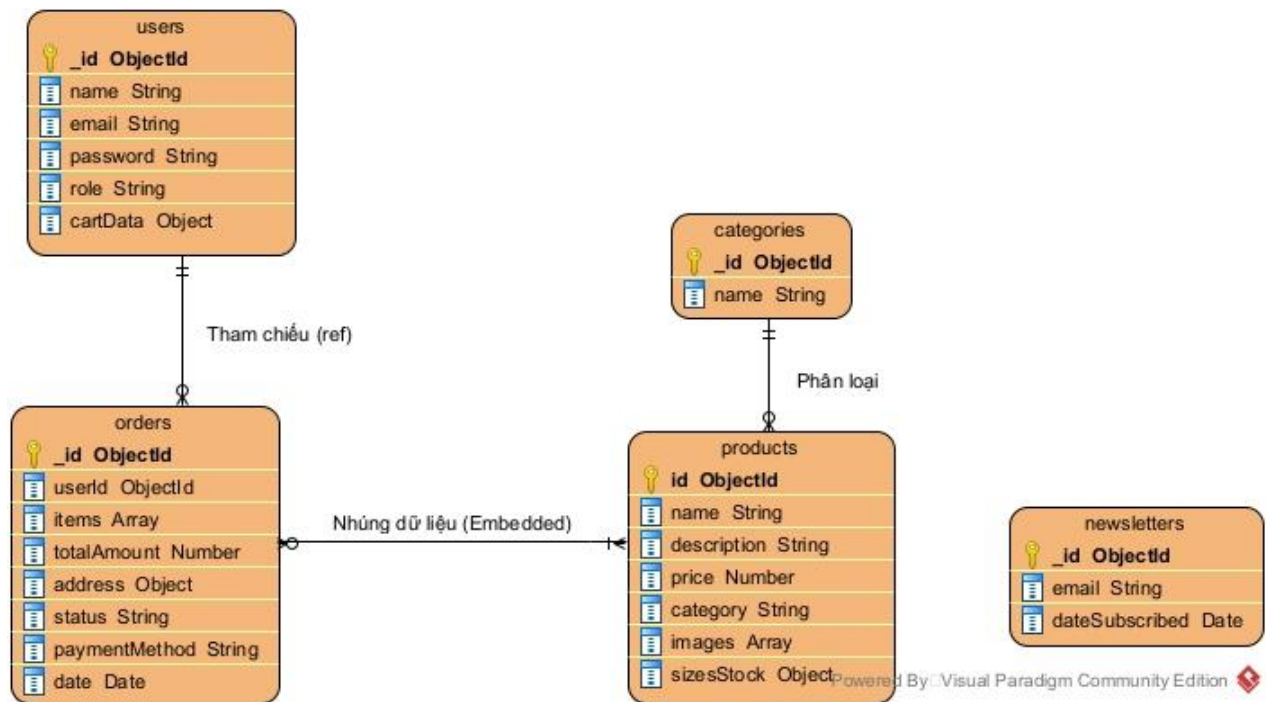


Hình 20 - Cấu trúc sơ đồ lớp

3.2.3 Thiết kế cơ sở dữ liệu (Database Design)

3.2.3.1 Sơ đồ ERD (Mô hình Dữ liệu NoSQL)

Do đặc thù của hệ thống sử dụng **MongoDB**, mô hình dữ liệu được thiết kế theo hướng Dữ liệu Phi quan hệ (NoSQL Data Model). Khác với sơ đồ ERD truyền thống của SQL, sơ đồ dưới đây thể hiện cấu trúc các **Collections** (Tập hợp), **Documents** (Tài liệu JSON) và các mảng dữ liệu nhúng (Embedded Objects). Các mối quan hệ được xử lý thông qua cơ chế tham chiếu (Reference - *ref*) thay vì khóa ngoại (Foreign Key).



Hình 21 - Sơ đồ Cấu trúc MongoDB

3.2.3.2 Mô tả các bảng/collection

Trong kiến trúc cơ sở dữ liệu của hệ thống, các tài liệu (Documents) được tổ chức thành 5 Collections chính. Mỗi Collection được định nghĩa cấu trúc chặt chẽ thông qua Mongoose Schema tại tầng Backend nhằm đảm bảo tính toàn vẹn dữ liệu.

1. Collection: users

Mô tả: Lưu trữ thông tin định danh, tài khoản bảo mật và trạng thái mua sắm hiện tại của người dùng.

▪ Cấu trúc dữ liệu:

- `_id (ObjectId)`: Khóa chính, được hệ thống tự động khởi tạo.
- `name (String)`: Họ và tên đầy đủ của người dùng.
- `email (String)`: Địa chỉ email dùng để đăng nhập (Định dạng duy nhất - Unique).
- `password (String)`: Mật khẩu đã được mã hóa an toàn (Hashed).
- `role (String)`: Quyền truy cập hệ thống. Mặc định là "User", có thể cấp quyền "Admin".
- `cartData (Object)`: Dữ liệu giỏ hàng hiện tại, lưu trữ dưới dạng object nhúng (Embedded) để truy xuất nhanh mà không cần join bảng. Mặc định là {}.
- `usedVouchers (Array)`: Mảng lưu vết danh sách các mã giảm giá (Vouchers) mà khách hàng này đã áp dụng thành công. Hỗ trợ đối chiếu 2 lớp để đảm bảo mỗi tài khoản chỉ được dùng mã một lần duy nhất.

2. Collection: products

Mô tả: Quản lý thông tin chi tiết của các mặt hàng đang được kinh doanh trên hệ thống INFINITY.

- Cấu trúc dữ liệu:
 - `_id (ObjectId)`: Khóa chính tự động của sản phẩm.
 - `name (String)`: Tên hiển thị của sản phẩm.
 - `description (String)`: Đoạn văn bản mô tả chi tiết sản phẩm.
 - `price (Number)`: Giá bán của sản phẩm.
 - `category (ObjectId)`: Tham chiếu (ref) đến khóa chính của collection categories.
 - `images (Array<String>)`: Mảng chứa các URL đường dẫn hình ảnh sản phẩm (được lưu trữ trên nền tảng Cloudinary).
 - `sizesStock (Object)`: Object nhúng quản lý biến thể tồn kho theo kích cỡ. Ví dụ: { S: 10, M: 25, L: 5 }.

3. Collection: orders

Mô tả: Lưu vết toàn bộ lịch sử giao dịch. Tận dụng thế mạnh của NoSQL để nhúng trực tiếp chi tiết đơn hàng vào trong một document duy nhất.

- Cấu trúc dữ liệu:
 - `_id (ObjectId)`: Khóa chính, đồng thời là mã vận đơn của hệ thống.
 - `userId (ObjectId)`: Tham chiếu (ref) đến khách hàng trong collection users.
 - `items (Array<Object>)`: Mảng nhúng (Embedded Array) chứa danh sách các sản phẩm (tên, giá, size, số lượng) ngay tại thời điểm chốt đơn.
 - `totalAmount (Number)`: Tổng số tiền khách hàng phải thanh toán.
 - `address (Object)`: Object nhúng lưu trữ thông tin giao hàng (Tên người nhận, SĐT, Địa chỉ cụ thể).
 - `status (String)`: Trạng thái xử lý đơn hàng. Bắt buộc áp dụng Enum Validation ở mức CSDL (Chỉ nhận các giá trị định trước như Order Placed, Shipped, Delivered, Cancelled) để chống rác dữ liệu.
 - `paymentMethod (String)`: Phương thức thanh toán (COD, Stripe, VNPAY...).
 - `date (Date)`: Thời gian khởi tạo đơn hàng.

4. Collection: categories

Mô tả: Danh mục hệ thống để phân loại, hỗ trợ thuật toán lọc và tìm kiếm sản phẩm trên giao diện.

- Cấu trúc dữ liệu:
 - `_id (ObjectId)`: Khóa chính của danh mục.

- *name (String)*: Tên danh mục (Ví dụ: Áo thun, Quần Jeans, Phụ kiện...). Bắt buộc là duy nhất (Unique).

******(Lưu ý kỹ thuật: Để tránh rủi ro bất đồng bộ dữ liệu, collection này tuyệt đối không lưu trữ trường "Tổng doanh thu". Doanh thu của danh mục được tính toán động (on-the-fly) tại thời điểm gọi API thông qua Aggregation Pipeline).

5. Collection: newsletters

Mô tả: Hỗ trợ chiến dịch Marketing, lưu trữ danh sách khách hàng muốn nhận thông báo ưu đãi qua email.

- Cấu trúc dữ liệu:
 - *_id (ObjectId)*: Khóa chính tự động.
 - *email (String)*: Email đăng ký nhận bản tin (Unique).
 - *dateSubscribed (Date)*: Thời gian thực hiện đăng ký.

3.2.4 Thiết kế API (RESTful API)

Hệ thống INFINITY áp dụng kiến trúc RESTful API được xây dựng trên nền tảng Node.js và Express.js. Các API đóng vai trò là cầu nối giao tiếp chuẩn hóa giữa Client (Frontend) và Server (Backend), nhận các yêu cầu (Request) qua giao thức HTTP và trả về dữ liệu (Response) dưới định dạng JSON. Hệ thống sử dụng JWT (JSON Web Token) để xác thực người dùng trong các API yêu cầu tính bảo mật.

Dưới đây là danh sách các API Endpoint cốt lõi được chia theo từng module nghiệp vụ của hệ thống:

- **Module Người dùng & Xác thực (User & Authentication)** Các API xử lý việc đăng ký, đăng nhập và phân quyền.
 - POST /api/user/register: Đăng ký tài khoản mới. Nhận dữ liệu name, email, password, tiến hành băm (hash) mật khẩu bằng Bcrypt và lưu vào database.
 - POST /api/user/login: Xác thực đăng nhập. Kiểm tra thông tin, nếu hợp lệ sẽ trả về chuỗi Token (JWT) để Client lưu trữ.
 - POST /api/user/admin: Kiểm tra quyền truy cập của Admin (đối chiếu trực tiếp với biến môi trường trong file .env).
 - GET /api/user/list: Lấy danh sách người dùng (Chỉ Admin). Hỗ trợ nhận các query params: search, page, limit, sortOrder để xử lý phân trang và tìm kiếm hoàn toàn tại Server-side bằng Aggregation (\$facet, \$regex).
 - POST /api/user/status: Cập nhật trạng thái khóa/mở khóa tài khoản (Active/Blocked).
 - POST /api/user/delete: Xóa người dùng và tự động kích hoạt luồng Cascade Delete dọn dẹp đơn hàng và bản tin liên quan.
- **Module Sản phẩm (Products)** Quản lý danh mục hàng hóa hiển thị trên cửa hàng và trang quản trị.

- GET /api/product/list: Lấy danh sách toàn bộ sản phẩm (Public). Trả về mảng các đối tượng sản phẩm bao gồm cả thông tin tồn kho (sizesStock).
- POST /api/product/add: Thêm một sản phẩm mới (Chỉ Admin). Tích hợp Middleware Multer để xử lý file upload và đẩy ảnh lên Cloudinary để lấy URL an toàn trước khi lưu.
- POST /api/product/update-stock: Cập nhật nhập thêm hàng (Restock). Sử dụng toán tử \$inc của MongoDB để cộng dồn chính xác số lượng mới vào trường sizesStock hiện tại.
- POST /api/product/remove: Xóa một sản phẩm khỏi hệ thống (Chỉ Admin). Yêu cầu truyền vào _id.
- POST /api/product/single: Lấy thông tin chi tiết của một sản phẩm cụ thể dựa trên _id.
- **Module Giỏ hàng (Cart)** Quản lý trạng thái mua sắm của từng người dùng cụ thể. Yêu cầu phải có Token xác thực ở Header.
 - POST /api/cart/add: Thêm một sản phẩm (kèm kích cỡ và số lượng) vào đối tượng cartData của User.
 - POST /api/cart/update: Cập nhật lại số lượng sản phẩm hoặc thay đổi kích cỡ (Size) trong giỏ hàng.
 - POST /api/cart/get: Lấy toàn bộ dữ liệu giỏ hàng hiện tại của người dùng để hiển thị lên giao diện (Fetch User Cart).
- **Module Đơn hàng (Orders)** Xử lý quy trình thanh toán và theo dõi vận đơn với các logic nghiệp vụ phức tạp.
 - POST /api/order/place: Tiến hành đặt hàng. Hệ thống tự động tính toán tổng tiền tại Server-side dựa trên giá gốc, thực hiện xác thực Voucher 2 lớp, và sử dụng toán tử \$inc để trừ tồn kho nguyên tử. Nếu thành công, lưu bản ghi và xóa giỏ hàng của User.
 - POST /api/order/stripe: (Nâng cao) Tiến hành đặt hàng và gọi cổng thanh toán trực tuyến (Stripe/VNPay).
 - POST /api/order/userorders: Lấy danh sách lịch sử các đơn hàng mà một người dùng cụ thể đã đặt.
 - GET /api/order/list: Lấy danh sách toàn bộ đơn hàng của tất cả khách hàng (Chỉ Admin).
 - POST /api/order/status: Cập nhật trạng thái đơn hàng. Đặc biệt: Nếu trạng thái chuyển sang "Cancelled", hệ thống tự động kích hoạt Trigger hoàn trả lại số lượng tồn kho nguyên vẹn cho các sản phẩm trong đơn.
- **Module Bản tin & Khuyến mãi (Newsletter)**

- POST /api/newsletter/subscribe: Tiếp nhận Email đăng ký của khách hàng, kiểm tra trùng lặp và tự động phản hồi cấp phát mã Voucher ưu đãi (VD: BARCA20).
- **Module Thống kê (Dashboard)** Cung cấp các API chuyên biệt xử lý dữ liệu lớn bằng kỹ thuật truy vấn gộp (Aggregation Pipeline).
 - GET /api/dashboard/stats: Lấy toàn bộ dữ liệu thống kê tổng quan (Chỉ Admin). Điểm nhấn kỹ thuật là dùng truy vấn phức tạp để tính doanh thu 12 tháng (chỉ lọc đơn hàng "Delivered" phục vụ vẽ biểu đồ Recharts), tính tỷ lệ MoM, phân bổ trạng thái đơn, và quét danh sách sản phẩm sắp hết hàng (tồn kho < 10).
 - GET /api/dashboard/best-sellers: Lấy danh sách Top 5 sản phẩm bán chạy nhất (Public). Tái sử dụng hàm aggregateBestSellers để tính doanh số thực tế (on-the-fly) từ các đơn hàng thành công thay vì đánh dấu tĩnh (hard-code).
- **Module Danh mục (Categories)** Quản lý cấu trúc phân loại của cửa hàng, hỗ trợ điều hướng và lọc sản phẩm.
 - GET /api/category/list: Lấy danh sách danh mục phân loại (Public). Trả về mảng danh mục đã được Backend tính toán doanh thu động thông qua Aggregation và sắp xếp giảm dần.
 - POST /api/category/add: Thêm một danh mục sản phẩm mới vào hệ thống (Chỉ Admin).
 - POST /api/category/remove: Xóa danh mục khỏi hệ thống (Chỉ Admin). Có tích hợp kiểm tra ràng buộc (Dependency Check) để chặn xóa nếu danh mục vẫn đang chứa sản phẩm.
- Tham số Header mặc định cho các API bảo mật:

Đối với các API yêu cầu định danh (Cart, Order, Admin Dashboard), Client bắt buộc phải đính kèm Token vào HTTP Header khi gửi Request thông qua Middleware auth.js hoặc adminAuth.js:

 - headers: { token: "chuỗi_jwt_token_lấy_từ_lúc_login" }

3.2.5 Thiết kế giao diện (UI/UX)

Giao diện người dùng (UI) và Trải nghiệm người dùng (UX) của hệ thống INFINITY được thiết kế với mục tiêu tối ưu hóa hành trình mua sắm của khách hàng, đảm bảo tính thẩm mỹ, hiện đại và dễ sử dụng. Hệ thống áp dụng triết lý thiết kế kết hợp với Responsive Web Design để giao diện tự động thích ứng hoàn hảo trên mọi kích thước màn hình (Desktop, Tablet, Mobile).

3.2.5.1 Nguyên tắc thiết kế UI/UX cốt lõi:

- Tính nhất quán (Consistency): Sử dụng bộ font chữ, bảng màu chủ đạo (Theme colors) và các thành phần UI (Nút bấm, Input, Card sản phẩm) đồng nhất xuyên suốt tất cả các trang.

- Tối giản và Tập trung (Minimalism & Focus): Lược bỏ các chi tiết thừa. Làm nổi bật các Lời kêu gọi hành động (Call to Action - CTA) quan trọng như nút "Thêm vào giỏ hàng" (Add to Cart) hay "Thanh toán" (Checkout).
- Điều hướng trực quan (Intuitive Navigation): Thanh điều hướng (Navbar) được ghim ở đầu trang, kết hợp với cây thư mục (Breadcrumbs) và chức năng lọc/tìm kiếm giúp người dùng dễ dàng tìm thấy sản phẩm mong muốn.

3.2.5.2 Cấu trúc giao diện các phân hệ chính:

1. Giao diện dành cho Khách hàng (Client Side):

- Trang Đăng nhập / Đăng ký: Giao diện tối giản, tập trung vào nhiệm vụ xác thực. Form Đăng nhập và Đăng ký được tích hợp thông minh trên cùng một trang, cho phép người dùng dễ dàng chuyển đổi qua lại, đảm bảo luồng trải nghiệm không bị ngắt quãng.

INFINITY

HOME COLLECTION ABOUT CONTACT

Q 0

Login —

Email

Password

Forgot your password? Create Account

Sign In

INFINITY

At INFINITY, we deliver a football lifestyle for true fans, offering high-quality kits and sportswear inspired by the Blaugrana spirit.

COMPANY

Home
About us
Delivery
Privacy policy

CONTACT

☎ (+84) 888149225
✉ hoangtrong2962005@gmail.com
📍 hm.trong296
👤 Hoang Trong

Copyright 2026 hm.trong296 - All Right Reserved.

Hình 22 - Giao diện đăng nhập

Sign Up —

Name

Email

Password

Forgot your password?

Login Here

Sign Up

COMPANY

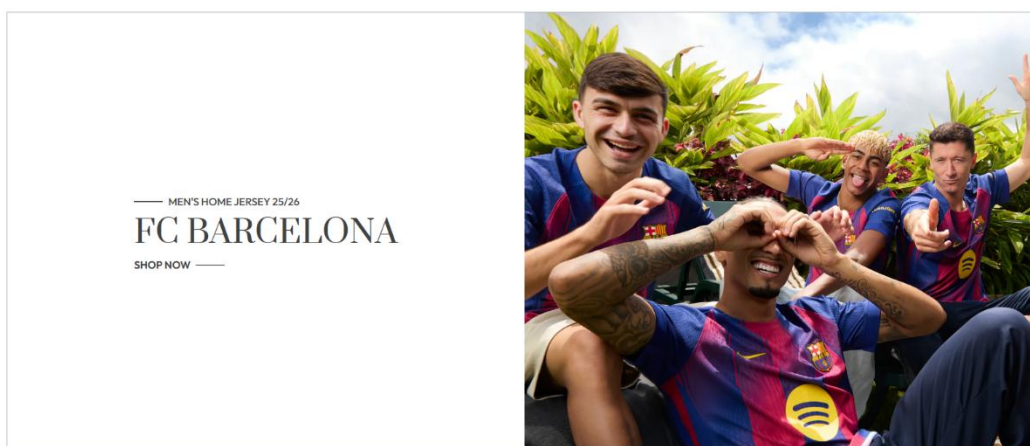
Home
About us
Delivery
Privacy policy

CONTACT

(+84) 888149225
hoangtrong2962005@gmail.com
hm.trong296
Hoang Trong

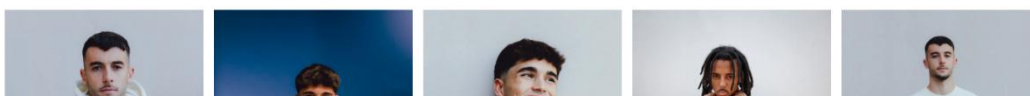
Hình 23 - Giao diện đăng ký

- Trang Chủ: Đây là bộ mặt của website, trang đầu tiên người dùng nhìn thấy khi truy cập INFINITY. Giao diện được thiết kế để tạo ấn tượng mạnh với Hero Banner hiển thị các chiến dịch khuyến mãi cốt lõi, theo sau là các phần (Sections) giới thiệu Bộ sưu tập mới nhất (Latest Collections) và Sản phẩm bán chạy (Best Sellers) để kích thích nhu cầu mua sắm ngay lập tức.



LATEST COLLECTIONS —

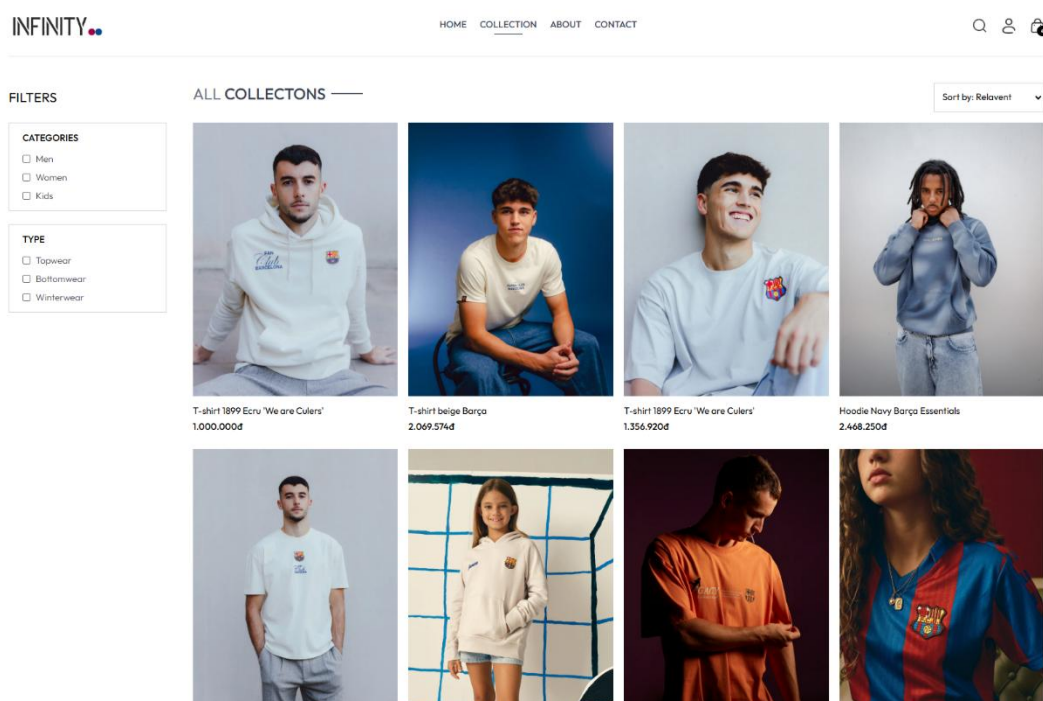
Bộ sưu tập áo đấu mới nhất



Hình 24 - Giao diện trang chủ

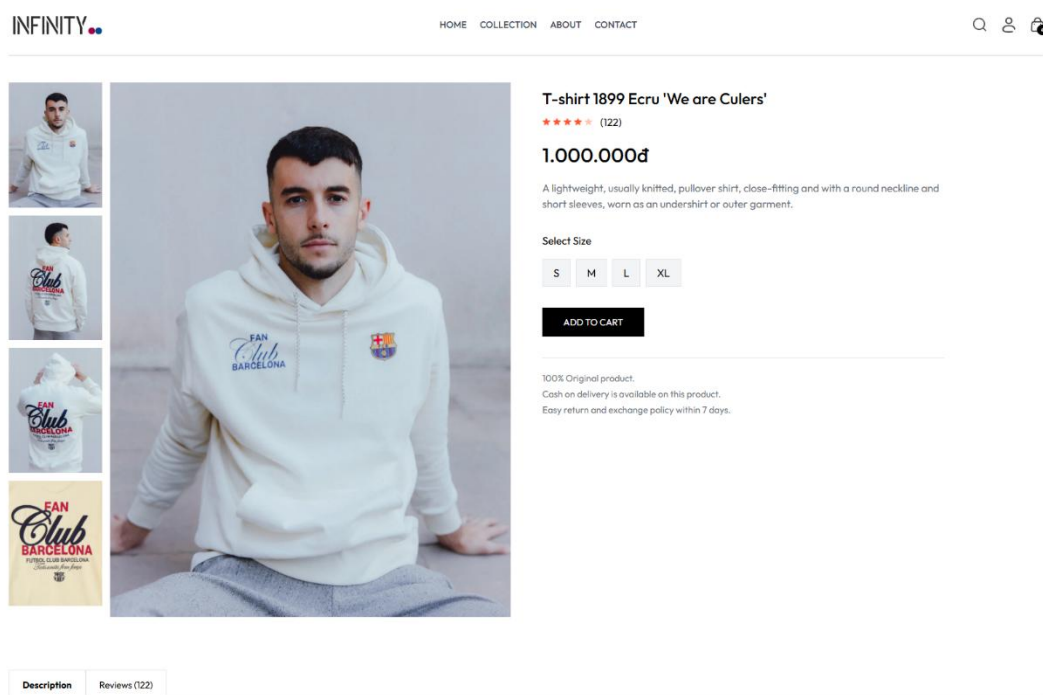
- Trang Danh mục sản phẩm: Giao diện này cung cấp cái nhìn tổng quan về toàn bộ hệ thống sản phẩm. Sản phẩm được hiển thị dưới dạng lưới (Grid) sắc nét. Điểm nhấn

của trang là tích hợp bộ lọc trực quan bên trái màn hình, cho phép khách hàng dễ dàng thu hẹp phạm vi tìm kiếm theo Category giúp tăng tỷ lệ chuyển đổi.



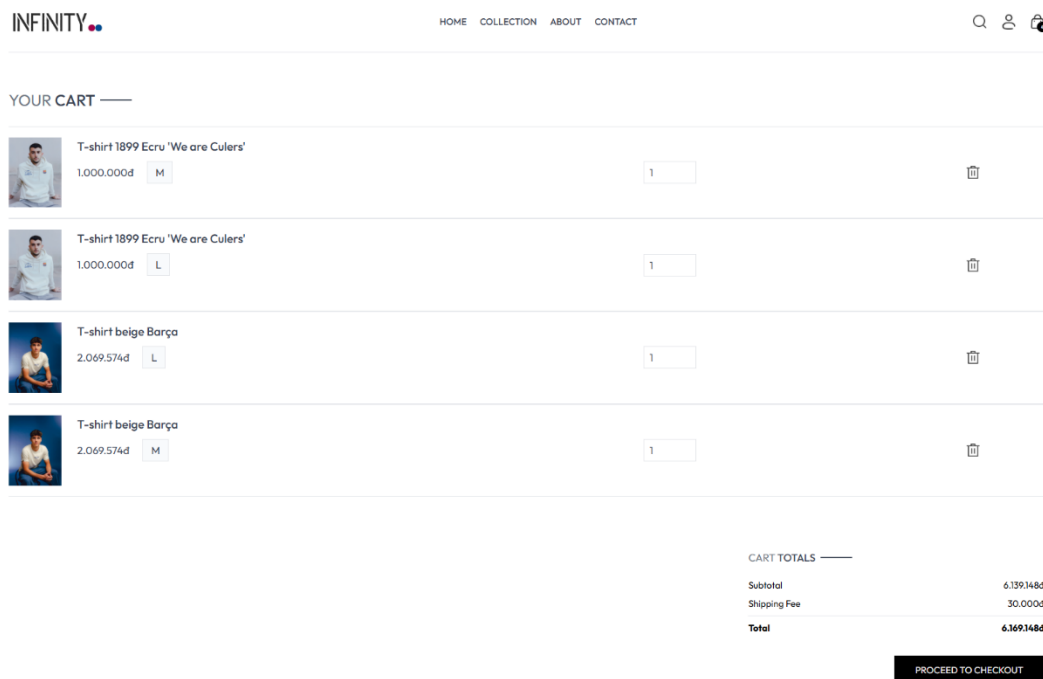
Hình 25 - Giao diện danh mục sản phẩm

- Trang Chi tiết sản phẩm: Đây là nơi người dùng đưa ra quyết định mua sắm cuối cùng. Trang cung cấp thư viện ảnh sản phẩm đa góc cạnh (Thumbnail list). Bố cục tập trung vào thông tin mô tả chi tiết, giá cả và đặc biệt là ràng buộc người dùng bắt buộc phải chọn Kích cỡ (Size) trước khi có thể bấm nút "Thêm vào giỏ hàng" (Add to Cart), nhằm giảm thiểu sai sót khi đặt hàng.



Hình 26 - Giao diện chi tiết sản phẩm

- Trang Giỏ hàng: Tập hợp danh sách các sản phẩm người dùng dự định mua. Giao diện tối giản giúp khách hàng dễ dàng rà soát lại thông tin sản phẩm, kích cỡ, đơn giá và cho phép điều chỉnh nhanh số lượng hoặc xóa sản phẩm khỏi giỏ hàng một cách thuận tiện.



Hình 27 - Giao diện giỏ hàng

- Trang Thanh toán: Đây là bước cuối cùng để hoàn tất quy trình mua sắm. Giao diện tối giản được chia làm 2 phần rõ rệt: Form nhập thông tin giao hàng (có tích hợp Vietnam Provinces API v2 để chọn địa chỉ tự động) và phần lựa chọn Phương thức thanh toán, áp dụng mã Voucher. Nút CTA chốt đơn "PLACE ORDER" được làm to bản, tương phản mạnh để thúc đẩy tối đa hành động mua hàng.

Hình 28 - Giao diện thanh toán

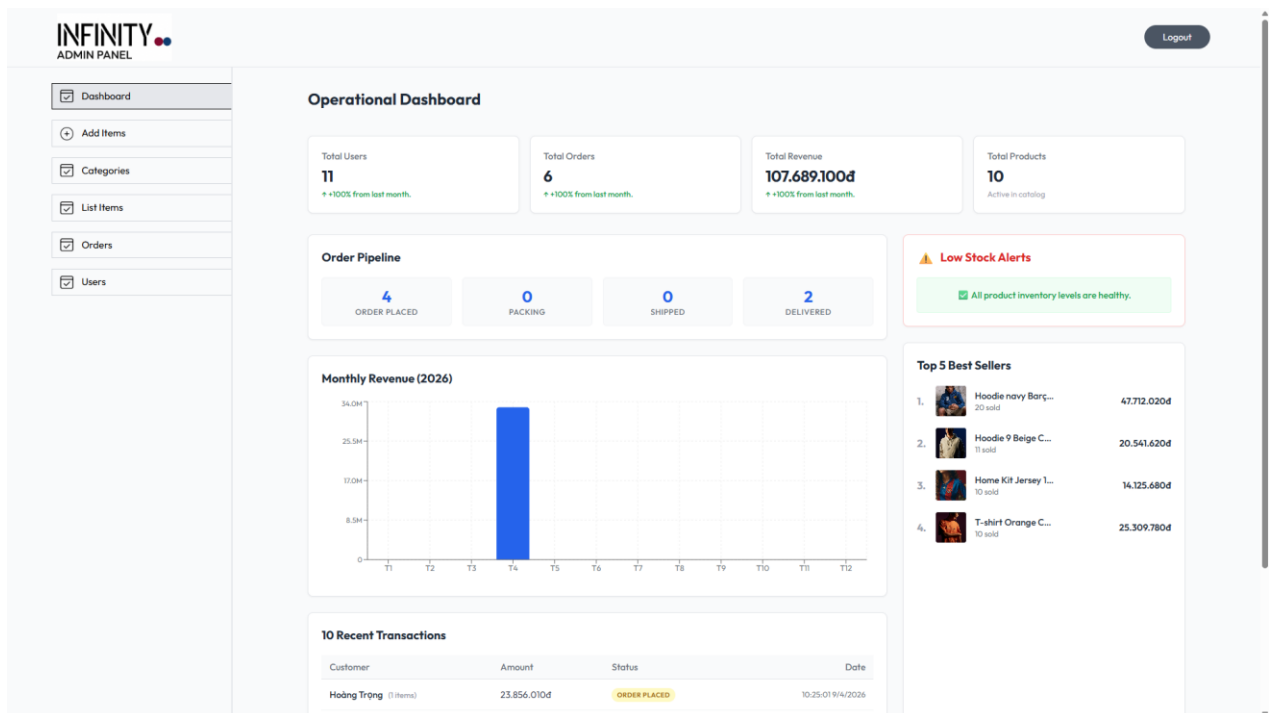
- Trang Lịch sử đơn hàng: Cung cấp không gian cá nhân hóa để khách hàng theo dõi toàn bộ trạng thái của các đơn hàng họ đã đặt (Ví dụ: Order Placed, Shipped, Delivered), giúp tăng tính minh bạch và lòng tin của người dùng đối với hệ thống.

Hình 29 - Giao diện lịch sử đơn hàng

2. Giao diện dành cho Quản trị viên (Admin Panel)

- Trang Tổng quan: Giao diện bảo mật riêng biệt, đóng vai trò là "trung tâm đầu não" cung cấp cái nhìn tổng quát về hiệu suất hoạt động của cửa hàng. Hệ thống trực quan

hóa các số liệu thống kê cốt lõi (Tổng doanh thu, Số lượng đơn hàng, Tỷ lệ tăng trưởng) được tính toán động bằng Aggregation Pipeline. Bên cạnh đó, trang còn tích hợp biểu đồ doanh thu (Recharts), theo dõi tiến trình đơn hàng và module cảnh báo tồn kho tự động để hỗ trợ Admin quản lý hiệu quả.



Hình 30 - Giao diện tổng quan

- Trang Thêm sản phẩm: Giao diện Form được thiết kế logic và chặt chẽ, hỗ trợ Admin dễ dàng nhập tên, mô tả, giá, chọn danh mục và đặc biệt là tích hợp upload mảng ảnh trực tiếp lên Cloudinary. Phần quản lý tồn kho theo size (sizesStock) được trình bày khoa học để Admin nhập liệu chuẩn xác.

INFINITY ADMIN PANEL

Upload Image: Four 'Upload' buttons for image uploads.

Product name: Type here

Product description: Write content here

Product category: Men (dropdown)

Sub category: Topwear (dropdown)

Product Price: 250000

Product Quantity: 10

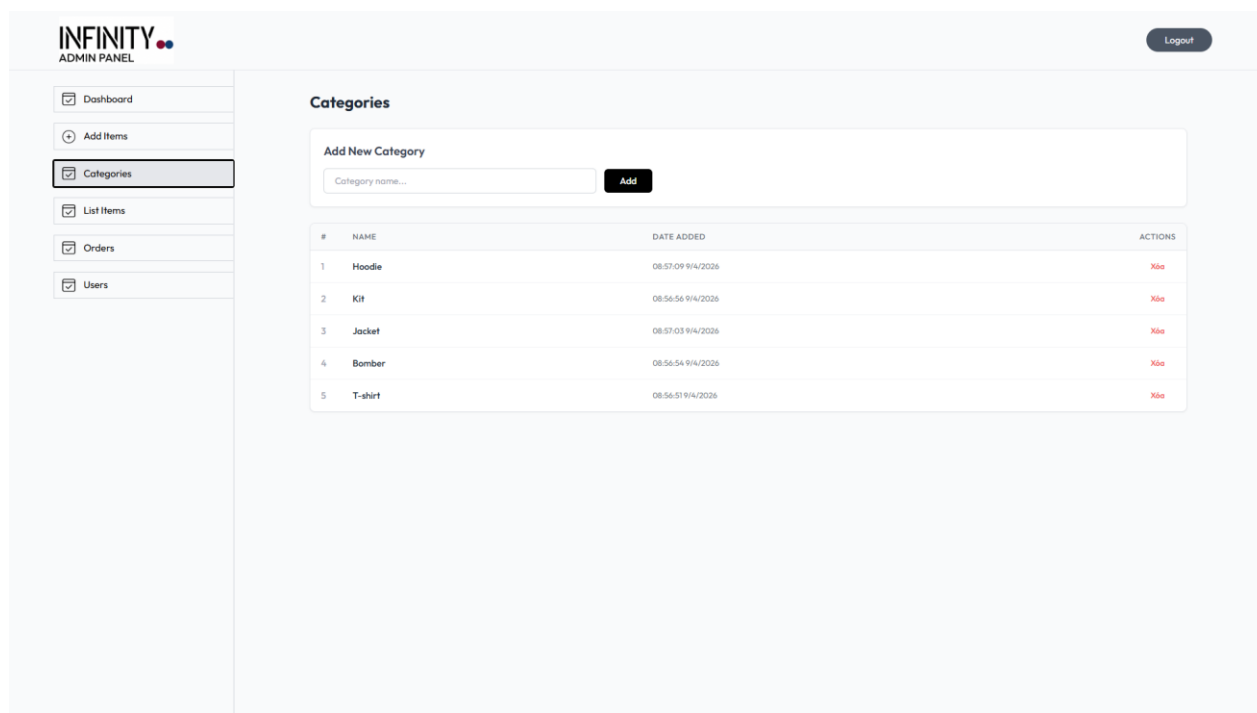
Product Sizes: S, M, L, XL, XXL (checkboxes)

Add to bestseller: ☐

Add: Button

Hình 31 - Giao diện thêm sản phẩm

- Trang Quản lý Danh mục (Categories): Cung cấp giao diện trực quan để Admin thực hiện nhanh các nghiệp vụ quản lý phân loại hàng hóa. Bố cục được thiết kế tinh gọn với Form tạo mới danh mục ở phía trên và Bảng dữ liệu (Data Table) quản lý danh sách ở phía dưới. Danh sách danh mục được sắp xếp theo doanh thu sản phẩm bán ra. Các thao tác Thêm/Xóa được cập nhật tức thời trên UI và được bảo vệ bởi cơ chế kiểm tra ràng buộc (Dependency Check) từ Backend, đảm bảo không thể xóa nhầm danh mục đang chứa sản phẩm.



- Trang Danh sách sản phẩm: Bảng quản lý tập hợp toàn bộ sản phẩm hiện có trên hệ thống. Hỗ trợ Admin tìm kiếm, lọc nhanh và đặc biệt là tích hợp các nút thao tác xóa/sửa sản phẩm ngay trên từng dòng, giúp quy trình quản lý danh mục hàng hóa trở nên nhanh chóng, gọn gàng.

INFINITY

ADMIN PANEL

Logout

Dashboard

Add Items

List Items

Orders

Users

All Products Lists

Image	Name	Category	Sizes	Price	Stock	Action
	T-shirt 1899 Ecu 'We are Culers'	Men	S M L XL	1.000.000đ	Tổng: 53 + Restock	X
	T-shirt beige Barça	Men	S M L XL	2.069.574đ	Tổng: 393 + Restock	X
	T-shirt 1899 Ecu 'We are Culers'	Men	S M L XL	1.356.920đ	Tổng: 407 + Restock	X
	Hoodie Navy Barça Essentials	Men	S M L XL	2.468.250đ	Tổng: 392 + Restock	X
	T-shirt Ecu Fan Club Barça	Men	S M L XL	1.589.740đ	Tổng: 398 + Restock	X
	Hoodie Grey Panaf Barça - Junior	Kids	S M L XL	2.237.804đ	Tổng: 400 + Restock	X
	T-shirt Orange Cruyff Barça	Men	S M L XL	2.530.978đ	Tổng: 400 + Restock	X
	Home Kit Jersey 1990-92 - Junior	Kids	S M L XL	1.412.568đ	Tổng: 400 + Restock	X

Hình 32 - Giao diện danh sách sản phẩm

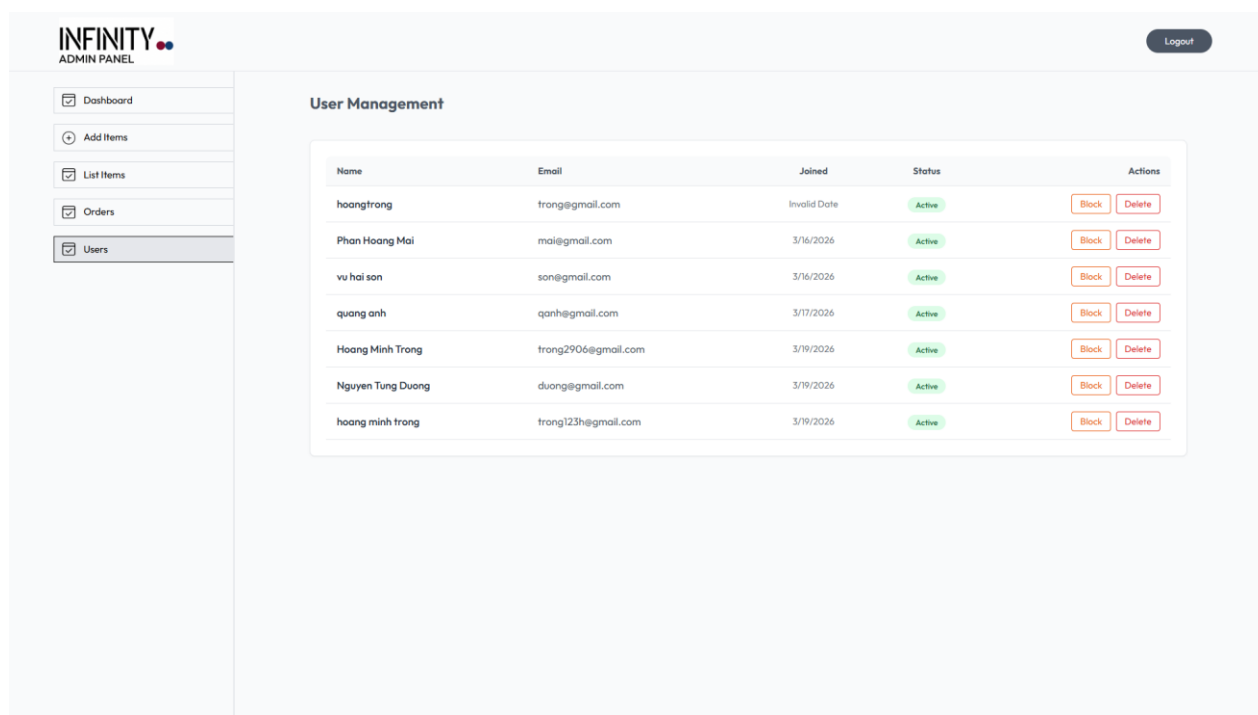
- Trang Quản lý Đơn hàng: Giao diện tập trung vào quản lý vòng đời đơn hàng. Bảng hiển thị rõ mã đơn, tên khách hàng, tổng tiền và phương thức thanh toán. Điểm quan trọng nhất là Admin có thể nhấp vào menu thả xuống (status) để cập nhật trạng thái vận chuyển cho từng giao dịch cụ thể.

INFINITY ADMIN PANEL		Logout					
Dashboard Add Items List Items Orders Users		Order Page					
	x 1 M , x 1 M Van A Nguyen 123 Đường ABC, Hồ Chí Minh, HCM 0901234567	Items : 2 Method : COD Payment : Pending Date : 3/19/2026, 8:37:31 AM	2.455.659đ	Shipped			
	T-shirt 1899 Ecu "We are Culers" x 1 L , Hoodie Navy Barça Essentials x 1 M , Hoodie Navy Barça Essentials x 1 L , T-shirt beige Barça x 1 L ok Hoàng số 4 ngõ 143 đường chiến thắng, Quận Hà Đông, Hà Nội 0888149225	Items : 4 Method : COD Payment : Pending Date : 3/19/2026, 12:27:33 AM	8.362.994đ	Out for delivery Order Placed Packing Shipped Out for delivery Delivered Cancelled			
	x 1 M Van A Nguyen 123 Đường ABC, Hồ Chí Minh, HCM 0901234567	Items : 1 Method : COD Payment : Pending Date : 3/19/2026, 12:38:32 AM	800.000đ	Delivered			
	testing name x 1 L Nguyen Van A 123 Đường ABC, TP.HCM, HCM 0901234567	Items : 1 Method : COD Payment : Pending Date : 3/17/2026, 4:53:20 PM	2.400.000đ	Order Placed			

Hình 33 - Giao diện quản lý đơn hàng

- Trang Quản lý Người dùng: Bảng dữ liệu trung tâm quản lý toàn bộ tài khoản khách hàng. Giao diện được tích hợp công cụ tìm kiếm và phân trang (Pagination) xử lý tối ưu tại Server-side. Admin có thể dễ dàng theo dõi chỉ số Tổng chi tiêu (Total Spent),

linh hoạt khóa/mở (Block/Active) tài khoản, hoặc thực hiện lệnh Xóa an toàn được bảo vệ bởi cơ chế *Cascade Delete* giúp tự động dọn dẹp sạch sẽ các dữ liệu liên đới.



Hình 34 - Giao diện quản lý người dùng

CHƯƠNG 4. TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Môi trường triển khai hệ thống

Để đảm bảo hệ thống thương mại điện tử INFINITY hoạt động ổn định và hiệu quả, quá trình xây dựng và triển khai được thực hiện trên môi trường với các thông số kỹ thuật chuẩn hóa như sau:

4.1.1 Môi trường phần cứng

- CPU: Intel Core i5 / AMD Ryzen 5 (hoặc tương đương) trở lên.
- RAM: Tối thiểu 8GB (Khuyến nghị 16GB để chạy mượt mà cả 3 máy chủ cục bộ cùng lúc).
- Ổ cứng: SSD trống tối thiểu 10GB.

4.1.2 Môi trường phần mềm

- Hệ điều hành: Windows 10/11, macOS hoặc Linux.
- Nền tảng thực thi: Node.js (Phiên bản v18.x.x LTS trở lên).
- Trình duyệt kiểm thử: Google Chrome (Sử dụng tích hợp React Developer Tools) và Safari để kiểm tra độ tương thích (Cross-browser).

4.1.3 Công cụ hỗ trợ

- Visual Studio Code, Antigravity: Trình soạn thảo mã nguồn chính.
- Postman: Công cụ giả lập Client để gọi và kiểm thử các RESTful API độc lập trước khi tích hợp vào Frontend.
- Git & GitHub: Hệ thống quản lý phiên bản mã nguồn, hỗ trợ lưu trữ và làm việc nhóm.
- MongoDB Atlas: Dịch vụ lưu trữ Cơ sở dữ liệu đám mây (Cloud Database) NoSQL.
- Cloudinary: Nền tảng CDN hỗ trợ lưu trữ và tối ưu hóa hình ảnh sản phẩm trực tuyến.

4.2 Triển khai hệ thống

Mã nguồn của hệ thống thương mại điện tử INFINITY được thiết kế theo kiến trúc phân tách rõ ràng thành 3 phân hệ độc lập: Backend (Máy chủ xử lý API), Frontend (Giao diện người dùng cuối) và Admin (Giao diện quản trị). Dưới đây là quy trình cấu hình và khởi chạy chi tiết cho toàn bộ hệ thống trên môi trường phát triển cục bộ (Localhost).

4.2.1 Cấu hình biến môi trường (.env)

Nguyên tắc bảo mật hàng đầu của hệ thống là tuyệt đối không đưa các thông tin nhạy cảm (Credentials) trực tiếp vào mã nguồn (Hard-code). Toàn bộ các khóa bí mật, chuỗi kết nối cơ sở dữ liệu và API Key được lưu trữ độc lập trong tệp tin .env tại thư mục gốc của phân hệ backend và admin.

Cấu trúc tệp .env tiêu chuẩn tại Backend:

- Cấu hình cổng máy chủ

PORT=4000

- Chuỗi kết nối cơ sở dữ liệu đám mây

MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/infinity

- Khóa bí mật dùng để mã hóa và giải mã JSON Web Token

JWT_SECRET=chuoi_khoa_bi_mat_de_tao_token

- Cấu hình dịch vụ lưu trữ hình ảnh Cloudinary

CLOUDINARY_NAME=ten_tai_khoan_cloudinary

CLOUDINARY_API_KEY=ma_api_key_cloudinary

CLOUDINARY_SECRET_KEY=ma_secret_key_cloudinary

4.2.2 Cấu hình Cơ sở dữ liệu đám mây (MongoDB Atlas)

Hệ thống sử dụng cơ sở dữ liệu phi quan hệ (NoSQL) được lưu trữ trên nền tảng điện toán đám mây MongoDB Atlas nhằm đảm bảo tính sẵn sàng cao và dễ dàng mở rộng, không phụ thuộc vào máy chủ vật lý.

- Khởi tạo Cluster (Cụm máy chủ) trên MongoDB Atlas và tạo một cơ sở dữ liệu mang tên infinity.
- Thiết lập Network Access (Bảo mật mạng). Cấu hình Whitelist IP thành 0.0.0.0/0 để cho phép các máy chủ Frontend và Admin có thể kết nối đến cơ sở dữ liệu từ bất kỳ địa chỉ IP nào trong quá trình phát triển.
- Khởi tạo Database User, cấp quyền đọc/ghi (Read/Write) và tích hợp chuỗi kết nối (Connection String) vào tệp .env.

4.2.3 Cấu hình Dịch vụ lưu trữ hình ảnh (Cloudinary)

Để tối ưu hóa tốc độ tải trang và giảm tải băng thông cho máy chủ Backend, hệ thống không lưu trữ file hình ảnh sản phẩm trực tiếp trong cơ sở dữ liệu hay bộ nhớ cục bộ. Thay vào đó, dự án tích hợp dịch vụ lưu trữ đám mây chuyên dụng Cloudinary.

Luồng thực thi: Khi Quản trị viên tải hình ảnh lên từ phân hệ Admin, Backend sẽ sử dụng SDK của Cloudinary và thư viện multer để đẩy trực tiếp file vật lý lên máy chủ Cloudinary. Cloudinary sau đó trả về một đường dẫn an toàn (Secure URL). Backend chỉ việc lưu trữ chuỗi URL này vào trường images (kiểu Array) của Collection products trong MongoDB.

Kết quả: Phương pháp này giúp ứng dụng Frontend render hình ảnh cực kỳ nhanh chóng thông qua mạng lưới CDN toàn cầu, đồng thời giữ cho kích thước Database luôn tối ưu.

4.2.4 Triển khai phân hệ Backend (Máy chủ API)

Backend đóng vai trò là trung tâm xử lý logic nghiệp vụ, giao tiếp với MongoDB và cung cấp các RESTful API cho các ứng dụng Client.

- Môi trường thực thi: Node.js và Express.js.
- Quy trình khởi chạy:
 - Mở Terminal, điều hướng vào thư mục chứa mã nguồn: `cd backend`
 - Cài đặt toàn bộ thư viện phụ thuộc (Dependencies) khai báo trong package.json: `npm install`
 - Khởi động máy chủ: `npm run server`
- Kết quả mong đợi: Terminal thông báo kết nối thành công với MongoDB và máy chủ bắt đầu lắng nghe các yêu cầu HTTP tại cổng `http://localhost:4000`.

4.2.5 Triển khai phân hệ Frontend (Giao diện Khách hàng)

Frontend là ứng dụng trang đơn (Single Page Application - SPA) được xây dựng bằng React.js, xử lý việc hiển thị sản phẩm, giỏ hàng và tương tác của người mua.

- Công cụ đóng gói: Vite (Giúp tối ưu tốc độ khởi động).
- Quy trình khởi chạy:
 - Mở Terminal mới, điều hướng vào thư mục Frontend: `cd frontend`
 - Cài đặt các gói thư viện giao diện (react-router-dom, axios,...): `npm install`
 - Khởi chạy máy chủ phát triển (Development Server): `npm run dev`
- Kết quả mong đợi: Mã nguồn biên dịch thành công, trình duyệt tự động mở giao diện trang chủ tại địa chỉ `http://localhost:5173`.

4.2.6 Triển khai phân hệ Admin (Giao diện Quản trị viên)

Phân hệ Admin là một SPA độc lập khác, được thiết kế với tính năng bảo mật nội bộ, chuyên dùng để quản lý sản phẩm, tồn kho và cập nhật trạng thái đơn hàng.

- Công cụ đóng gói: Vite (Giúp tối ưu tốc độ khởi động).
- Quy trình khởi chạy:
 - Mở Terminal thứ ba, điều hướng vào thư mục Admin: `cd admin`
 - Cài đặt các thư viện quản trị: `npm install`
 - Khởi chạy máy chủ phát triển: `npm run dev`
- Kết quả mong đợi: Giao diện đăng nhập của Ban quản trị xuất hiện trên trình duyệt tại địa chỉ <http://localhost:5174>.

4.3 Kiểm thử hệ thống (System Testing)

Quá trình kiểm thử hệ thống INFINITY được thực hiện theo phương pháp từ dưới lên (Bottom-up): Bắt đầu từ việc kiểm chứng độ chính xác của các API độc lập tại tầng Backend,

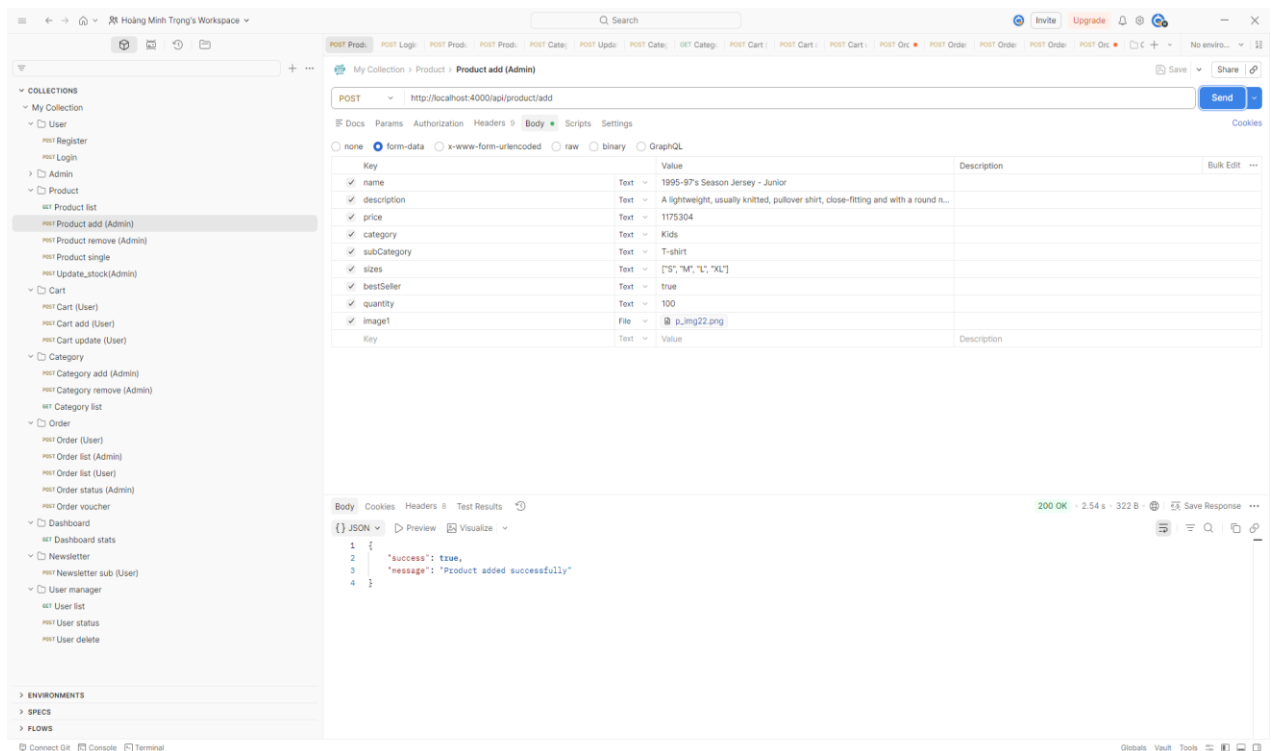
tiếp đến là kiểm thử luồng chức năng trên giao diện Frontend, và cuối cùng là đánh giá khả năng xử lý ngoại lệ của hệ thống.

4.3.1 Kiểm thử API (Kiểm thử độc lập với Postman)

Trước khi tích hợp dữ liệu lên giao diện, toàn bộ các RESTful API của hệ thống được kiểm thử độc lập bằng công cụ Postman. Mục tiêu là đảm bảo Backend nhận đúng tham số (Request Payload) và trả về dữ liệu (JSON Response) kèm mã trạng thái (Status Code) chuẩn xác.

Một số kịch bản kiểm thử API cốt lõi:

- API Đăng nhập (POST /api/user/login):
 - Dữ liệu gửi lên (Body): { "email": "admin@infinity.com", "password": "password123" }
 - Kết quả mong đợi: Hệ thống xác thực thành công, trả về Status: 200 OK, kèm theo thông tin User và chuỗi bảo mật token.
- API Lấy danh sách sản phẩm (GET /api/product/list):
 - Hành động: Gửi yêu cầu GET không mang theo token.
 - Kết quả mong đợi: Trả về Status: 200 OK cùng một mảng (Array) chứa toàn bộ object dữ liệu sản phẩm hiện có trong cơ sở dữ liệu MongoDB.
- Thực tế: Pass (Đạt).



Hình 35 - Ví dụ test thử API Product-Add bằng Postman

4.3.2 Kiểm thử chức năng (Functional Testing)

Hệ thống áp dụng phương pháp **Kiểm thử hộp đen (Black-box Testing)**, đóng vai trò là một người dùng thực thụ thao tác trực tiếp trên giao diện trình duyệt web (UI) để kiểm tra các luồng nghiệp vụ (Business Flows).

- Kịch bản TC-01: Thêm sản phẩm vào giỏ hàng
 - Hành động thực hiện (Input): Truy cập trang Chi tiết sản phẩm, tiến hành chọn kích cỡ (ví dụ: Size M) và bấm nút "Add to Cart".
 - Kết quả mong đợi (Expected Output): Số lượng trên icon giỏ hàng tăng lên. Dữ liệu sản phẩm hiển thị chính xác bên trong trang Giỏ hàng (đúng tên, size, giá tiền).
 - Đánh giá: Đạt (PASS) - Giao diện cập nhật mượt mà, State của giỏ hàng thay đổi chuẩn xác.
- Kịch bản TC-02: Thanh toán đơn hàng (Checkout)
 - Hành động thực hiện (Input): Tại trang Checkout, thao tác chọn địa chỉ thông qua Vietnam Provinces API, nhập mã giảm giá (VD: BARCA20) vào ô Voucher, chọn thanh toán COD và bấm "Place Order".
 - Kết quả mong đợi (Expected Output): Hệ thống gọi API hành chính mượt mà. Backend xác thực Voucher hợp lệ (chưa từng sử dụng) và tự động tính lại tổng tiền. Cơ sở dữ liệu ghi nhận đơn hàng mới, cập nhật mã Voucher vào hồ sơ User và quan trọng nhất: Số lượng tồn kho của sản phẩm phải bị trừ đi chính xác (bằng toán tử \$inc). Giỏ hàng được làm trống.
 - Đánh giá: Đạt (PASS) - Luồng thanh toán bảo mật hoạt động hoàn hảo, CSDL nhất quán tuyệt đối.
- Kịch bản TC-03: Quản trị viên thêm sản phẩm mới
 - Hành động thực hiện (Input): Đăng nhập tài khoản Admin, truy cập trang Thêm sản phẩm (Add Product). Điền đầy đủ form thông tin, tải ảnh lên từ máy tính và bấm nút "Add".
 - Kết quả mong đợi (Expected Output): Ảnh được upload thành công lên Cloudinary và trả về đường dẫn URL. Dữ liệu sản phẩm lưu vào Database. Sản phẩm mới xuất hiện ngay lập tức trên trang chủ dành cho Khách hàng.
 - Đánh giá: Đạt (PASS) - Tích hợp Cloudinary hoạt động ổn định, dữ liệu đồng bộ nhanh chóng lên giao diện người dùng.
- Kịch bản TC-04: Quản trị viên cập nhật trạng thái đơn hàng
 - Hành động thực hiện (Input): Tại trang Quản lý đơn hàng (Orders) của Admin, chọn một đơn hàng và đổi trạng thái sang "Cancelled".

- Kết quả mong đợi (Expected Output): Trạng thái đơn hàng cập nhật thành "Cancelled". Ngay lập tức, Trigger tự động ngầm của hệ thống được kích hoạt, cộng trả lại toàn bộ số lượng tồn kho của các sản phẩm trong đơn hàng đó về lại CSDL.
- Đánh giá: Đạt (PASS) - Cơ chế Cascade Restock (Tự động hoàn kho) hoạt động chính xác mà không cần Admin phải cộng tay lại từng sản phẩm.

4.3.3 Kiểm thử các trường hợp đặc biệt (Edge Cases)

Đây là bước đánh giá khả năng "chịu lỗi" (Fault Tolerance) và cơ chế phản hồi thông minh của hệ thống khi người dùng thao tác sai, nhằm tránh hiện tượng sập trang (Crash) hoặc gửi dữ liệu rác vào CSDL.

- Trường hợp 1: Thiếu dữ liệu bắt buộc khi mua hàng
 - Tình huống: Người dùng vội vàng bấm nút "Add to Cart" nhưng cố tình chưa chọn Kích cỡ (Size).
 - Xử lý của hệ thống: Giao diện chặn ngay thao tác gửi Request, đồng thời hiển thị thông báo bật lên (Toast Message) màu đỏ với nội dung "Select Product Size". Giỏ hàng không bị cộng dồn dữ liệu sai.
- Trường hợp 2: Xác thực sai thông tin (Invalid Credentials)
 - Tình huống: Nhập sai mật khẩu hoặc email không tồn tại tại trang Login.
 - Xử lý của hệ thống: Backend từ chối truy cập, trả về Status 400, Frontend bắt lỗi và thông báo cho người dùng "Invalid credentials". Không có Token nào được sinh ra.
- Trường hợp 3: Kiểm soát quyền truy cập (Route Protection)
 - Tình huống: Một người dùng không đăng nhập (không có Token) nhưng cố tình gõ URL chuyển thẳng vào trang /cart hoặc trang Admin.
 - Xử lý của hệ thống: Hệ thống điều hướng (Router) phát hiện không có quyền, tự động đẩy người dùng vắng ra ngoài trang Đăng nhập để đảm bảo an toàn dữ liệu nội bộ.
- Trường hợp 4: Ràng buộc bảo toàn dữ liệu (Dependency Check) khi Xóa
 - Tình huống: Admin vào trang Danh mục (Categories), cố tình bấm Xóa một danh mục (Ví dụ: "Hoodie") đang chứa rất nhiều sản phẩm bên trong.
 - Xử lý của hệ thống: Backend thực hiện quét ràng buộc và chặn đứng lệnh xóa. Trả về mã lỗi HTTP 400 Bad Request kèm thông báo: "Không thể xóa danh mục này vì vẫn còn sản phẩm đang sử dụng". Bản ghi danh mục vẫn được giữ nguyên an toàn.
 - Đánh giá: Đạt (PASS) - Hệ thống chống chặn thành công việc tạo ra "dữ liệu mồ côi" (Orphan data), bảo vệ tính toàn vẹn của CSDL.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trải qua quá trình nghiên cứu, phân tích và trực tiếp xây dựng mã nguồn, đồ án đã hoàn thành mục tiêu ban đầu là phát triển thành công hệ thống thương mại điện tử INFINITY dựa trên kiến trúc công nghệ hiện đại MERN Stack (MongoDB, Express.js, React.js, Node.js). Hệ thống đã đáp ứng tốt các yêu cầu nghiệp vụ khắt khe của một website bán hàng trực tuyến, cụ thể:

- Về mặt công nghệ: Ứng dụng thành công mô hình Single Page Application (SPA) với React Router giúp điều hướng mượt mà. Xây dựng hệ thống RESTful API chuẩn mực, bảo mật cao với JSON Web Token (JWT). Khai thác tối đa sức mạnh của cơ sở dữ liệu NoSQL với Aggregation Framework (tính toán doanh thu động, phân trang server-side) và bảo vệ tính toàn vẹn dữ liệu bằng các Trigger tự động (Cascade Delete, Tự động hoàn kho bằng toán tử \$inc). Tích hợp thành công các API bên thứ ba như Cloudinary (lưu trữ ảnh) và Vietnam Provinces API (dữ liệu hành chính).
- Về mặt chức năng Khách hàng: Hoàn thiện luồng mua sắm liền mạch từ khâu xem danh mục, lọc sản phẩm, quản lý giỏ hàng cho đến bước thanh toán (Checkout) xử lý giá hoàn toàn tại Server-side và theo dõi tiến trình đơn hàng.
- Về mặt chức năng Quản trị (Admin): Xây dựng trang Admin Panel độc lập, cung cấp Dashboard trực quan (Recharts). Giúp ban quản trị dễ dàng thực hiện các thao tác thêm mới sản phẩm, kiểm soát số lượng tồn kho nguyên tử theo từng biến thể kích cỡ và xử lý trạng thái đơn hàng.
- Về mặt giao diện (UI/UX): Thiết kế giao diện hiện đại, sang trọng theo phong cách Minimalism, tuân thủ nguyên tắc Responsive Web Design bằng Tailwind CSS, đảm bảo trải nghiệm mua sắm mượt mà trên đa nền tảng (Desktop, Tablet, Mobile).

5.2 Hạn chế của hệ thống

Bên cạnh những kết quả tích cực đã đạt được, do giới hạn về mặt thời gian và tài nguyên máy chủ, hệ thống INFINITY vẫn còn tồn tại một số điểm hạn chế mang tính thực tế:

- Hạn chế về tính năng: Hệ thống chưa tích hợp chức năng Đánh giá và Bình luận (Review & Rating), làm giảm kênh tương tác hai chiều với khách hàng. Bộ lọc sản phẩm hiện tại vẫn ở mức cơ bản, chưa hỗ trợ lọc sâu theo khoảng giá động (Price slider) hoặc màu sắc trực quan.
- Hạn chế về thanh toán: Hiện tại hệ thống mới chỉ xử lý tron tru phương thức thanh toán khi nhận hàng (COD). Việc tích hợp trực tiếp các cổng thanh toán/ví điện tử phổ biến tại Việt Nam (như VNPay, MoMo) vẫn chưa được triển khai hoàn thiện.
- Hạn chế về bảo mật và quy mô: Cơ chế định danh người dùng chưa có bước xác minh OTP qua Email (Email Verification) hoặc khôi phục mật khẩu (Forgot Password). Ngoài ra, dù đã tối ưu truy vấn bằng Aggregation, nhưng khi dữ liệu mở rộng lên

mức hàng triệu bản ghi, hệ thống vẫn thiếu cơ chế Bộ nhớ đệm (Caching) ở tầng trung gian để giảm tải cho Database.

5.3 Hướng phát triển trong tương lai

Dựa trên những hạn chế đã phân tích, phương hướng phát triển và nâng cấp hệ thống INFINITY trong giai đoạn tiếp theo sẽ tập trung vào các hạng mục chuyên sâu:

- Nâng cấp Trải nghiệm người dùng: Tích hợp tính năng Đánh giá có kèm hình ảnh thực tế. Nghiên cứu và áp dụng các thuật toán Machine Learning cơ bản để xây dựng hệ thống Gợi ý sản phẩm (Recommendation System) dựa trên lịch sử mua hàng. Bổ sung đăng nhập bằng Mạng xã hội (OAuth 2.0 với Google/Facebook).
- Mở rộng Hệ sinh thái Dịch vụ: Kết nối webhook với các cổng thanh toán trực tuyến nội địa (VNPay, MoMo) và tích hợp API của các đối tác vận chuyển (Giao Hàng Nhanh, Viettel Post) để tự động hóa khâu tính phí ship và cập nhật mã vận đơn (Tracking) theo thời gian thực.
- Tối ưu hóa Hiệu năng và Kiến trúc: Triển khai hệ thống Redis In-memory Database để làm bộ nhớ đệm (Caching) cho các dữ liệu ít thay đổi (Danh mục, Top sản phẩm bán chạy). Tích hợp Elasticsearch để nâng cao tốc độ tìm kiếm.
- Phát triển Ứng dụng Di động: Tận dụng lại toàn bộ hệ thống Backend API hiện tại (Code reusability) để xây dựng thêm phiên bản Mobile App dành riêng cho iOS và Android bằng framework React Native.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Duy Phương (2012), *Giáo trình Cơ sở dữ liệu*, Học viện Công nghệ Bưu chính Viễn thông.
- [2] Trần Đình Quế, Nguyễn Mạnh Hùng (2015), *Giáo trình Phân tích và Thiết kế hệ thống thông tin*, Học viện Công nghệ Bưu chính Viễn thông.
- [3] Trịnh Nhật Tiến, Nguyễn Văn Vy (2014), *Giáo trình Công nghệ phần mềm*, Nhà xuất bản Đại học Quốc gia Hà Nội.
- [4] Vasan Subramanian (2019), *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node* (2nd Edition), Nhà xuất bản Apress.
- [5] Alex Banks, Eve Porcello (2020), *Learning React: Modern Patterns for Developing React Apps*, Nhà xuất bản O'Reilly Media.
- [6] Mario Casciaro, Luciano Mammino (2020), *Node.js Design Patterns* (3rd Edition), Nhà xuất bản Packt Publishing.
- [7] Meta Open Source (2024), *React Official Documentation - The library for web and native user interfaces*, Tài liệu hướng dẫn kỹ thuật chính thức.
- [8] OpenJS Foundation (2024), *Node.js and Express.js Documentation*, Tài liệu kỹ thuật nền tảng.
- [9] MongoDB Inc. (2024), *MongoDB Official Documentation - The Developer Data Platform*, Tài liệu quản trị cơ sở dữ liệu.
- [10] Tailwind Labs (2024), *Tailwind CSS Core Documentation*, Tài liệu thiết kế giao diện.
- [11] Cloudinary (2024), *Cloudinary Image and Video Management*, Tài liệu tích hợp dịch vụ lưu trữ đám mây.
- [12] IETF - Internet Engineering Task Force (2015), *RFC 7519: JSON Web Token (JWT)*, Tài liệu tiêu chuẩn bảo mật mạng.
- [13] Brad Traversy (2023), *MERN eCommerce From Scratch*, Tài liệu thực hành dự án mã nguồn mở.

PHỤ LỤC CÀI ĐẶT VÀ TRIỂN KHAI

1. Thiết lập môi trường

Để hệ thống INFINITY có thể khởi chạy thành công, môi trường máy chủ cục bộ (Localhost) cần đáp ứng các yêu cầu sau:

- Node.js: Phiên bản v18.x.x LTS trở lên.
- Git: Cài đặt Git Bash để tải mã nguồn.
- Tài khoản dịch vụ (Bắt buộc):
 - Tài khoản MongoDB Atlas (để lấy chuỗi kết nối MONGO_URI).
 - Tài khoản Cloudinary (để quản lý hình ảnh với API_KEY, SECRET_KEY).

2. Cài đặt và khởi chạy hệ thống

Bước 1: Tải mã nguồn và cấu hình

- Tải mã nguồn: git clone <https://github.com/Trongpro296/infinity-mern-ecommerce-ttcs>
- Cấu hình .env: Tạo file .env tại phân hệ backend và admin, điền các thông số bảo mật tương ứng.

Bước 2: Khởi chạy các phân hệ Mở 3 cửa sổ Terminal độc lập và chạy các lệnh sau:

- Backend: cd backend -> npm install -> npm run server (Chạy tại cổng 4000).
- Frontend: cd frontend -> npm install -> npm run dev (Chạy tại cổng 5173).
- Admin: cd admin -> npm install -> npm run dev (Chạy tại cổng 5174).

3. Hướng dẫn trải nghiệm thực tế

Để quan sát trực quan nhất toàn bộ luồng nghiệp vụ và giao diện của hệ thống thay vì các hình ảnh tĩnh, kính mời Hội đồng thao tác trực tiếp thông qua các tài khoản kiểm thử hoặc theo dõi Video Demo:

- Tài khoản trải nghiệm Admin:
 - Link truy cập: <http://localhost:5174>
 - Email: admin@infinity.com
 - Mật khẩu: trong123
- Tài khoản trải nghiệm Khách hàng:
 - Link truy cập: <http://localhost:5173>
 - Email: trong@gmail.com
 - Mật khẩu: 12345678

CÁC SẢN PHẨM KÈM THEO

1. Source code hệ thống

- Kho lưu trữ: GitHub Repository
- Đường dẫn truy cập: <https://github.com/Trongpro296/infinity-mern-ecommerce-ttcs>
- Bao gồm toàn bộ mã nguồn (Frontend, Backend, Admin) và hướng dẫn cài đặt.

2. File thiết kế tương tác (UI/UX)

- Nền tảng: Figma
- Đường dẫn truy cập:
 - User:
<https://www.figma.com/design/Cw6la5MAhESEIMAyMkrJni/User?node-id=0-1&t=Y1zexIELVb2FdRTn-1>
 - Admin:
<https://www.figma.com/design/WHgG5Vv5cUxsoT1FNz6fJX/Admin?node-id=0-1&t=v46LbO97pYhubReZ-1>
- Mô tả giao diện và luồng tương tác người dùng.

3. File phân tích thiết kế UML và Sơ đồ cơ sở dữ liệu:

- Nền tảng: Lưu trữ trực tuyến cùng mã nguồn trên GitHub.
- Thư mục đính kèm truy cập chi tiết tại: <https://github.com/Trongpro296/infinity-mern-ecommerce-ttcs/tree/main/docs/diagrams>

4. Slide thuyết trình và Quyền báo cáo

- Slide thuyết trình:
https://www.canva.com/design/DAHFK_Tyow4/2Q16YzeocaDSWwkaxuJ4ag/edit?utm_content=DAHFK_Tyow4&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton
- Quyền báo cáo:

5. Video demo sản phẩm (Video Walkthrough)

- Nền tảng lưu trữ: Google Drive
- Đường dẫn truy cập: [here]
- Nội dung: Video quay lại cảnh thao tác chạy sản phẩm thực tế từ phía người dùng (Client) và luồng quản lý của Ban quản trị (Admin).

